

1

SECURITY IN THE WORLD OF WEB APPLICATIONS

To provide proper context for the technical discussions later in the book, it seems prudent to first of all explain what the field of security engineering tries to achieve and then to outline why, in this otherwise well-studied context, web applications deserve special treatment. So, shall we?

Information Security in a Nutshell

On the face of it, the field of information security appears to be a mature, well-defined, and accomplished branch of computer science. Resident experts eagerly assert the importance of their area of expertise by pointing to large sets of neatly cataloged security flaws, invariably attributed to security-illiterate developers, while their fellow theoreticians note how all these problems would have been prevented by adhering to this year's hottest security methodology.

A commercial industry thrives in the vicinity, offering various nonbinding security assurances to everyone, from casual computer users to giant international corporations.

Yet, for several decades, we have in essence completely failed to come up with even the most rudimentary usable frameworks for understanding and assessing the security of modern software. Save for several brilliant treatises and limited-scale experiments, we do not even have any real-world success stories to share. The focus is almost exclusively on reactive, secondary security measures (such as vulnerability management, malware and attack detection, sandboxing, and so forth) and perhaps on selectively pointing out flaws in somebody else's code. The frustrating, jealously guarded secret is that when it comes to enabling others to develop secure systems, we deliver far less value than should be expected; the modern Web is no exception.

Let's look at some of the most alluring approaches to ensuring information security and try to figure out why they have not made a difference so far.

Flirting with Formal Solutions

Perhaps the most obvious tool for building secure programs is to algorithmically prove they behave just the right way. This is a simple premise that intuitively should be within the realm of possibility—so why hasn't this approach netted us much?

Well, let's start with the adjective *secure* itself: What is it supposed to convey, precisely? Security seems like an intuitive concept, but in the world of computing, it escapes all attempts to usefully define it. Sure, we can restate the problem in catchy yet largely unhelpful ways, but you know there's a problem when one of the definitions most frequently cited by practitioners^{*} is this:

A system is secure if it behaves precisely in the manner intended—and does nothing more.

This definition is neat and vaguely outlines an abstract goal, but it tells very little about how to achieve it. It's computer science, but in terms of specificity, it bears a striking resemblance to a poem by Victor Hugo:

Love is a portion of the soul itself, and it is of the same nature as the celestial breathing of the atmosphere of paradise.

One could argue that practitioners are not the ones to be asked for nuanced definitions, but go ahead and pose the same question to a group of academics and they'll offer you roughly the same answer. For example, the following common academic definition traces back to the Bell-La Padula security model, published in the 1960s. (This was one of about a dozen attempts to formalize the requirements for secure systems, in this case in terms of a finite state machine;¹ it is also one of the most notable ones.)

A system is secure if and only if it starts in a secure state and cannot enter an insecure state.

^{*}The quote is attributed originally to Ivan Arce, a renowned vulnerability hunter, circa 2000; since then, it has been used by Crispin Cowan, Michael Howard, Anton Chuvakin, and scores of other security experts.

Definitions along these lines are fundamentally true, of course, and may serve as the basis for dissertations or even a couple of government grants. But in practice, models built on these foundations are bound to be nearly useless for generalized, real-world software engineering for at least three reasons:

- **There is no way to define desirable behavior for a sufficiently complex computer system.** No single authority can define what the “intended manner” or “secure states” should be for an operating system or a web browser. The interests of users, system owners, data providers, business process owners, and software and hardware vendors tend to differ significantly and shift rapidly—when the stakeholders are capable and willing to clearly and honestly disclose their interests to begin with. To add insult to injury, sociology and game theory suggest that computing a simple sum of these particular interests may not actually result in a beneficial outcome. This dilemma, known as “the tragedy of the commons,” is central to many disputes over the future of the Internet.
- **Wishful thinking does not automatically map to formal constraints.** Even if we can reach a perfect, high-level agreement about how the system should behave in a subset of cases, it is nearly impossible to formalize such expectations as a set of permissible inputs, program states, and state transitions, which is a prerequisite for almost every type of formal analysis. Quite simply, intuitive concepts such as “I do not want my mail to be read by others,” do not translate to mathematical models particularly well. Several exotic approaches will allow such vague requirements to be at least partly formalized, but they put heavy constraints on software-engineering processes and often result in rulesets and models that are far more complicated than the validated algorithms themselves. And, in turn, they are likely to need their own correctness to be proven . . . *ad infinitum*.
- **Software behavior is very hard to conclusively analyze.** Static analysis of computer programs with the intent to prove that they will always behave according to a detailed specification is a task that no one has managed to believably demonstrate in complex, real-world scenarios (though, as you might expect, limited success in highly constrained settings or with very narrow goals is possible). Many cases are likely to be impossible to solve in practice (due to computational complexity) and may even turn out to be completely undecidable due to the halting problem.*

Perhaps more frustrating than the vagueness and uselessness of the early definitions is that as the decades have passed, little or no progress has been made toward something better. In fact, an academic paper released in 2001 by the Naval Research Laboratory backtracks on some of the earlier work and arrives at a much more casual, enumerative definition of software security—one that explicitly disclaims its imperfection and incompleteness.²

* In 1936, Alan Turing showed that (paraphrasing slightly) it is not possible to devise an algorithm that can generally decide the outcome of other algorithms. Naturally, some algorithms are very much decidable by conducting case-specific proofs, just not all of them.

A system is secure if it adequately protects information that it processes against unauthorized disclosure, unauthorized modification, and unauthorized withholding (also called denial of service). We say “adequately” because no practical system can achieve these goals without qualification; security is inherently relative.

The paper also provides a retrospective assessment of earlier efforts and the unacceptable sacrifices made to preserve the theoretical purity of said models:

Experience has shown that, on one hand, the axioms of the Bell-La Padula model are overly restrictive: they disallow operations that users require in practical applications. On the other hand, trusted subjects, which are the mechanism provided to overcome some of these restrictions, are not restricted enough. . . . Consequently, developers have had to develop ad hoc specifications for the desired behavior of trusted processes in each individual system.

In the end, regardless of the number of elegant, competing models introduced, all attempts to understand and evaluate the security of real-world software using algorithmic foundations seem bound to fail. This leaves developers and security experts with no method to make authoritative, future-looking statements about the quality of produced code. So, what other options are on the table?

Enter Risk Management

In the absence of formal assurances and provable metrics, and given the frightening prevalence of security flaws in key software relied upon by modern societies, businesses flock to another catchy concept: *risk management*.

The idea of risk management, applied successfully to the insurance business (with perhaps a bit less success in the financial world), simply states that system owners should learn to live with vulnerabilities that cannot be addressed in a cost-effective way and, in general, should scale efforts according to the following formula:

$$\text{risk} = \text{probability of an event} \times \text{maximum loss}$$

For example, according to this doctrine, if having some unimportant workstation compromised yearly won’t cost the company more than \$1,000 in lost productivity, the organization should just budget for this loss and move on, rather than spend say \$100,000 on additional security measures or contingency and monitoring plans to prevent the loss. According to the doctrine of risk management, the money would be better spent on isolating, securing, and monitoring the mission-critical mainframe that churns out billing records for all customers.

Naturally, it's prudent to prioritize security efforts. The problem is that when risk management is done strictly by the numbers, it does little to help us to understand, contain, and manage real-world problems. Instead, it introduces a dangerous fallacy: that structured inadequacy is almost as good as adequacy and that underfunded security efforts *plus* risk management are about as good as properly funded security work.

Guess what? No dice.

- **In interconnected systems, losses are not capped and are not tied to an asset.** Strict risk management depends on the ability to estimate typical and maximum cost associated with the compromise of a resource. Unfortunately, the only way to do this is to overlook the fact that many of the most spectacular security breaches—such as the attacks on TJX^{*} or Microsoft[†]—began at relatively unimportant and neglected entry points. These initial intrusions soon escalated and eventually resulted in the nearly complete compromise of critical infrastructure, bypassing any superficial network compartmentalization on their way. In typical by-the-numbers risk management, the initial entry point is assigned a lower weight because it has a low value when compared to other nodes. Likewise, the internal escalation path to more sensitive resources is downplayed as having a low probability of ever being abused. Still, neglecting them both proves to be an explosive mix.
- **The nonmonetary costs of intrusions are hard to offset with the value contributed by healthy systems.** Loss of user confidence and business continuity, as well as the prospect of lawsuits and the risk of regulatory scrutiny, are difficult to meaningfully insure against. These effects can, at least in principle, make or break companies or even entire industries, and any superficial valuations of such outcomes are almost purely speculative.
- **Existing data is probably not representative of future risks.** Unlike the participants in a fender bender, attackers will not step forward to helpfully report break-ins and will not exhaustively document the damage caused. Unless the intrusion is painfully evident (due to the attacker's sloppiness or disruptive intent), it will often go unnoticed. Even though industry-wide, self-reported data may be available, there is simply no reliable way of telling how complete it is or how much extra risk one's current business practice may be contributing.

^{*} Sometime in 2006, several intruders, allegedly led by Albert Gonzalez, attacked an unsecured wireless network at a retail location and subsequently made their way through the corporate networks of the retail giant. They copied the credit card data of about 46 million customers and the Social Security numbers, home addresses, and so forth of about 450,000 more. Eleven people were charged in connection with the attack, one of whom committed suicide.

[†] Microsoft's formally unpublished and blandly titled presentation *Threats Against and Protection of Microsoft's Internal Network* outlines a 2003 attack that began with the compromise of an engineer's home workstation that enjoyed a long-lived VPN session to the inside of the corporation. Methodical escalation attempts followed, culminating with the attacker gaining access to, and leaking data from, internal source code repositories. At least to the general public, the perpetrator remains unknown.

- **Statistical forecasting is not a robust predictor of individual outcomes.** Simply because on average people in cities are more likely to be hit by lightning than mauled by a bear does not mean you should bolt a lightning rod to your hat and then bathe in honey. The likelihood that a compromise will be associated with a particular component is, on an individual scale, largely irrelevant: Security incidents are nearly certain, but out of thousands of exposed nontrivial resources, any service can be used as an attack vector—and no one service is likely to see a volume of events that would make statistical forecasting meaningful within the scope of a single enterprise.

Enlightenment Through Taxonomy

The two schools of thought discussed above share something in common: Both assume that it is possible to define security as a set of computable goals and that the resulting unified theory of a secure system or a model of acceptable risk would then elegantly trickle down, resulting in an optimal set of low-level actions needed to achieve perfection in application design.

Some practitioners preach the opposite approach, which owes less to philosophy and more to the natural sciences. These practitioners argue that, much like Charles Darwin of the information age, by gathering sufficient amounts of low-level, experimental data, we will be able to observe, reconstruct, and document increasingly more sophisticated laws in order to arrive some sort of a unified model of secure computing.

This latter worldview brings us projects like the Department of Homeland Security–funded Common Weakness Enumeration (CWE), the goal of which, in the organization’s own words, is to develop a unified “Vulnerability Theory”; “improve the research, modeling, and classification of software flaws”; and “provide a common language of discourse for discussing, finding and dealing with the causes of software security vulnerabilities.” A typical, delightfully baroque example of the resulting taxonomy may be this:

Improper Enforcement of Message or Data Structure

Failure to Sanitize Data into a Different Plane

Improper Control of Resource Identifiers

Insufficient Filtering of File and Other Resource Names
for Executable Content

Today, there are about 800 names in the CWE dictionary, most of which are as discourse-enabling as the one quoted here.

A slightly different school of naturalist thought is manifested in projects such as the Common Vulnerability Scoring System (CVSS), a business-backed collaboration that aims to strictly quantify known security problems in terms of a set of basic, machine-readable parameters. A real-world example of the resulting vulnerability descriptor may be this:

AV:LN / AC:L / Au:M / C:C / I:N / A:P / E:F / RL:T / RC:UR /
CDP:MH / TD:H / CR:M / IR:L / AR:M

Organizations and researchers are expected to transform this 14-dimensional vector in a carefully chosen, use-specific way in order to arrive at some sort of objective, verifiable, numerical conclusion about the significance of the underlying bug (say, “42”), precluding the need to judge the nature of security flaws in any more subjective fashion.

Yes, I am poking gentle fun at the expense of these projects, but I do not mean to belittle their effort. CWE, CVSS, and related projects serve noble goals, such as bringing a more manageable dimension to certain security processes implemented by large organizations. Still, none has yielded a grand theory of secure software, and I doubt such a framework is within sight.

Toward Practical Approaches

All signs point to security being largely a nonalgorithmic problem for now. The industry is understandably reluctant to openly embrace this notion, because it implies that there are no silver bullet solutions to preach (or better yet, commercialize); still, when pressed hard enough, eventually everybody in the security field falls back to a set of rudimentary, empirical recipes. These recipes are deeply incompatible with many business management models, but they are all that have really worked for us so far. They are as follows:

- **Learning from (preferably other people’s) mistakes.** Systems should be designed to prevent known classes of bugs. In the absence of automatic (or even just elegant) solutions, this goal is best achieved by providing ongoing design guidance, ensuring that developers know what could go wrong, and giving them the tools to carry out otherwise error-prone tasks in the simplest manner possible.
- **Developing tools to detect and correct problems.** Security deficiencies typically have no obvious side effects until they’re discovered by a malicious party: a pretty costly feedback loop. To counter this problem, we create security quality assurance (QA) tools to validate implementations and perform audits periodically to detect casual mistakes (or systemic engineering deficiencies).
- **Planning to have everything compromised.** History teaches us that major incidents will occur despite our best efforts to prevent them. It is important to implement adequate component separation, access control, data redundancy, monitoring, and response procedures so that service owners can react to incidents before an initially minor hiccup becomes a disaster of biblical proportions.

In all cases, a substantial dose of patience, creativity, and real technical expertise is required from all the information security staff.

Naturally, even such simple, commonsense rules—essentially basic engineering rigor—are often dressed up in catchphrases, sprinkled liberally with a selection of acronyms (such as *CIA: confidentiality, integrity, availability*), and then called “methodologies.” Frequently, these methodologies are thinly veiled attempts to pass off one of the most frustrating failures of the security industry as yet another success story and, in the end, sell another cure-all

product or certification to gullible customers. But despite claims to the contrary, such products are no substitute for street smarts and technical prowess—at least not today.

In any case, through the remainder of this book, I will shy away from attempts to establish or reuse any of the aforementioned grand philosophical frameworks and settle for a healthy dose of anti-intellectualism instead. I will review the exposed surface of modern browsers, discuss how to use the available tools safely, which bits of the Web are commonly misunderstood, and how to control collateral damage when things go boom.

And that is, pretty much, the best take on security engineering that I can think of.

A Brief History of the Web

The Web has been plagued by a perplexing number, and a remarkable variety, of security issues. Certainly, some of these problems can be attributed to one-off glitches in specific client or server implementations, but many are due to capricious, often arbitrary design decisions that govern how the essential mechanisms operate and mesh together on the browser end.

Our empire is built on shaky foundations—but why? Perhaps due to simple shortsightedness: After all, back in the innocent days, who could predict the perils of contemporary networking and the economic incentives behind today's large-scale security attacks?

Unfortunately, while this explanation makes sense for truly ancient mechanisms such as SMTP or DNS, it does not quite hold water here: The Web is relatively young and took its current shape in a setting not that different from what we see today. Instead, the key to this riddle probably lies in the tumultuous and unusual way in which the associated technologies have evolved.

So, pardon me another brief detour as we return to the roots. The pre-history of the Web is fairly mundane but still worth a closer look.

Tales of the Stone Age: 1945 to 1994

Computer historians frequently cite a hypothetical desk-sized device called the Memex as one of the earliest fossil records, postulated in 1945 by Vannevar Bush.³ Memex was meant to make it possible to create, annotate, and follow cross-document links in microfilm, using a technique that vaguely resembled modern-day bookmarks and hyperlinks. Bush boldly speculated that this simple capability would revolutionize the field of knowledge management and data retrieval (amusingly, a claim still occasionally ridiculed as uneducated and naïve until the early 1990s). Alas, any useful implementation of the design was out of reach at that time, so, beyond futuristic visions, nothing much happened until transistor-based computers took center stage.

The next tangible milestone, in the 1960s, was the arrival of IBM's Generalized Markup Language (GML), which allowed for the annotation of documents with machine-readable directives indicating the function of each block of text, effectively saying “this is a header,” “this is a numbered list of items,” and so on. Over the next 20 years or so, GML (originally used by only

a handful of IBM text editors on bulky mainframe computers) became the foundation for Standard Generalized Markup Language (SGML), a more universal and flexible language that traded an awkward colon- and period-based syntax for a familiar angle-bracketed one.

While GML was developing into SGML, computers were growing more powerful and user friendly. Several researchers began experimenting with Bush's cross-link concept, applying it to computer-based document storage and retrieval, in an effort to determine whether it would be possible to cross-reference large sets of documents based on some sort of key. Adventurous companies and universities pursued pioneering projects such as ENQUIRE, NLS, and Xanadu, but most failed to make a lasting impact. Some common complaints about the various projects revolved around their limited practical usability, excess complexity, and poor scalability.

By the end of the decade, two researchers, Tim Berners-Lee and Dan Connolly, had begun working on a new approach to the cross-domain reference challenge—one that focused on simplicity. They kicked off the project by drafting HyperText Markup Language (HTML), a bare-bones descendant of SGML, designed specifically for annotating documents with hyperlinks and basic formatting. They followed their work on HTML with the development of HyperText Transfer Protocol (HTTP), an extremely basic, dedicated scheme for accessing HTML resources using the existing concepts of Internet Protocol (IP) addresses, domain names, and file paths. The culmination of their work, sometime between 1991 and 1993, was Tim Berners-Lee's World Wide Web (Figure 1-1), a rudimentary browser that parsed HTML and allowed users to render the resulting data on the screen, and then navigate from one page to another with a mouse click.

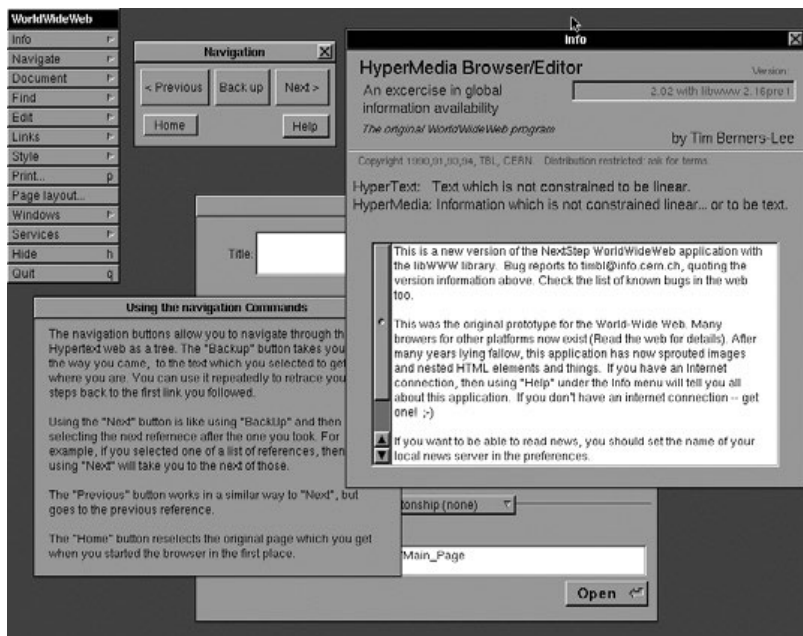


Figure 1-1: Tim Berners-Lee's World Wide Web

To many people, the design of HTTP and HTML must have seemed a significant regression from the loftier goals of competing projects. After all, many of the earlier efforts boasted database integration, security and digital rights management, or cooperative editing and publishing; in fact, even Berners-Lee's own project, ENQUIRE, appeared more ambitious than his current work. Yet, because of its low entry requirements, immediate usability, and unconstrained scalability (which happened to coincide with the arrival of powerful and affordable computers and the expansion of the Internet), the unassuming WWW project turned out to be a sudden hit.

All right, all right, it turned out to be a "hit" by the standards of the mid-1990s. Soon, there were no fewer than dozens of web servers running on the Internet. By 1993, HTTP traffic accounted for 0.1 percent of all bandwidth in the National Science Foundation backbone network. The same year also witnessed the arrival of Mosaic, the first reasonably popular and sophisticated web browser, developed at the University of Illinois. Mosaic extended the original World Wide Web code by adding features such as the ability to embed images in HTML documents and submit user data through forms, thus paving the way for the interactive, multimedia applications of today.

Mosaic made browsing prettier, helping drive consumer adoption of the Web. And through the mid-1990s, it served as the foundation for two other browsers: Mosaic Netscape (later renamed Netscape Navigator) and Spyglass Mosaic (ultimately acquired by Microsoft and renamed Internet Explorer). A handful of competing non-Mosaic engines emerged as well, including Opera and several text-based browsers (such as Lynx and w3m). The first search engines, online newspapers, and dating sites followed soon after.

The First Browser Wars: 1995 to 1999

By the mid-1990s, it was clear that the Web was here to stay and that users were willing to ditch many older technologies in favor of the new contender. Around that time, Microsoft, the desktop software behemoth that had been slow to embrace the Internet before, became uncomfortable and began to allocate substantial engineering resources to its own browser, eventually bundling it with the Windows operating system in 1996.* Microsoft's actions sparked a period colloquially known as the "browser wars."

The resulting arms race among browser vendors was characterized by the remarkably rapid development and deployment of new features in the competing products, a trend that often defied all attempts to standardize or even properly document all the newly added code. Core HTML tweaks ranged from the silly (the ability to make text blink, a Netscape invention that became the butt of jokes and a telltale sign of misguided web design) to notable ones, such as the ability to change typefaces or embed external documents in so-called frames. Vendors released their products with embedded programming languages such as JavaScript and Visual Basic, plug-ins to execute platform-independent Java

* Interestingly, this decision turned out to be a very controversial one. On one hand, it could be argued that in doing so, Microsoft contributed greatly to the popularization of the Internet. On the other, it undermined the position of competing browsers and could be seen as anti-competitive. In the end, the strategy led to a series of protracted legal battles over the possible abuse of monopoly by the company, such as *United States v. Microsoft*.

or Flash applets on the user's machine, and useful but tricky HTTP extensions such as cookies. Only a limited degree of superficial compatibility, sometimes hindered by patents and trademarks,^{*} would be maintained.

As the Web grew larger and more diverse, a sneaky disease spread across browser engines under the guise of fault tolerance. At first, the reasoning seemed to make perfect sense: If browser A could display a poorly designed, broken page but browser B refused to (for any reason), users would inevitably see browser B's failure as a bug in that product and flock in droves to the seemingly more capable client, browser A. To make sure that their browsers could display almost any web page correctly, engineers developed increasingly complicated and undocumented heuristics designed to second-guess the intent of sloppy webmasters, often sacrificing security and occasionally even compatibility in the process. Unfortunately, each such change further reinforced bad web design practices[†] and forced the remaining vendors to catch up with the mess to stay afloat. Certainly, the absence of sufficiently detailed, up-to-date standards did not help to curb the spread of this disease.

In 1994, in order to mitigate the spread of engineering anarchy and govern the expansion of HTML, Tim Berners-Lee and a handful of corporate sponsors created the World Wide Web Consortium (W3C). Unfortunately for this organization, for a long while it could only watch helplessly as the format was randomly extended and tweaked. Initial W3C work on HTML 2.0 and HTML 3.2 merely tried to catch up with the status quo, resulting in half-baked specs that were largely out-of-date by the time they were released to the public. The consortium also tried to work on some novel and fairly well-thought-out projects, such as Cascading Style Sheets, but had a hard time getting buy-in from the vendors.

Other efforts to standardize or improve already implemented mechanisms, most notably HTTP and JavaScript, were driven by other auspices such as the European Computer Manufacturers Association (ECMA), the International Organization for Standardization (ISO), and the Internet Engineering Task Force (IETF). Sadly, the whole of these efforts was seldom in sync, and some discussions and design decisions were dominated by vendors or other stakeholders who did not care much about the long-term prospects of the technology. The results were a number of dead standards, contradictory advice, and several frightening examples of harmful cross-interactions between otherwise neatly designed protocols—a problem that will be particularly evident when we discuss a variety of content isolation mechanisms in Chapter 9.

The Boring Period: 2000 to 2003

As the efforts to wrangle the Web floundered, Microsoft's dominance grew as a result of its operating system–bundling strategy. By the beginning of the new decade, Netscape Navigator was on the way out, and Internet Explorer

^{*} For example, Microsoft did not want to deal with Sun to license a trademark for JavaScript (a language so named for promotional reasons and not because it had anything to do with Java), so it opted to name its almost-but-not-exactly-identical version “JScript.” Microsoft's official documentation still refers to the software by this name.

[†] Prime examples of misguided and ultimately lethal browser features are content and character set–sniffing mechanisms, both of which will be discussed in Chapter 13.

held an impressive 80 percent market share—a number roughly comparable to what Netscape had held just five years before. On both sides of the fence, security and interoperability were the two most notable casualties of the feature war, but one could hope now that the fighting was over, developers could put differences aside and work together to fix the mess.

Instead, dominance bred complacency: Having achieved its goals brilliantly, Microsoft had little incentive to invest heavily in its browser. Although through version 5, major releases of Internet Explorer (IE) arrived yearly, it took two years for version 6 to surface, then five full years for Internet Explorer 6 to be updated to Internet Explorer 7. Without Microsoft's interest, other vendors had very little leverage to make disruptive changes; most sites were unwilling to make improvements that would work for only a small fraction of their visitors.

On the upside, the slowdown in browser development allowed the W3C to catch up and to carefully explore some new concepts for the future of the Web. New initiatives finalized around the year 2000 included HTML 4 (a cleaned-up language that deprecated or banned many of the redundant or politically incorrect features embraced by earlier versions) and XHTML 1.1 (a strict and well-structured XML-based format that was easier to unambiguously parse, with no proprietary heuristics allowed). The consortium also made significant improvements to JavaScript's Document Object Model and to Cascading Style Sheets. Regrettably, by the end of the century, the Web was too mature to casually undo some of the sins of the old, yet too young for the security issues to be pressing and evident enough for all to see. Syntax was improved, tags were deprecated, validators were written, and deck chairs were rearranged, but the browsers remained pretty much the same: bloated, quirky, and unpredictable.

But soon, something interesting happened: Microsoft gave the world a seemingly unimportant, proprietary API, confusingly named *XMLHttpRequest*. This trivial mechanism was meant to be of little significance, merely an attempt to scratch an itch in the web-based version of Microsoft Outlook. But *XMLHttpRequest* turned out to be far more, as it allowed for largely unconstrained asynchronous HTTP communications between client-side JavaScript and the server without the need for time-consuming and disruptive page transitions. In doing so, the API contributed to the emergence of what would later be dubbed *web 2.0*—a range of complex, unusually responsive, browser-based applications that enabled users to operate on complex data sets, collaborate and publish content, and so on, invading the sacred domain of “real,” installable client software in the process. Understandably, this caused quite a stir.

Web 2.0 and the Second Browser Wars: 2004 and Beyond

XMLHttpRequest, in conjunction with the popularity of the Internet and the broad availability of web browsers, pushed the Web to some new, exciting frontiers—and brought us a flurry of security bugs that impacted both individual users and businesses. By about 2002, worms and browser vulnerabilities had emerged as a frequently revisited theme in the media. Microsoft, by virtue of its market dominance and a relatively dismissive security posture,

took much of the resulting PR heat. The company casually downplayed the problem, but the trend eventually created an atmosphere conducive to a small rebellion.

In 2004, a new contender in the browser wars emerged: Mozilla Firefox (a community-supported descendant of Netscape Navigator) took the offensive, specifically targeting Internet Explorer's poor security track record and standards compliance. Praised by both IT journalists and security experts, Firefox quickly secured a 20 percent market share. While the newcomer soon proved to be nearly as plagued by security bugs as its counterpart from Redmond, its open source nature and the freedom from having to cater to stubborn corporate users allowed developers to fix issues much faster.

NOTE *Why would vendors compete so feverishly? Strictly speaking, there is no money to be made by having a particular market share in the browser world. That said, pundits have long speculated that it is a matter of power: By bundling, promoting, or demoting certain online services (even as simple as the default search engine), whoever controls the browser controls much of the Internet.*

Firefox aside, Microsoft had other reasons to feel uneasy. Its flagship product, the Windows operating system, was increasingly being used as an (expedient?) launch pad for the browser, with more and more applications (from document editors to games) moving to the Web. This could not be good.

These facts, combined with the sudden emergence of Apple's Safari browser and perhaps Opera's advances in the world of smartphones, must have had Microsoft executives scratching their heads. They had missed the early signs of the importance of the Internet in the 1990s; surely they couldn't afford to repeat the mistake. Microsoft put some steam behind Internet Explorer development again, releasing drastically improved and somewhat more secure versions 7, 8, and 9 in rapid succession.

Competitors countered with new features and claims of even better (if still superficial) standards compliance, safer browsing, and performance improvements. Caught off guard by the unexpected success of *XMLHttpRequest* and quick to forget other lessons from the past, vendors also decided to experiment boldly with new ideas, sometimes unilaterally rolling out half-baked or somewhat insecure designs like *globalStorage* in Firefox or *httponly* cookies in Internet Explorer, just to try their luck.

To further complicate the picture, frustrated by creative differences with W3C, a group of contributors created a wholly new standards body called the Web Hypertext Application Technology Working Group (WHATWG). The WHATWG has been instrumental in the development of HTML5, the first holistic and security-conscious revision of existing standards, but it is reportedly shunned by Microsoft due to patent policy disputes.

Throughout much of its history, the Web has enjoyed a unique, highly competitive, rapid, often overly political, and erratic development model with no unifying vision and no one set of security principles. This state of affairs has left a profound mark on how browsers operate today and how secure the user data handled by browsers can be.

Chances are, this situation is not going to change anytime soon.

The Evolution of a Threat

Clearly, web browsers, and their associated document formats and communication protocols, evolved in an unusual manner. This evolution may explain the high number of security problems we see, but by itself it hardly proves that these problems are unique or noteworthy. To wrap up this chapter, let's take a quick look at the very special characteristics behind the most prevalent types of online security threats and explore why these threats had no particularly good equivalents in the years before the Web.

The User as a Security Flaw

Perhaps the most striking (and entirely nontechnical) property of web browsers is that most people who use them are overwhelmingly unskilled. Sure, nonproficient users have been an amusing, fringe problem since the dawn of computing. But the popularity of the Web, combined with its remarkably low barrier to entry, means we are facing a new foe: Most users simply don't know enough to stay safe.

For a long time, engineers working on general-purpose software have made seemingly arbitrary assumptions about the minimal level of computer proficiency required of their users. Most of these assumptions have been without serious consequences; the incorrect use of a text editor, for instance, would typically have little or no impact on system security. Incompetent users simply would not be able to get their work done, a wonderfully self-correcting issue.

Web browsers do not work this way, however. Unlike certain complicated software, they can be *successfully* used by people with virtually no computer training, people who may not even know how to use a text editor. But at the same time, browsers can be operated *safely* only by people with a pretty good understanding of computer technology and its associated jargon, including topics such as Public-Key Infrastructure. Needless to say, this prerequisite is not met by most users of some of today's most successful web applications.

Browsers still look and feel as if they were designed by geeks and for geeks, complete with occasional cryptic and inconsistent error messages, complex configuration settings, and a puzzling variety of security warnings and prompts. A notable study by Berkeley and Harvard researchers in 2006 demonstrated that casual users are almost universally oblivious to signals that surely make perfect sense to a developer, such as the presence or absence of lock icons in the status bar.⁴ In another study, Stanford and Microsoft researchers reached similar conclusions when they examined the impact of the modern "green URL bar" security indicator. The mechanism, designed to offer a more intuitive alternative to lock icons, actually made it easier to trick users by teaching the audience to trust a particular shade of green, no matter where this color appeared.⁵

Some experts argue that the ineptitude of the casual user is not the fault of software vendors and hence not an engineering problem at all. Others note that when creating software so easily accessible and so widely distributed, it is irresponsible to force users to make security-critical decisions that depend on technical prowess not required to operate the program in the first place.

To blame browser vendors alone is just as unfair, however: The computing industry as a whole has no robust answers in this area, and very little research is available on how to design comparably complex user interfaces (UIs) in a bulletproof way. After all, we barely get it right for ATMs.

The Cloud, or the Joys of Communal Living

Another peculiar characteristic of the Web is the dramatically understated separation between unrelated applications and the data they process.

In the traditional model followed by virtually all personal computers over the last 15 years or so, there are very clear boundaries between high-level data objects (documents), user-level code (applications), and the operating system kernel that arbitrates all cross-application communications and hardware input/output (I/O) and enforces configurable security rules should an application go rogue. These boundaries are well studied and useful for building practical security schemes. A file opened in your text editor is unlikely to be able to steal your email, unless a really unfortunate conjunction of implementation flaws subverts all these layers of separation at once.

In the browser world, this separation is virtually nonexistent: Documents and code live as parts of the same intermingled blobs of HTML, isolation between completely unrelated applications is partial at best (with all sites nominally sharing a global JavaScript environment), and many types of interaction between sites are implicitly permitted with few, if any, flexible, browser-level security arbitration frameworks.

In a sense, the model is reminiscent of CP/M, DOS, and other principally nonmultitasking operating systems with no robust memory protection, CPU preemption, or multiuser features. The obvious difference is that few users depended on these early operating systems to simultaneously run multiple untrusted, attacker-supplied applications, so there was no particular reason for alarm.

In the end, the seemingly unlikely scenario of a text file stealing your email is, in fact, a frustratingly common pattern on the Web. Virtually all web applications must heavily compensate for unsolicited, malicious cross-domain access and take cumbersome steps to maintain at least some separation of code and the displayed data. And sooner or later, virtually all web applications fail. Content-related security issues, such as cross-site scripting or cross-site request forgery, are extremely common and have very few counterparts in dedicated, compartmentalized client architectures.

Nonconvergence of Visions

Fortunately, the browser security landscape is not entirely hopeless, and despite limited separation between web applications, several selective security mechanisms offer rudimentary protection against the most obvious attacks. But this brings us to another characteristic that makes the Web such an interesting subject: There is no shared, holistic security model to grasp and live by. We are not looking for a grand vision for world peace, mind you, but simply a common set of flexible paradigms that would apply to most, if not all, of the

relevant security logic. In the Unix world, for example, the *rwx* user/group permission model is one such strong unifying theme. But in the browser realm?

In the browser realm, a mechanism called *same-origin policy* could be considered a candidate for a core security paradigm, but only until one realizes that it governs a woefully small subset of cross-domain interactions. That detail aside, even within its scope, it has no fewer than seven distinct varieties, each of which places security boundaries between applications in a slightly different place.* Several dozen additional mechanisms, with no relation to the same-origin model, control other key aspects of browser behavior (essentially implementing what each author considered to be the best approach to security controls that day).

As it turns out, hundreds of small, clever hacks do not necessarily add up to a competent security opus. The unusual lack of integrity makes it very difficult even to decide where a single application ends and a different one begins. Given this reality, how does one assess attack surfaces, grant or take away permissions, or accomplish just about any other security-minded task? Too often, “by keeping your fingers crossed” is the best response we can give.

Curiously, many well-intentioned attempts to improve security by defining new security controls only make the problem worse. Many of these schemes create new security boundaries that, for the sake of elegance, do not perfectly align with the hairy juxtaposition of the existing ones. When the new controls are finer grained, they are likely to be rendered ineffective by the legacy mechanisms, offering a false sense of security; when they are more coarse grained, they may eliminate some of the subtle assurances that the Web depends on right now. (Adam Barth and Collin Jackson explore the topic of destructive interference between browser security policies in their academic work.)⁶

Cross-Browser Interactions: Synergy in Failure

The overall susceptibility of an ecosystem composed of several different software products could be expected to be equal to a simple sum of the flaws contributed by each of the applications. In some cases, the resulting exposure may be less (diversity improves resilience), but one would not expect it to be more.

The Web is once again an exception to the rule. The security community has discovered a substantial number of issues that cannot be attributed to any particular piece of code but that emerge as a real threat when various browsers try to interact with each other. No particular product can be easily singled out for blame: They are all doing their thing, and the only problem is that no one has bothered to define a common etiquette for all of them to obey.

For example, one browser may assume that, in line with its own security model, it is safe to pass certain URLs to external applications or to store or read back certain types of data from disk. For each such assumption, there likely exists at least one browser that strongly disagrees, expecting other

*The primary seven varieties, as discussed throughout Part II of this book, include the security policy for JavaScript DOM access; *XMLHttpRequest* API; HTTP cookies; local storage APIs; and plug-ins such as Flash, Silverlight, or Java.

parties to follow its rules instead. The exploitability of these issues is greatly aggravated by vendors' desire to get their foot in the door and try to allow web pages to switch to their browser on the fly without the user's informed consent. For example, Firefox allows pages to be opened in its browser by registering a *firefoxurl:* protocol; Microsoft installs its own .NET gateway plug-in in Firefox; Chrome does the same to Internet Explorer via a protocol named *cf:*.

NOTE *Especially in the case of such interactions, pinning the blame on any particular party is a fool's errand. In a recent case of a bug related to *firefoxurl:*, Microsoft and half of the information security community blamed Mozilla, while Mozilla and the other half of experts blamed Microsoft.⁷ It did not matter who was right: The result was still a very real mess.*

Another set of closely related problems (practically unheard of in the days before the Web) are the incompatibilities in superficially similar security mechanisms implemented in each browser. When the security models differ, a sound web application-engineering practice in one product may be inadequate and misguided in another. In fact, several classes of rudimentary tasks, such as serving a user-supplied plaintext file, cannot be safely implemented in certain browsers at all. This fact, however, will not be obvious to developers unless they are working in one of the affected browsers—and even then, they need to hit just the right spot.

In the end, all the characteristics outlined in this section contribute to a whole new class of security vulnerabilities that a taxonomy buff might call a *failure to account for undocumented diversity*. This class is very well populated today.

The Breakdown of the Client-Server Divide

Information security researchers enjoy the world of static, clearly assigned roles, which are a familiar point of reference when mapping security interactions in the otherwise complicated world. For example, we talk about Alice and Bob, two wholesome, hardworking users who want to communicate, and Mallory, a sneaky attacker who is out to get them. We then have client software (essentially dumb, sometimes rogue I/O terminals that frivolously request services) and humble servers, carefully fulfilling the clients' whim. Developers learn these roles and play along, building fairly comprehensible and testable network-computing environments in the process.

The Web began as a classical example of a proper client-server architecture, but the functional boundaries between client and server responsibilities were quickly eroded. The culprit is JavaScript, a language that offers the HTTP servers a way to delegate application logic to the browser ("client") side and gives them two very compelling reasons to do so. First, such a shift often results in more responsive user interfaces, as servers do not need to synchronously participate in each tiny UI state change imaginable. Second, server-side CPU and memory requirements (and hence service-provisioning costs) can decrease drastically when individual workstations across the globe chip in to help with the bulk of the work.

The client-server diffusion process began innocently enough, but it was only a matter of time before the first security mechanisms followed to the client side too, along with all the other mundane functionality. For example, what was the point of carefully scrubbing HTML on the server side when the data was only dynamically rendered by JavaScript on the client machine?

In some applications, this trend was taken to extremes, eventually leaving the server as little more than a dumb storage device and moving almost all the parsing, editing, display, and configuration tasks into the browser itself. In such designs, the dependency on a server could even be fully severed by using offline web extensions such as HTML5 persistent storage.

A simple shift in where the entire application magic happens is not necessarily a big deal, but not all security responsibilities can be delegated to the client as easily. For example, even in the case of a server acting as dumb storage, clients cannot be given indiscriminate access to all the data stored on the server for other users, and they cannot be trusted to enforce access controls. In the end, because it was not desirable to keep all the application security logic on the server side, and it was impossible to migrate it fully to the client, most applications ended up occupying some arbitrary middle ground instead, with no easily discernible and logical separation of duties between the client and server components. The resulting unfamiliar designs and application behaviors simply had no useful equivalents in the elegant and wholesome world of security role-play.

The situation has resulted in more than just a design-level mess; it has led to irreducible complexity. In a traditional client-server model with well-specified APIs, one can easily evaluate a server's behavior without looking at the client, and vice versa. Moreover, within each of these components, it is possible to easily isolate smaller functional blocks and make assumptions about their intended operation. With the new model, coupled with the opaque, one-off application APIs common on the Web, these analytical tools, and the resulting ease of reasoning about the security of a system, have been brutally taken away.

The unexpected failure of standardized security modeling and testing protocols is yet another problem that earns the Web a very special—and scary—place in the universe of information security.

Global browser market share, May 2011

Vendor	Browser Name	Market Share	
Microsoft	Internet Explorer 6	10%	52%
	Internet Explorer 7	7%	
	Internet Explorer 8	31%	
	Internet Explorer 9	4%	
Mozilla	Firefox 3	12%	22%
	Firefox 4+	10%	
Google	Chrome	13%	
Apple	Safari	7%	
Opera Software	Opera	3%	

Source: Data drawn from public Net Applications reports.¹

PART I

ANATOMY OF THE WEB

The first part of this book focuses on the principal concepts that govern the operation of web browsers, namely, the protocols, document formats, and programming languages that make it all tick. Because all the familiar, user-visible security mechanisms employed in modern browsers are profoundly intertwined with these inner workings, the bare internals deserve a fair bit of attention before we wander off deeper into the woods.

2

IT STARTS WITH A URL

The most recognizable hallmark of the Web is a simple text string known as the *Uniform Resource Locator (URL)*. Each well-formed, fully qualified URL is meant to conclusively address and uniquely identify a single resource on a remote server (and in doing so, implement a couple of related, auxiliary functions). The URL syntax is the cornerstone of the address bar, the most important user interface (UI) security indicator in every browser.

In addition to true URLs used for content retrieval, several classes of *pseudo-URLs* use a similar syntax to provide convenient access to browser-level features, including the integrated scripting engine, several special document-rendering modes, and so on. Perhaps unsurprisingly, these pseudo-URL actions can have a significant impact on the security of any site that decides to link to them.

The ability to figure out how a particular URL will be interpreted by the browser, and the side effects it will have, is one of the most basic and common security tasks attempted by humans and web applications alike, but it can

be a problematic one. The generic URL syntax, the work of Tim Berners-Lee, is codified primarily in RFC 3986;¹ its practical uses on the Web are outlined in RFCs 1738,² 2616,³ and a couple of other, less-significant standards. These documents are remarkably detailed, resulting in a fairly complex parsing model, but they are not precise enough to lead to harmonious, compatible implementations in all client software. In addition, individual software vendors have chosen to deviate from the specifications for their own reasons.

Let's have a closer look at how the humble URL works in practice.

Uniform Resource Locator Structure

Figure 2-1 shows the format of a *fully qualified absolute URL*, one that specifies all information required to access a particular resource and that does not depend in any way on where the navigation began. In contrast, a *relative URL*, such as `../file.php?text=hello+world`, omits some of this information and must be interpreted in the context of a base URL associated with the current browsing context.

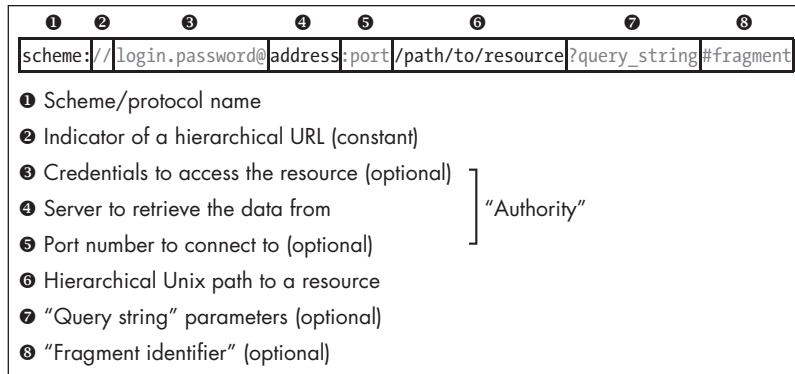


Figure 2-1: Structure of an absolute URL

The segments of the absolute URL seem intuitive, but each comes with a set of gotchas, so let's review them now.

Scheme Name

The *scheme name* is a case-insensitive string that ends with a single colon, indicating the protocol to be used to retrieve the resource. The official registry of valid URL schemes is maintained by the *Internet Assigned Numbers Authority* (IANA), a body more widely known for its management of the IP address space.⁴ IANA's current list of valid scheme names includes several dozen entries such as `http:`, `https:`, and `ftp:`; in practice, a much broader set of schemes is informally recognized by common browsers and third-party applications, some which have special security consequences. (Of particular interest are several types of pseudo-URLs, such as `data:` or `javascript:`, as discussed later in this chapter and throughout the remainder of this book.)

Before they can do any further parsing, browsers and web applications need to distinguish fully qualified absolute URLs from relative ones. The presence of a valid scheme in front of the address is meant to be the key difference, as defined in RFC 1738: In a compliant absolute URL, only the alphanumerics “+”, “-”, and “.” may appear before the required “:”. In practice, however, browsers deviate from this guidance a bit. All ignore leading newlines and white spaces. Internet Explorer ignores the entire nonprintable character range of ASCII codes 0x01 to 0x1F. Chrome additionally skips 0x00, the NUL character. Most implementations also ignore newlines and tabs in the middle of scheme names, and Opera accepts high-bit characters in the string.

Because of these incompatibilities, applications that depend on the ability to differentiate between relative and absolute URLs must conservatively reject any anomalous syntax—but as we will soon find out, even this is not enough.

Indicator of a Hierarchical URL

In order to comply with the generic syntax rules laid out in RFC 1738, every absolute, hierarchical URL is required to contain the fixed string “//” right before the authority section. If the string is missing, the format and function of the remainder of the URL is undefined for the purpose of that specification and must be treated as an opaque, scheme-specific value.

NOTE *An example of a nonhierarchical URL is the `mailto:` protocol, used to specify email addresses and possibly a subject line (`mailto:user@example.com?subject=Hello+world`). Such URLs are passed down to the default mail client without making any further attempt to parse them.*

The concept of a generic, hierarchical URL syntax is, in theory, an elegant one. It ought to enable applications to extract some information about the address without knowing how a particular scheme works. For example, without a preconceived notion of the *wacky-widget:* protocol, and by applying the concept of generic URL syntax alone, the browser could decide that `http://example.com/test1/` and `wacky-widget://example.com/test2/` reference the same, trusted remote host.

Regrettably, the specification has an interesting flaw: The aforementioned RFC says nothing about what the implementer should do when encountering URLs where the scheme is known to be nonhierarchical but where a “//” prefix still appears, or vice versa. In fact, a reference parser implementation provided in RFC 1630 contains an unintentional loophole that gives a counterintuitive meaning to the latter class of URLs. In RFC 3986, published some years later, the authors sheepishly acknowledge this flaw and permit implementations to try to parse such URLs for compatibility reasons. As a consequence, many browsers interpret the following examples in unexpected ways:

- ***http:example.com/*** In Firefox, Chrome, and Safari, this address may be treated identically to `http://example.com/` when no fully qualified base URL context exists and as a relative reference to a directory named `example.com` when a valid base URL is available.

- ***javascript://example.com/%0Aalert(1)*** This string is interpreted as a valid nonhierarchical pseudo-URL in all modern browsers, and the JavaScript *alert(1)* code will execute, showing a simple dialog window.
- ***mailto://user@example.com*** Internet Explorer accepts this URL as a valid nonhierarchical reference to an email address; the “//” part is simply skipped. Other browsers disagree.

Credentials to Access the Resource

The credentials portion of the URL is optional. This location can specify a username, and perhaps a password, that may be required to retrieve the data from the server. The method through which these credentials are exchanged is not specified as a part of the abstract URL syntax, and it is always protocol specific. For those protocols that do not support authentication, the behavior of a credential-bearing URL is simply undefined.

When no credentials are supplied, the browser will attempt to fetch the resource anonymously. In the case of HTTP and several other protocols, this means not sending any authentication data; for FTP, it involves logging into a guest account named *ftp* with a bogus password.

Most browsers accept almost any characters, other than general URL section delimiters, in this section with two exceptions: Safari, for unclear reasons, rejects a broader set of characters, including “<”, “>”, “[”, and “]”, while Firefox also rejects newlines.*

Server Address

For all fully qualified hierarchical URLs, the server address section must specify a case-insensitive DNS name (such as *example.com*), a raw IPv4 address (such as *127.0.0.1*), or an IPv6 address in square brackets (such as *[0:0:0:0:0:0:1]*), indicating the location of a server hosting the requested resource. Firefox will also accept IPv4 addresses and hostnames in square brackets, but other implementations reject them immediately.

Although the RFC permits only canonical notations for IP addresses, standard C libraries used by most applications are much more relaxed, accepting noncanonical IPv4 addresses that mix octal, decimal, and hexadecimal notation or concatenate some or all of the octets into a single integer. As a result, the following options are recognized as equivalent:

- ***http://127.0.0.1/*** This is a canonical representation of an IPv4 address.
- ***http://0x7f.1/*** This is a representation of the same address that uses a hexadecimal number to represent the first octet and concatenates all the remaining octets into a single decimal value.
- ***http://017700000001/*** The same address is denoted using a 0-prefixed octal value, with all octets concatenated into a single 32-bit integer.

* This is possibly out of the concern for FTP, which transmits user credentials without any encoding; in this protocol, a newline transmitted as is would be misinterpreted by the server as the beginning of a new FTP command. Other browsers may transmit FTP credentials in noncompliant percent-encoded form or simply strip any problematic characters later on.

A similar laid-back approach can be seen with DNS names. Theoretically, DNS labels need to conform to a very narrow character set (specifically, alphanumeric, “.”, and “-”, as defined in RFC 1035⁵), but many browsers will happily ask the underlying operating system resolver to look up almost anything, and the operating system will usually also not make a fuss. The exact set of characters accepted in the hostname and passed to the resolver varies from client to client. Safari is most rigorous, while Internet Explorer is the most permissive. Perhaps of note, several control characters in the 0x0A–0x0D and 0xA0–0xAD ranges are ignored by most browsers in this portion of the URL.

NOTE *One fascinating behavior of the URL parsers in all of the mainstream browsers is their willingness to treat the character “○” (ideographic full stop, Unicode point U+3002) identically to a period in hostnames but not anywhere else in the URL. This is reportedly because certain Chinese keyboard mappings make it much easier to type this symbol than the expected 7-bit ASCII value.*

Server Port

This server port is an optional section that describes a nonstandard network port to connect to on the previously specified server. Virtually all application-level protocols supported by browsers and third-party applications use TCP or UDP as the underlying transport method, and both TCP and UDP rely on 16-bit port numbers to separate traffic between unrelated services running on a single machine. Each scheme is associated with a default port on which servers for that protocol are customarily run (80 for HTTP, 21 for FTP, and so on), but the default can be overridden at the URL level.

NOTE *An interesting and unintended side effect of this feature is that browsers can be tricked into sending attacker-supplied data to random network services that do not speak the protocol the browser expects them to. For example, one may point a browser to `http://mail.example.com:25/`, where 25 is a port used by the Simple Mail Transfer Protocol (SMTP) service rather than HTTP. This fact has caused a range of security problems and prompted a number of imperfect workarounds, as discussed in more detail in Part II of this book.*

Hierarchical File Path

The next portion of the URL, the hierarchical file path, is envisioned as a way to identify a specific resource to be retrieved from the server, such as `/documents/2009/my_diary.txt`. The specification quite openly builds on top of the Unix directory semantics, mandating the resolution of “/..” and “/./” segments in the path and providing a directory-based method for sorting out relative references in non-fully qualified URLs.

Using the filesystem model must have seemed like a natural choice in the 1990s, when web servers acted as simple gateways to a collection of static files and the occasional executable script. But since then, many contemporary web application frameworks have severed any remaining ties with the filesystem, interfacing directly with database objects or registered locations in resident program code. Mapping these data structures to well-behaved URL

paths is possible but not always practiced or practiced carefully. All of this makes automated content retrieval, indexing, and security testing more complicated than it should be.

Query String

The query string is an optional section used to pass arbitrary, nonhierarchical parameters to the resource earlier identified by the path. One common example is passing user-supplied terms to a server-side script that implements the search functionality, such as:

```
http://example.com/search.php?query=Hello+world
```

Most web developers are accustomed to a particular layout of the query string; this familiar format is generated by browsers when handling HTML-based forms and follows this syntax:

```
name1=value1&name2=value2...
```

Surprisingly, such layout is not mandated in the URL RFCs. Instead, the query string is treated as an opaque blob of data that may be interpreted by the final recipient as it sees fit, and unlike the path, it is not encumbered with specific parsing rules.

Hints of the commonly used format can be found in an informational RFC 1630,⁶ in a mail-related RFC 2368,⁷ and in HTML specifications dealing with forms.⁸ None of this is binding, and therefore, while it may be impolite, it is not a mistake for web applications to employ arbitrary formats for whatever data they wish to put in that part of the URL.

Fragment ID

The fragment ID is an opaque value with a role similar to the query string but that provides optional instructions for the client application rather than the server. (In fact, the value is not supposed to be sent to the server at all.) Neither the format nor function of the fragment ID is clearly specified in the RFCs, but it is hinted that it may be used to address “subresources” in the retrieved document or to provide other document-specific rendering cues.

In practice, fragment identifiers have only a single sanctioned use in the browser: that of specifying the name of an anchor HTML element for in-document navigation. The logic is simple. If an anchor name is supplied in the URL and a matching HTML tag can be located, the document will be scrolled to that location for viewing; otherwise, nothing happens. Because the information is encoded in the URL, this particular view of a lengthy document could be easily shared with others or bookmarked. In this use, the meaning of a fragment ID is limited to scrolling an existing document, so there is no need to retrieve any new data from the server when only this portion of the URL is updated in response to user actions.

This interesting property has led to another, more recent and completely ad hoc use of this value: to store miscellaneous state information needed by client-side scripts. For example, consider a map-browsing application that puts the currently viewed map coordinates in the fragment identifier so that it will know to resume from that same location if the link is bookmarked or shared. Unlike updating the query string, changing the fragment ID on-the-fly will not trigger a time-consuming page reload, making this data-storage trick a killer feature.

Putting It All Together Again

Each of the aforementioned URL segments is delimited by certain reserved characters: slashes, colons, question marks, and so on. To make the whole approach usable, these delimiting characters should not appear anywhere in the URL for any other purpose. With this assumption in mind, imagine a sample algorithm to split absolute URLs into the aforementioned functional parts in a manner at least vaguely consistent with how browsers accomplish this task. A reasonably decent example of such an algorithm could be:

STEP 1: Extract the scheme name.

Scan for the first “:” character. The part of the URL to its left is the scheme name. Bail out if the scheme name does not conform to the expected set of characters; the URL may need to be treated as a relative one if so.

STEP 2: Consume the hierarchical URL identifier.

The string “//” should follow the scheme name. Skip it if found; bail out if not.

NOTE *In some parsing contexts, implementations will be just as happy with zero, one, or even three or more slashes instead of two, for usability reasons. In the same vein, from its inception, Internet Explorer accepted backslashes (\) in lieu of slashes in any location in the URL, presumably to assist inexperienced users.* All browsers other than Firefox eventually followed this trend and recognize URLs such as http:\\example.com\\.*

STEP 3: Grab the authority section.

Scan for the next “/”, “?”, or “#”, whichever comes first, to extract the authority section from the URL. As mentioned above, most browsers will also accept “\” as a delimiter in place of a forward slash, which may need to be accounted for. The semicolon (;) is another acceptable authority delimiter in browsers other than Internet Explorer and Safari; the reason for this decision is unknown.

* Unlike UNIX-derived operating systems, Microsoft Windows uses backslashes instead of slashes to delimit file paths (say, `c:\windows\system32\calc.exe`). Microsoft probably tried to compensate for the possibility that users would be confused by the need to type a different type of a slash on the Web or hoped to resolve other possible inconsistencies with *file*: URLs and similar mechanisms that would be interfacing directly with the local filesystem. Other Windows filesystem specifics (such as case insensitivity) are not replicated, however.

STEP 3A: Find the credentials, if any.

Once the authority section is extracted, locate the at symbol (@) in the substring. If found, the leading snippet constitutes login credentials, which should be further tokenized at the first occurrence of a colon (if present) to split the login and password data.

STEP 3B: Extract the destination address.

The remainder of the authority section is the destination address. Look for the first colon to separate the hostname from the port number. A special case is needed for bracket-enclosed IPv6 addresses, too.

STEP 4: Identify the path (if present).

If the authority section is followed immediately by a forward slash—or for some implementations, a backslash or semicolon, as noted earlier—scan for the next “?”, “#”, or end-of-string, whichever comes first. The text in between constitutes the path section, which should be normalized according to Unix path semantics.

STEP 5: Extract the query string (if present).

If the last successfully parsed segment is followed by a question mark, scan for the next “#” character or end-of-string, whichever comes first. The text in between is the query string.

STEP 6: Extract the fragment identifier (if present).

If the last successfully parsed segment is followed by “#”, everything from that character to the end-of-string is the fragment identifier. Either way, you’re done!

This algorithm may seem mundane, but it reveals subtle details that even seasoned programmers normally don’t think about. It also illustrates that it is extremely difficult for casual users to understand how a particular URL may be parsed. Let’s start with this fairly simple case:

```
http://example.com&gibberish=1234@167772161/
```

The target of this URL—a concatenated IP address that decodes to 10.0.0.1—is not readily apparent to a nonexpert, and many users would believe they are visiting *example.com* instead.* But all right, that was an easy one! So let’s have a peek at this syntax instead:

```
http://example.com\coredump.cx/
```

In Firefox, that URL will take the user to *coredump.cx*, because *example.com* will be interpreted as a valid value for the login field. In almost all other browsers, “\” will be interpreted as a path delimiter, and the user will land on *example.com* instead.

*This particular @-based trick was quickly embraced to facilitate all sorts of online fraud targeted at casual users. Attempts to mitigate its impact ranged from the heavy-handed and oddly specific (e.g., disabling URL-based authentication in Internet Explorer or crippling it with warnings in Firefox) to the fairly sensible (e.g., hostname highlighting in the address bar of several browsers).

An even more frustrating example exists for Internet Explorer. Consider this:

```
http://example.com;.coredump.cx/
```

Microsoft's browser permits ";" in the hostname and successfully resolves this label, thanks to the appropriate configuration of the *coredump.cx* domain. Most other browsers will autocorrect the URL to *http://example.com/;.coredump.cx* and take the user to *example.com* instead (except for Safari, where the syntax causes an error). If this looks messy, remember that we are just getting started with how browsers work!

Reserved Characters and Percent Encoding

The URL-parsing algorithm outlined in the previous section relies on the assumption that certain reserved, syntax-delimiting characters will not appear literally in the URL in any other capacity (that is, they won't be a part of the user-name, request path, and so on). These generic, syntax-disrupting delimiters are:

```
: / ? # [ ] @
```

The RFC also names a couple of lower-tier delimiters without giving them any specific purpose, presumably to allow scheme- or application-specific features to be implemented within any of the top-level sections:

```
! $ & ' ( ) * + , ; =
```

All of the above characters are in principle off-limits, but there are legitimate cases where one would want to include them in the URL (for example, to accommodate arbitrary search terms entered by the user and passed to the server in the query string). Therefore, rather than ban them, the standard provides a method to encode all spurious occurrences of these values. The method, simply called *percent encoding* or *URL encoding*, substitutes characters with a percent sign (%) followed by two hexadecimal digits representing a matching ASCII value. For example, "/" will be encoded as *%2F* (uppercase is customary but not enforced). It follows that to avoid ambiguity, the naked percent sign itself must be encoded as *%25*. Any intermediaries that handle existing URLs (browsers and web applications included) are further compelled never to attempt to decode or encode reserved characters in relayed URLs, because the meaning of such a URL may suddenly change.

Regrettably, the immutability of reserved characters in existing URLs is at odds with the need to respond to any URLs that are technically illegal because they misuse these characters and that are encountered by the browser in the wild. This topic is not covered by the specifications at all, which forces browser vendors to improvise and causes cross-implementation inconsistencies. For example, should the URL *http://a@b@c/* be translated to *http://a@b%40c/* or perhaps to *http://a%40b@c/?* Internet Explorer and Safari think the former makes more sense; other browsers side with the latter view.

The remaining characters not in the reserved set are not supposed to have any particular significance within the URL syntax itself. However, some (such as nonprintable ASCII control characters) are clearly incompatible with the idea that URLs should be human readable and transport-safe. Therefore, the RFC outlines a confusingly named subset of *unreserved* characters (consisting of alphanumerics, “-”, “.”, “_”, and “~”) and says that only this subset and the reserved characters in their intended capacity are formally allowed to appear in the URL as is.

NOTE *Curiously, these unreserved characters are only allowed to appear in an unescaped form; they are not required to do so. User agents may encode or decode them at whim, and doing so does not change the meaning of the URL at all. This property brings up yet another way to confuse users: the use of noncanonical representations of unreserved characters. Specifically, all of the following are equivalent:*

- `http://example.com/`
- `http://%65xample.%63om/`
- `http://%65%78%61%6d%70%6c%65%2e%63%6f%6d/*`

A number of otherwise nonreserved, printable characters are excluded from the so-called unreserved set. Because of this, strictly speaking, the RFCs require them to be unconditionally percent encoded. However, since browsers are not explicitly tasked with the enforcement of this rule, it is not taken very seriously. In particular, all browsers allow “^”, “{”, “|”, and “}” to appear in URLs without escaping and will send these characters to the server as is. Internet Explorer further permits “<”, “>”, and “~” to go through; Internet Explorer, Firefox, and Chrome all accept “\”; Chrome and Internet Explorer will permit a double quote; and Opera and Internet Explorer both pass the nonprintable character 0x7F (DEL) as is.

Lastly, contrary to the requirements spelled out in the RFC, most browsers also do not encode fragment identifiers at all. This poses an unexpected challenge to client-side scripts that rely on this string and expect certain potentially unsafe characters never to appear literally. We will revisit this topic in Chapter 6.

Handling of Non-US-ASCII Text

Many languages used around the globe rely on characters outside the basic, 7-bit ASCII character set or the default 8-bit code page traditionally used by all PC-compatible systems (CP437). Heck, some languages depend on alphabets that are not based on Latin at all.

In order to accommodate the needs of an often-ignored but formidable non-English user base, various 8-bit code pages with an alternative set of high-bit characters were devised long before the emergence of the Web: ISO 8859-1,

^{*}Similar noncanonical encodings were widely used for various types of social engineering attacks, and consequently, various countermeasures have been deployed through the years. As usual, some of these countermeasures are disruptive (for example, Firefox flat out rejects percent-encoded text in hostnames), and some are fairly good (such as the forced “canonicalization” of the address bar by decoding all the unnecessarily encoded text for display purposes).

CP850, and Windows 1252 for Western European languages; ISO 8859-2, CP852, and Windows 1250 for Eastern and Central Europe; and KOI8-R and Windows 1251 for Russia. And, because several alphabets could not be accommodated in the 256-character space, we saw the rise of complex variable-width encodings, such as Shift JIS for katakana.

The incompatibility of these character maps made it difficult to exchange documents between computers configured for different code pages. By the early 1990s, this growing problem led to the creation of *Unicode*—a sort of universal character set, too large to fit within 8 bits but meant to encompass practically all regional scripts and specialty pictographs known to man. Unicode was followed by UTF-8, a relatively simple, variable-width representation of these characters, which was theoretically safe for all applications capable of handling traditional 8-bit formats. Unfortunately, UTF-8 required more bytes to encode high-bit characters than did most of its competitors, and to many users, this seemed wasteful and unnecessary. Because of this criticism, it took well over a decade for UTF-8 to gain traction on the Web, and it only did so long after all the relevant protocols had solidified.

This unfortunate delay had some bearing on the handling of URLs that contain user input. Browsers needed to accommodate such use very early on, but when the developers turned to the relevant standards, they found no meaningful advice. Even years later, in 2005, the RFC 3986 had just this to say:

In local or regional contexts and with improving technology, users might benefit from being able to use a wider range of characters; such use is not defined by this specification.

Percent-encoded octets . . . may be used within a URI to represent characters outside the range of the US-ASCII coded character set if this representation is allowed by the scheme or by the protocol element in which the URI is referenced. Such a definition should specify the character encoding used to map those characters to octets prior to being percent-encoded for the URI.

Alas, despite this wishful thinking, none of the remaining standards addressed this topic. It was always possible to put raw high-bit characters in a URL, but without knowing the code page they should be interpreted in, the server would not be able to tell if that `%B1` was supposed to mean “±”, “ą”, or some other squiggly character specific to the user’s native script.

Sadly, browser vendors have not taken the initiative and come up with a consistent solution to this problem. Most browsers internally transcode URL path segments to UTF-8 (or ISO 8859-1, if sufficient), but then they generate the query string in the code page of the referring page instead. In certain cases, when URLs are entered manually or passed to certain specialized APIs, high-bit characters may be also downgraded to their 7-bit US-ASCII look-alikes, replaced with question marks, or even completely mangled due to implementation flaws.

Poorly implemented or not, the ability to pass non-English characters in query strings and paths scratched an evident itch. The traditional percent-encoding approach left just one URL segment completely out in the cold: High-bit input could not be allowed as is when specifying the name of the destination server, because at least in principle, the well-established DNS standard permitted only period-delimited alphanumerics and dashes to appear in domain names—and while nobody adhered to the rules, the set of exceptions varied from one name server to another.

An astute reader might wonder why this limitation would matter; that is, why was it important to have localized domain names in non-Latin alphabets, too? That question may be difficult to answer now. Quite simply, several folks thought a lack of these encodings would prevent businesses and individuals around the world from fully embracing and enjoying the Web—and, rightly or not, they were determined to make it happen.

This pursuit led to the formation of the Internationalized Domain Names in Applications (IDNA). First, RFC 3490,⁹ which outlined a rather contrived scheme to encode arbitrary Unicode strings using alphanumerics and dashes, and then RFC 3492,¹⁰ which described a way to apply this encoding to DNS labels using a format known as *Punycode*. Punycode looked roughly like this:

```
xn--[US-ASCII part]-[encoded Unicode data]
```

A compliant browser presented with a technically illegal URL that contained a literal non-US-ASCII character anywhere in the hostname was supposed to transform the name to Punycode before performing a DNS lookup. Consequently, when presented with Punycode in an existing URL, it should put a decoded, human-readable form of the string in the address bar.

NOTE *Combining all these incompatible encoding strategies can make for an amusing mix. Consider this example URL of a made-up Polish-language towel shop:*

```
Intent: http://www.ręcniki.pl/ręcnik?model=Jaś#Złóż_zamówienie
Actual URL: http://www.xn--rczniki-98a.pl/r%C4%99cnik?model=Ja%B6#Złóż_zamówienie
```

 Label converted to Punycode Path converted to UTF-8 Query string converted to ISO 8859-2 Literal UTF-8

Of all the URL-based encoding approaches, IDNA soon proved to be the most problematic. In essence, the domain name in the URL shown in the browser's address bar is one of the most important security indicators on the Web, as it allows users to quickly differentiate sites they trust and have done business with from the rest of the Internet. When the hostname shown by the browser consists of 38 familiar and distinctive characters, only fairly careless victims will be tricked into thinking that their favorite *example.com* domain and an impostor *exampl**I**e.com* site are the same thing. But IDNA casually and indiscriminately extended these 38 characters to some 100,000 glyphs supported by Unicode, many of which look exactly alike and are separated from each other based on functional differences alone.

How bad is it? Let's consider Cyrillic, for example. This alphabet has a number of homoglyphs that look practically identical to their Latin counterparts but that have completely different Unicode values and resolve to completely different Punycode DNS names:

Latin	a	c	e	i	j	o	p	s	x	y
	U+0061	U+0063	U+0065	U+0069	U+006A	U+006F	U+0070	U+0073	U+0078	U+0079
Cyrillic	а	с	е	и	ј	о	р	ѕ	х	у
	U+0430	U+0441	U+0435	U+0456	U+0458	U+043E	U+0440	U+0455	U+0445	U+0443

When IDNA was proposed and first implemented in browsers, nobody seriously considered the consequences of this issue. Browser vendors apparently assumed that DNS registrars would prevent people from registering look-alike names, and registrars figured it was the browser vendors' problem to have unambiguous visuals in the address bar.

In 2002 the significance of the problem was finally recognized by all parties involved. That year, Evgeniy Gabrilovich and Alex Gontmakher published "The Homograph Attack,"¹¹ a paper exploring the vulnerability in great detail. They noted that any registrar-level work-arounds, even if implemented, would have a fatal flaw. An attacker could always purchase a wholesome top-level domain and then, on his own name server, set up a subdomain record that, with the IDNA transformation applied, would decode to a string visually identical to *example.com/* (the last character being merely a nonfunctional look-alike of the actual ASCII slash). The result would be:

`http://example.com. wholesome-domain.com/`

This only looks like a real slash.

There is nothing that a registrar can do to prevent this attack, and the ball is in the browser vendors' court. But what options do they have, exactly?

As it turns out, there aren't many. We now realize that the poorly envisioned IDNA standard cannot be fixed in a simple and painless way. Browser developers have responded to this risk by reverting to incomprehensible Punycode when a user's locale does not match the script seen in a particular DNS label (which causes problems when browsing foreign sites or when using imported or simply misconfigured computers); permitting IDNA only in certain country-specific, top-level domains (ruling out the use of internationalized domain names in *.com* and other high-profile TLDs); and blacklisting certain "bad" characters that resemble slashes, periods, white spaces, and so forth (a fool's errand, given the number of typefaces used around the world).

These measures are drastic enough to severely hinder the adoption of internationalized domain names, probably to a point where the standard's lingering presence causes more security problems than it brings real usability benefits to non-English users.

Common URL Schemes and Their Function

Let's leave the bizarre world of URL parsing behind us and go back to the basics. Earlier in this chapter, we implied that certain schemes may have unexpected security consequences and that because of this, any web application handling user-supplied URLs must be cautious. To explain this point a bit better, it is useful to review all the URL schemes commonly supported in a typical browser environment. These can be combined into four basic groups.

Browser-Supported, Document-Fetching Protocols

These schemes, handled internally by the browser, offer a way to retrieve arbitrary content using a particular transport protocol and then display it using common, browser-level rendering logic. This is the most rudimentary and the most expected function of a URL.

The list of commonly supported schemes in this category is surprisingly short: *http:* (RFC 2616), the primary transport mode used on the Web and the focus of the next chapter of this book; *https:*, an encrypted version of HTTP (RFC 2818¹²); and *ftp:*, an older file transfer protocol (RFC 959¹³). All browsers also support *file:* (previously also known as *local:*), a system-specific method for accessing the local filesystem or NFS and SMB shares. (This last scheme is usually not directly accessible through Internet-originating pages, though.)

Two additional, obscure cases also deserve a brief mention: built-in support for the *gopher:* scheme, one of the failed predecessors of the Web (RFC 1436¹⁴), which is still present in Firefox, and *shhttp:*, an alternative, failed take on HTTPS (RFC 2660¹⁵), still recognized in Internet Explorer (but today, simply aliased to HTTP).

Protocols Claimed by Third-Party Applications and Plug-ins

For these schemes, matching URLs are simply dispatched to external, specialized applications that implement functionality such as media playback, document viewing, or IP telephony. At this point, the involvement of the browser (mostly) ends.

Scores of external protocol handlers exist today, and it would take another thick book to cover them all. Some of the most common examples include the *acrobat:* scheme, predictably routed to Adobe Acrobat Reader; *callto:* and *sip:* schemes claimed by all sorts of instant messengers and telephony software; *daap:*, *itpc:*, and *itms:* schemes used by Apple iTunes; *mailto:*, *news:*, and *nntp:* protocols claimed by mail and Usenet clients; *mmst:*, *mmsu:*, *msbd:*, and *rtsp:* protocols for streaming media players; and so on. Browsers are sometimes also included on the list. The previously mentioned *firefoxurl:* scheme launches Firefox from within another browser, while *cf:* gives access to Chrome from Internet Explorer.

For the most part, when these schemes appear in URLs, they usually have no impact on the security of the web applications that allow them to go through (although this is not guaranteed, especially in the case of plug-in-supported content). It is worth noting that third-party protocol handlers tend to be notoriously buggy and are sometimes abused to compromise the

operating system. Therefore, restricting the ability to navigate to mystery protocols is a common courtesy to the user of any reasonably trustworthy website.

Nonencapsulating Pseudo-Protocols

An array of protocols is reserved to provide convenient access to the browser's scripting engine and other internal functions, without actually retrieving any remote content and perhaps without establishing an isolated document context to display the result. Many of these pseudo-protocols are highly browser-specific and are either not directly accessible from the Internet or are incapable of doing harm. However, there are several important exceptions to this rule.

Perhaps the best-known exception is the *javascript:* scheme (in earlier years, also available under aliases such as *livescript:* or *mocha:* in Netscape browsers). This scheme gives access to the JavaScript-programming engine in the context of the currently viewed website. In Internet Explorer, *vbscript:* offers similar capabilities through the proprietary Visual Basic interface.

Another important case is the *data:* protocol (RFC 2397¹⁶), which permits short, inline documents to be created without any extra network requests and sometimes inherits much of their operating context from the referring page. An example of a *data:* URL is:

```
data:text/plain,Why,%20hello%20there!
```

These externally accessible pseudo-URLs are of acute significance to site security. When navigated to, their payload may execute in the context of the originating domain, possibly stealing sensitive data or altering the appearance of the page for the affected user. We'll discuss the specific capabilities of browser scripting languages in Chapter 6, but as you might expect, they are substantial. (URL context inheritance rules, on the other hand, are the focus of Chapter 10.)

Encapsulating Pseudo-Protocols

This special class of pseudo-protocols may be used to prefix any other URL in order to force a special decoding or rendering mode for the retrieved resource. Perhaps the best-known example is the *view-source:* scheme supported by Firefox and Chrome, used to display the pretty-printed source of an HTML page. This scheme is used in the following way:

```
view-source:http://www.example.com/
```

Other protocols that function similarly include *jar:*, which allows content to be extracted from ZIP files on the fly in Firefox; *wyciwyg:* and *view-cache:*, which give access to cached pages in Firefox and Chrome respectively; an oddball *feed:* scheme, which is meant to access news feeds in Safari;¹⁷ and a host of poorly documented protocols associated with the Windows help subsystem and other components of Microsoft Windows (*hcp:*, *its:*, *mhtml:*, *mk:*, *ms-help:*, *ms-its:*, and *ms-itss:*).

The common property of many encapsulating protocols is that they allow the attacker to hide the actual URL that will be ultimately interpreted by the browser from naïve filters: `view-source:javascript:` (or even `view-source:view-source:javascript:`) followed by malicious code is a simple way to accomplish this. Some security restrictions may be present to limit such trickery, but they should not be relied upon. Another significant problem, recurring especially with Microsoft's *mhtml:*, is that using the protocol may ignore some of the content directives provided by the server on HTTP level, possibly leading to widespread misery.¹⁸

Closing Note on Scheme Detection

The sheer number of pseudo-protocols is the primary reason why web applications need to carefully screen user-supplied URLs. The wonky and browser-specific URL-parsing patterns, coupled with the open-ended nature of the list of supported schemes, means that it is unsafe to simply blacklist known bad schemes; for example, a check for *javascript:* may be circumvented if this keyword is spliced with a tab or a newline, replaced with *vbscript:*, or prefixed with another encapsulating scheme.

Resolution of Relative URLs

Relative URLs have been mentioned on several occasions earlier in the chapter, and they deserve some additional attention at this point, too. The reason for their existence is that on almost every web page on the Internet, a considerable number of URLs will reference resources hosted on that same server, perhaps in the same directory. It would be inconvenient and wasteful to require a fully qualified URL to appear in the document every time such a reference is needed, so short, relative URLs (such as *../other_file.txt*) are used instead. The missing details are inferred from the URL of the referring document.

Because relative URLs are allowed to appear in exactly the same scenarios in which any absolute URL may appear, a method to distinguish between the two is necessary within the browser. Web applications also benefit from the ability to make the distinction, because most types of URL filters may want to scrutinize absolute URLs only and allow local references through as is.

The specification may make this task seem very simple: If the URL string does not begin with a valid scheme name followed by a semicolon and, preferably, a valid `“//”` sequence, it should be interpreted as a relative reference. And if no context for parsing such a relative URL exists, it should be rejected. Everything else is a safe relative link, right?

Predictably, it's not as easy as it seems. First, as outlined in previous sections, the accepted set of characters in a valid scheme name, and the patterns accepted in lieu of `“//”`, vary from one implementation to another. Perhaps more interestingly, it is a common misconception that relative links can point only to resources on the same server; quite a few other, less-obvious variants of relative URLs exist.

Let's have a quick peek at the known classes of relative URLs to better illustrate this possibility.

Scheme, but no authority present (*http:foo.txt*)

This infamous loophole is hinted at in RFC 3986 and attributed to an oversight in one of the earlier specs. While said specs descriptively classified such URLs as (invalid) absolute references, they also provided a promiscuous reference-parsing algorithm keen on interpreting them incorrectly.

In the latter interpretation, these URLs would set a new protocol and path, query, or fragment ID but have the authority section copied over from the referring location. This syntax is accepted by several browsers, but inconsistently. For example, in some cases, *http:foo.txt* may be treated as a relative reference, while *https:example.com* may be parsed as an absolute one!

No scheme, but authority present (*//example.com*)

This is another notoriously confusing but at least well-documented quirk. While */example.com* is a reference to a local resource on the current server, the standard compels browsers to treat *//example.com* as a very different case: a reference to a different authority over the current protocol. In this scenario, the scheme will be copied over from the referring location, and all other URL details will be derived from the relative URL.

No scheme, no authority, but path present (*../notes.txt*)

This is the most common variant of a relative link. Protocol and authority information is copied over from the referring URL. If the relative URL does not start with a slash, the path will also be copied over up to the rightmost *"/*. For example, if the base URL is *http://www.example.com/files/*, the path is the same, but in *http://www.example.com/files/index.html*, the filename is truncated. The new path is then appended, and standard path normalization follows on the concatenated value. The query string and fragment ID are derived only from the relative URL.

No scheme, no authority, no path, but query string present (*?search=bunnies*)

In this scenario, protocol, authority, and path information are copied verbatim from the referring URL. The query string and fragment ID are derived from the relative URL.

Only fragment ID present (*#bunnies*)

All information except for the fragment ID is copied verbatim from the referring URL; only the fragment ID is substituted. Following this type of relative URL does not cause the page to be reloaded under normal circumstances, as noted earlier.

Because of the risk of potential misunderstandings between application-level URL filters and the browser when handling these types of relative references, it is a good design practice never to output user-supplied relative URLs verbatim. Where feasible, they should be explicitly rewritten to absolute references, and all security checks should be carried out against the resulting fully qualified address instead.

Security Engineering Cheat Sheet

When Constructing Brand-New URLs Based on User Input

- ☑ **If you allow user-supplied data in path, query, or fragment ID:** If one of the section delimiters manages to get through without proper escaping, the URL may have a different effect from what you intended (for example, linking one of the user-visible HTML buttons to the wrong server-side action). It is okay to err on the side of caution: When inserting an attacker-controlled field value, you can simply percent-escape everything but alphanumerics.
- ☑ **If you allow user-supplied scheme name or authority section:** This is a major code injection and phishing risk! Apply the relevant input-validation rules outlined below.

When Designing URL Input Filters

- ☑ **Relative URLs:** Disallow or explicitly rewrite them to absolute references to avoid trouble. Anything else is very likely unsafe.
- ☑ **Scheme name:** Permit only known prefixes, such as *http://*, *https://*, or *ftp://*. Do not use blacklisting instead; it is extremely unsafe.
- ☑ **Authority section:** Hostname should contain only alphanumerics, “-”, and “.” and can only be followed by “/”, “?”, “#”, or end-of-string. Allowing anything else will backfire. If you need to examine the hostname, make sure to make a proper right-hand substring match.

In rare cases, you might need to account for IDNA, IPv6 bracket notation, port numbers, or HTTP credentials in the URL. If so, you must fully parse the URL, validate all sections and reject anomalous values, and reserialize them into a nonambiguous, canonical, well-escaped representation.

When Decoding Parameters Received Through URLs

- ☑ Do not assume that any particular character will be escaped just because the standard says so or because your browser does it. Before echoing back any URL-derived values or putting them inside database queries, new URLs, and so on, scrub them carefully for dangerous characters.

3

HYPERTEXT TRANSFER PROTOCOL

The next essential concept we need to discuss is the Hypertext Transfer Protocol (HTTP): the core transfer mechanism of the Web and the preferred method for exchanging URL-referenced documents between servers and clients. Despite having hypertext in its name, HTTP and the actual hypertext content (the HTML language) often exist independent of each other. That said, they are intertwined in sometimes surprising ways.

The history of HTTP offers interesting insight into its authors' ambitions and the growing relevance of the Internet. Tim Berners-Lee's earliest 1991 draft of the protocol (HTTP/0.9¹) was barely one and a half pages long, and it failed to account for even the most intuitive future needs, such as extensibility needed to transmit non-HTML data.

Five years and several iterations of the specification later, the first official HTTP/1.0 standard (RFC 1945²) tried to rectify many of these shortcomings in about 50 densely packed pages of text. Fast-forward to 1999, and in HTTP/1.1 (RFC 2616³), the seven credited authors attempted to anticipate almost every possible use of the protocol, creating an opus over 150 pages long. That's not all: As of this writing, the current work on HTTPbis,⁴ essentially a replacement for the HTTP/1.1 specification, comes to 360 pages or so. While much of the gradually accumulated content is irrelevant to the modern Web, this progression makes it clear that the desire to tack on new features far outweighs the desire to prune failed ones.

Today, all clients and servers support a not-entirely-accurate superset of HTTP/1.0, and most can speak a reasonably complete dialect of HTTP/1.1, with a couple of extensions bolted on. Despite the fact that there is no practical need to do so, several web servers, and all common browsers, also maintain backward compatibility with HTTP/0.9.

Basic Syntax of HTTP Traffic

At a glance, HTTP is a fairly simple, text-based protocol built on top of TCP/IP.* Every HTTP session is initiated by establishing a TCP connection to the server, typically to port 80, and then issuing a request that outlines the requested URL. In response, the server returns the requested file and, in the most rudimentary use case, tears down the TCP connection immediately thereafter.

The original HTTP/0.9 protocol provided no room for any additional metadata to be exchanged between the participating parties. The client request always consisted of a single line, starting with GET, followed by the URL path and query string, and ending with a single CRLF newline (ASCII characters 0x0D 0x0A; servers were also advised to accept a lone LF). A sample HTTP/0.9 request might have looked like this:

```
GET /fuzzy_bunnies.txt
```

In response to this message, the server would have immediately returned the appropriate HTML payload. (The specification required servers to wrap lines of the returned document at 80 characters, but this advice wasn't really followed.)

The HTTP/0.9 approach has a number of substantial deficiencies. For example, it offers no way for browsers to communicate users' language preferences, supply a list of supported document types, and so on. It also gives servers no way to tell a client that the requested file could not be found, that it has moved to a different location, or that the returned file is not an HTML

* *Transmission Control Protocol (TCP)* is one of the core communications protocols of the Internet, providing the transport layer to any application protocols built on top of it. TCP offers reasonably reliable, peer-acknowledged, ordered, session-based connectivity between networked hosts. In most cases, the protocol is also fairly resilient against blind packet spoofing attacks attempted by other, nonlocal hosts on the Internet.

document to begin with. Finally, the scheme is not kind to server administrators: When the transmitted URL information is limited to only the path and query strings, it is impossible for a server to host multiple websites, distinguished by their hostnames, under one IP address—and unlike DNS records, IP addresses don't come cheap.

In order to fix these shortcomings (and to make room for future tweaks), HTTP/1.0 and HTTP/1.1 standards embrace a slightly different conversation format: The first line of a request is modified to include protocol version information, and it is followed by zero or more *name: value* pairs (also known as *headers*), each occupying a separate line. Common request headers included in such requests are *User-Agent* (browser version information), *Host* (URL hostname), *Accept* (supported MIME document types*), *Accept-Language* (supported language codes), and *Referer* (a misspelled field indicating the originating page for the request, if known).

These headers are terminated with a single empty line, which may be followed by any payload the client wishes to pass to the server (the length of which must be explicitly specified with an additional *Content-Length* header). The contents of the payload are opaque from the perspective of the protocol itself; in HTML, this location is commonly used for submitting form data in one of several possible formats, though this is in no way a requirement.

Overall, a simple HTTP/1.1 request may look like this:

```
POST /fuzzy_bunnies/bunny_dispenser.php HTTP/1.1
Host: www.fuzzybunnies.com
User-Agent: Bunny-Browser/1.7
Content-Type: text/plain
Content-Length: 17
Referer: http://www.fuzzybunnies.com/main.html
```

I REQUEST A BUNNY

The server is expected to respond to this query by opening with a line that specifies the supported protocol version, a numerical status code (used to indicate error conditions and other special circumstances), and an optional, human-readable status message. A set of self-explanatory headers comes next, ending with an empty line. The response continues with the contents of the requested resource:

```
HTTP/1.1 200 OK
Server: Bunny-Server/0.9.2
Content-Type: text/plain
Connection: close
```

BUNNY WISH HAS BEEN GRANTED

* MIME type (aka *Internet media type*) is a simple, two-component value identifying the class and format of any given computer file. The concept originated in RFC 2045 and RFC 2046, where it served as a way to describe email attachments. The registry of official values (such as *text/plain* or *audio/mpeg*) is currently maintained by IANA, but ad hoc types are fairly common.

RFC 2616 also permits the response to be compressed in transit using one of three supported methods (*gzip*, *compress*, *deflate*), unless the client explicitly opts out by providing a suitable *Accept-Encoding* header.

The Consequences of Supporting HTTP/0.9

Despite the improvements made in HTTP/1.0 and HTTP/1.1, the unwelcome legacy of the “dumb” HTTP/0.9 protocol lives on, even if it is normally hidden from view. The specification for HTTP/1.0 is partly to blame for this, because it requested that all future HTTP clients and servers support the original, half-baked draft. Specifically, section 3.1 says:

HTTP/1.0 clients must . . . understand any valid response in the format of HTTP/0.9 or HTTP/1.0.

In later years, RFC 2616 attempted to backtrack on this requirement (section 19.6: “It is beyond the scope of a protocol specification to mandate compliance with previous versions.”), but acting on the earlier advice, all modern browsers continue to support the legacy protocol as well.

To understand why this pattern is dangerous, recall that HTTP/0.9 servers reply with nothing but the requested file. There is no indication that the responding party actually understands HTTP and wishes to serve an HTML document. With this in mind, let’s analyze what happens if the browser sends an HTTP/1.1 request to an unsuspecting SMTP service running on port 25 of *example.com*:

```
GET /<html><body><h1>Hi! HTTP/1.1
Host: example.com:25
...
```

Because the SMTP server doesn’t understand what is going on, it’s likely to respond this way:

```
220 example.com ESMTP
500 5.5.1 Invalid command: "GET /<html><body><h1>Hi! HTTP/1.1"
500 5.1.1 Invalid command: "Host: example.com:25"
...
421 4.4.1 Timeout
```

All browsers willing to follow the RFC are compelled to accept these messages as the body of a valid HTTP/0.9 response and assume that the returned document is, indeed, HTML. These browsers will interpret the quoted attacker-controlled snippet appearing in one of the error messages as if it comes from the owners of a legitimate website at *example.com*. This profoundly interferes with the browser security model discussed in Part II of this book and, therefore, is pretty bad.

Newline Handling Quirks

Setting aside the radical changes between HTTP/0.9 and HTTP/1.0, several other core syntax tweaks were made later in the game. Perhaps most notably, contrary to the letter of earlier iterations, HTTP/1.1 asks clients not only to honor newlines in the CRLF and LF format but also to recognize a lone CR character. Although this recommendation is disregarded by the two most popular web servers (IIS and Apache), it is followed on the client side by all browsers except Firefox.

The resulting inconsistency makes it easier for application developers to forget that not only LF but also CR characters must be stripped from any attacker-controlled values that appear anywhere in HTTP headers. To illustrate the problem, consider the following server response, where a user-supplied and insufficiently sanitized value appears in one of the headers, as highlighted in bold:

```
HTTP/1.1 200 OK[CR][LF]
Set-Cookie: last_search_term=[CR][CR]<html><body><h1>Hi![CR][LF]
[CR][LF]
Action completed.
```

To Internet Explorer, this response may appear as:

```
HTTP/1.1 200 OK
Set-Cookie: last_search_term=
<html><body><h1>Hi!
Action completed.
```

In fact, the class of vulnerabilities related to HTTP header newline smuggling—be it due to this inconsistency or just due to a failure to filter any type of a newline—is common enough to have its own name: *header injection* or *response splitting*.

Another little-known and potentially security-relevant tweak is support for multiline headers, a change introduced in HTTP/1.1. According to the standard, any header line that begins with a whitespace is treated as a continuation of the previous one. For example:

```
X-Random-Comment: This is a very long string,
so why not wrap it neatly?
```

Multiline headers are recognized in client-issued requests by IIS and Apache, but they are not supported by Internet Explorer, Safari, or Opera. Therefore, any implementation that relies on or simply permits this syntax in any attacker-influenced setting may be in trouble. Thankfully, this is rare.

Proxy Requests

Proxies are used by many organizations and Internet service providers to intercept, inspect, and forward HTTP requests on behalf of their users. This may be done to improve performance (by allowing certain server responses to be cached on a nearby system), to enforce network usage policies (for example, to prevent access to porn), or to offer monitored and authenticated access to otherwise separated network environments.

Conventional HTTP proxies depend on explicit browser support: The application needs to be configured to make a modified request to the proxy system, instead of attempting to talk to the intended destination. To request an HTTP resource through such a proxy, the browser will normally send a request like this:

```
GET http://www.fuzzybunnies.com/ HTTP/1.1
User-Agent: Bunny-Browser/1.7
Host: www.fuzzybunnies.com
...
```

The key difference between the above example and the usual syntax is the presence of a fully qualified URL in the first line of the request (*http://www.fuzzybunnies.com/*), instructing the proxy where to connect to on behalf of the user. This information is somewhat redundant, given that the *Host* header already specifies the hostname; the only reason for this overlap is that the mechanisms evolved independent of each other. To avoid being fooled by co-conspiring clients and servers, proxies should either correct any mismatching *Host* headers to match the request URL or associate cached content with a particular URL-*Host* pair and not just one of these values.

Many HTTP proxies also allow browsers to request non-HTTP resources, such as FTP files or directories. In these cases, the proxy will wrap the response in HTTP, and perhaps convert it to HTML if appropriate, before returning it to the user.* That said, if the proxy does not understand the requested protocol, or if it is simply inappropriate for it to peek into the exchanged data (for example, inside encrypted sessions), a different approach must be used. A special type of a request, CONNECT, is reserved for this purpose but is not further explained in the HTTP/1.1 RFC. The relevant request syntax is instead outlined in a separate, draft-only specification from 1998.⁵ It looks like this:

```
CONNECT www.fuzzybunnies.com:1234 HTTP/1.1
User-Agent: Bunny-Browser/1.7
...
```

* In this case, some HTTP headers supplied by the client may be used internally by the proxy, but they will not be transmitted to the non-HTTP endpoint, which creates some interesting, if non-security-relevant, protocol ambiguities.

If the proxy is willing and able to connect to the requested destination, it acknowledges this request with a specific HTTP response code, and the role of this protocol ends. At that point, the browser will begin sending and receiving raw binary data within the established TCP stream; the proxy, in turn, is expected to forward the traffic between the two endpoints indiscriminately.

NOTE *Hilariously, due to a subtle omission in the draft spec, many browsers have incorrectly processed the nonencrypted, proxy-originating error responses returned during an attempt to establish an encrypted connection. The affected implementations interpreted such plaintext responses as though they originated from the destination server over a secure channel. This glitch effectively eliminated all assurances associated with the use of encrypted communications on the Web. It took over a decade to spot and correct the flaw.⁶*

Several other classes of lower-level proxies do not use HTTP to communicate directly with the browser but nevertheless inspect the exchanged HTTP messages to cache content or enforce certain rules. The canonical example of this is a transparent proxy that silently intercepts traffic at the TCP/IP level. The approach taken by transparent proxies is unusually dangerous: Any such proxy can look at the destination IP and the *Host* header sent in the intercepted connection, but it has no way of immediately telling if that destination IP is genuinely associated with the specified server name. Unless an additional lookup and correlation is performed, co-conspiring clients and servers can have a field day with this behavior. Without these additional checks, the attacker simply needs to connect to his or her home server and send a misleading *Host: www.google.com* header to have the response cached for all other users as though genuinely coming from *www.google.com*.

Resolution of Duplicate or Conflicting Headers

Despite being relatively verbose, RFC 2616 does a poor job of explaining how a compliant parser should resolve potential ambiguities and conflicts in the request or response data. Section 19.2 of this RFC (“Tolerant Applications”) recommends relaxed and error-tolerant parsing of certain fields in “unambiguous” cases, but the meaning of the term itself is, shall we say, not particularly unambiguous.

For example, because of a lack of specification-level advice, roughly half of all browsers will favor the first occurrence of a particular HTTP header, and the rest will favor the last one, ensuring that almost every header injection vulnerability, no matter how constrained, is exploitable for at least some percentage of targeted users. On the server side, the situation is similarly random: Apache will honor the first *Host* header seen, while IIS will completely reject a request with multiple instances of this field.

On a related note, the relevant RFCs contain no explicit prohibition on mixing potentially conflicting HTTP/1.0 and HTTP/1.1 headers and no requirement for HTTP/1.0 servers or clients to ignore all HTTP/1.1 syntax. Because of this design, it is difficult to predict the outcome of indirect conflicts between HTTP/1.0 and HTTP/1.1 directives that are responsible for the same thing, such as *Expires* and *Cache-Control*.

Finally, in some rare cases, header conflict resolution is outlined in the spec very clearly, but the purpose of permitting such conflicts to arise in the first place is much harder to understand. For example, HTTP/1.1 clients are required to send the *Host* header on all requests, but servers (not just proxies!) are also required to recognize absolute URLs in the first line of the request, as opposed to the traditional path- and query-only method. This rule permits a curiosity such as this:

```
GET http://www.fuzzybunnies.com/ HTTP/1.1
Host: www.bunnyoutlet.com
```

In this case, section 5.2 of RFC 2616 instructs clients to disregard the nonfunctional (but still mandatory!) *Host* header, and many implementations follow this advice. The problem is that underlying applications are likely to be unaware of this quirk and may instead make somewhat important decisions based on the inspected header value.

NOTE *When complaining about the omissions in the HTTP RFCs, it is important to recognize that the alternatives can be just as problematic. In several scenarios outlined in that RFC, the desire to explicitly mandate the handling of certain corner cases led to patently absurd outcomes. One such example is the advice on parsing dates in certain HTTP headers, at the request of section 3.3 in RFC 1945. The resulting implementation (the `prtime.c` file in the Firefox codebase⁷) consists of close to 2,000 lines of extremely confusing and unreadable C code just to decipher the specified date, time, and time zone in a sufficiently fault-tolerant way (for uses such as deciding cache content expiration).*

Semicolon-Delimited Header Values

Several HTTP headers, such as *Cache-Control* or *Content-Disposition*, use a semicolon-delimited syntax to cram several separate *name=value* pairs into a single line. The reason for allowing this nested notation is unclear, but it is probably driven by the belief that it will be a more efficient or a more intuitive approach than using several separate headers that would always have to go hand in hand.

Some use cases outlined in RFC 2616 permit *quoted-string* as the right-hand parameter in such pairs. *Quoted-string* is a syntax in which a sequence of arbitrary printable characters is surrounded by double quotes, which act as delimiters. Naturally, the quote mark itself cannot appear inside the string, but—importantly—a semicolon or a whitespace may, permitting many otherwise problematic values to be sent as is.

Unfortunately for developers, Internet Explorer does not cope with the *quoted-string* syntax particularly well, effectively rendering this encoding scheme useless. The browser will parse the following line (which is meant to indicate that the response is a downloadable file rather than an inline document) in an unexpected way:

```
Content-Disposition: attachment; filename="evil_file.exe;.txt"
```

In Microsoft's implementation, the filename will be truncated at the semicolon character and will appear to be *evil_file.exe*. This behavior creates a potential hazard to any application that relies on examining or appending a "safe" filename extension to an attacker-controlled filename and otherwise correctly checks for the quote character and newlines in this string.

NOTE *An additional quoted-pair mechanism is provided to allow quotes (and any other characters) to be used safely in the string when prefixed by a backslash. This mechanism appears to be specified incorrectly, however, and not supported by any major browser except for Opera. For quoted-pair to work properly, stray "\ characters would need to be banned from the quoted-string, which isn't the case in RFC 2616. Quoted-pair also permits any CHAR-type token to be quoted, including newlines, which is incompatible with other HTTP-parsing rules.*

It is also worth noting that when duplicate semicolon-delimited fields are found in a single HTTP header, their order of precedence is not defined by the RFC. In the case of *filename=* in *Content-Disposition*, all mainstream browsers use the first occurrence. But there is little consistency elsewhere. For example, when extracting the *URL=* value from the *Refresh* header (used to force reloading the page after a specified amount of time), Internet Explorer 6 will fall back to the last instance, yet all other browsers will prefer the first one. And when handling *Content-Type*, Internet Explorer, Safari, and Opera will use the first *charset=* value, while Firefox and Chrome will rely on the last.

NOTE *Food for thought: A fascinating but largely non-security-related survey of dozens of inconsistencies associated with the handling of just a single HTTP header—Content-Disposition—can be found on a page maintained by Julian Reschke: <http://greenbytes.de/tech/tc2231/>.*

Header Character Set and Encoding Schemes

Like the documents that laid the groundwork for URL handling, all subsequent HTTP specs have largely avoided the topic of dealing with non-US-ASCII characters inside header values. There are several plausible scenarios where non-English text may legitimately appear in this context (for example, the filename in *Content-Disposition*), but when it comes to this, the expected browser behavior is essentially undefined.

Originally, RFC 1945 permitted the TEXT token (a primitive broadly used to define the syntax of other fields) to contain 8-bit characters, providing the following definition:

OCTET	= <any 8-bit sequence of data>
CTL	= <any US-ASCII control character (octets 0 - 31) and DEL (127)>
TEXT	= <any OCTET except CTLs, but including LWS>

The RFC followed up with cryptic advice: When non-US-ASCII characters are encountered in a TEXT field, clients and servers *may* interpret them as ISO-8859-1, the standard Western European code page, but they don't have to. Later, RFC 2616 copied and pasted the same specification of TEXT tokens but added a note that non-ISO-8859-1 strings must be encoded using a format outlined in RFC 2047,⁸ originally created for email communications. Fair enough; in this simple scheme, the encoded string opens with a “=?” prefix, followed by a character-set name, a “?q?” or “?b?” encoding-type indicator (*quoted-printable*^{*} or *base64*,[†] respectively), and lastly the encoded string itself. The sequence ends with a “?” terminator. An example of this may be:

Content-Disposition: attachment; filename="=?utf-8?q?Hi=21.txt?="

NOTE *The RFC should also have stated that any spurious “=?...?” patterns must never be allowed as is in the relevant headers, in order to avoid unintended decoding of values that were not really encoded to begin with.*

Sadly, the support for this RFC 2047 encoding is spotty. It is recognized in some headers by Firefox and Chrome, but other browsers are less cooperative. Internet Explorer chooses to recognize URL-style percent encoding in the *Content-Disposition* field instead (a habit also picked up by Chrome) and defaults to UTF-8 in this case. Firefox and Opera, on the other hand, prefer supporting a peculiar percent-encoded syntax proposed in RFC 2231,⁹ a striking deviation from how HTTP syntax is supposed to look:

Content-Disposition: attachment; filename*=utf-8'en-us'Hi%21.txt

Astute readers may notice that there is no single encoding scheme supported by all browsers at once. This situation prompts some web application developers to resort to using raw high-bit values in the HTTP headers, typically interpreted as UTF-8, but doing so is somewhat unsafe. In Firefox, for example, a long-standing glitch causes UTF-8 text to be mangled when put

^{*} *Quoted-printable* is a simple encoding scheme that replaces any nonprintable or otherwise illegal characters with the equal sign (=) followed by a 2-digit hexadecimal representation of the 8-bit character value to be encoded. Any stray equal signs in the input text must be replaced with “=3D” as well.

[†] *Base64* is a non-human-readable encoding that encodes arbitrary 8-bit input using a 6-bit alphabet of case-sensitive alphanumerics, “+”, and “/”. Every 3 bytes of input map to 4 bytes of output. If the input does not end at a 3-byte boundary, this is indicated by appending one or two equal signs at the end of the output string.

in the *Cookie* header, permitting attacker-injected cookie delimiters to materialize in unexpected places.¹⁰ In other words, there are no easy and robust solutions to this mess.

When discussing character encodings, the problem of handling of the NUL character (0x00) probably deserves a mention. This character, used as a string terminator in many programming languages, is technically prohibited from appearing in HTTP headers (except for the aforementioned, dysfunctional *quoted-pair* syntax), but as you may recall, parsers are encouraged to be tolerant. When this character is allowed to go through, it is likely to have unexpected side effects. For example, *Content-Disposition* headers are truncated at NUL by Internet Explorer, Firefox, and Chrome but not by Opera or Safari.

Referer Header Behavior

As mentioned earlier in this chapter, HTTP requests may include a *Referer* header. This header contains the URL of a document that triggered the current navigation in some way. It is meant to help with certain troubleshooting tasks and to promote the growth of the Web by emphasizing cross-references between related web pages.

Unfortunately, the header may also reveal some information about user browsing habits to certain unfriendly parties, and it may leak sensitive information that is encoded in the URL query parameters on the referring page. Due to these concerns, and the subsequent poor advice on how to mitigate them, the header is often misused for security or policy enforcement purposes, but it is not up to the task. The main problem is that there is no way to differentiate between a client that is not providing the header because of user privacy preferences, one that is not providing it because of the type of navigation taking place, and one that is deliberately tricked into hiding this information by a malicious referring site.

Normally, this header is included in most HTTP requests (and preserved across HTTP-level redirects), except in the following scenarios:

- After organically entering a new URL into the address bar or opening a bookmarked page.
- When the navigation originates from a pseudo-URL document, such as *data:* or *javascript:*.
- When the request is a result of redirection controlled by the *Refresh* header (but not a *Location*-based one).
- Whenever the referring site is encrypted but the requested page isn't. According to RFC 2616 section 15.1.2, this is done for privacy reasons, but it does not make a lot of sense. The *Referer* string is still disclosed to third parties when one navigates from one encrypted domain to an unrelated encrypted one, and rest assured, the use of encryption is not synonymous with trustworthiness.
- If the user decides to block or spoof the header by tweaking browser settings or installing a privacy-oriented plug-in.

As should be apparent, four out of five of these conditions can be purposefully induced by any rogue site.

HTTP Request Types

The original HTTP/0.9 draft provided a single method (or “verb”) for requesting a document: GET. The subsequent proposals experimented with an increasingly bizarre set of methods to permit interactions other than retrieving a document or running a script, including such curiosities as SHOWMETHOD, CHECKOUT, or—why not—SPACEJUMP.¹¹

Most of these thought experiments have been abandoned in HTTP/1.1, which settles on a more manageable set of eight methods. Only the first two request types—GET and POST—are of any significance to most of the modern Web.

GET

The GET method is meant to signify information retrieval. In practice, it is used for almost all client-server interactions in the course of a normal browsing session. Regular GET requests carry no browser-supplied payloads, although they are not strictly prohibited from doing so.

The expectation is that GET requests should not have, to quote the RFC, “significance of taking an action other than retrieval” (that is, they should make no persistent changes to the state of the application). This requirement is increasingly meaningless in modern web applications, where the application state is often not even managed entirely on the server side; consequently, the advice is widely ignored by application developers.*

NOTE *In HTTP/1.1, clients may ask the server for any set of possibly noncontiguous or overlapping fragments of the target document by specifying the Range header on GET (and, less commonly, on some other types of requests). The server is not obliged to comply, but where the mechanism is available, browsers may use it to resume aborted downloads.*

POST

The POST method is meant for submitting information (chiefly HTML forms) to the server for processing. Because POST actions may have persistent side effects, many browsers ask the user to confirm before reloading any content retrieved with POST, but for the most part, GET and POST are used in a quasi-interchangeable manner.

POST requests are commonly accompanied by a payload, the length of which is indicated by the *Content-Length* header. In the case of plain HTML, the payload may consist of URL-encoded or MIME-encoded form data (a format detailed in Chapter 4), although again, the syntax is not constrained at the HTTP level in any special way.

*There is an anecdotal (and perhaps even true) tale of an unfortunate webmaster by the name of John Breckman. According to the story, John’s website has been accidentally deleted by a search engine-indexing robot. The robot simply unwittingly discovered an unauthenticated, GET-based administrative interface that John had built for his site . . . and happily followed every “delete” link it could find.

HEAD

HEAD is a rarely used request type that is essentially identical to GET but that returns only the HTTP headers, and not the actual payload, for the requested content. Browsers generally do not issue HEAD requests on their own, but the method is sometimes employed by search engine bots and other automated tools, for example, to probe for the existence of a file or to check its modification time.

OPTIONS

OPTIONS is a metarequest that returns the set of supported methods for a particular URL (or “*”, meaning the server in general) in a response header. The OPTIONS method is almost never used in practice, except for server fingerprinting; because of its limited value, the returned information may not be very accurate.

NOTE *For the sake of completeness, we need to note that OPTIONS requests are also a cornerstone of a proposed cross-domain request authorization scheme, and as such, they may gain some prominence soon. We will revisit this scheme, and explore many other upcoming browser security features, in Chapter 16.*

PUT

A PUT request is meant to allow files to be uploaded to the server at the specified target URL. Because browsers do not support PUT, intentional file-upload capabilities are almost always implemented through POST to a server-side script, rather than with this theoretically more elegant approach.

That said, some nonweb HTTP clients and servers may use PUT for their own purposes. Just as interestingly, some web servers may be misconfigured to process PUT requests indiscriminately, creating an obvious security risk.

DELETE

DELETE is a self-explanatory method that complements PUT (and that is equally uncommon in practice).

TRACE

TRACE is a form of “ping” request that returns information about all the proxy hops involved in processing a request and echoes the original request as well. TRACE requests are not issued by web browsers and are seldom used for legitimate purposes. TRACE’s primary use is for security testing, where it may reveal interesting details about the internal architecture of HTTP servers in a remote network. Precisely for this reason, the method is often disabled by server administrators.

CONNECT

The CONNECT method is reserved for establishing non-HTTP connections through HTTP proxies. It is not meant to be issued directly to servers. If the support for CONNECT request is enabled accidentally on a particular server, it may pose a security risk by offering an attacker a way to tunnel TCP traffic into an otherwise protected network.

Other HTTP Methods

A number of other request methods may be employed by other nonbrowser applications or browser extensions; the most popular set of HTTP extensions may be WebDAV, an authoring and version-control protocol described in RFC 4918.¹²

Further, the *XMLHttpRequest* API nominally allows client-side JavaScript to make requests with almost arbitrary methods to the originating server—although this last functionality is heavily restricted in certain browsers (we will look into this in Chapter 9).

Server Response Codes

Section 10 of RFC 2616 lists nearly 50 status codes that a server may choose from when constructing a response. About 15 of these are used in real life, and the rest are used to indicate increasingly bizarre or unlikely states, such as “402 Payment Required” or “415 Unsupported Media Type.” Most of the RFC-listed states do not map cleanly to the behavior of modern web applications; the only reason for their existence is that somebody hoped they eventually would.

A few codes are worth memorizing because they are common or carry special meaning, as discussed below.

200–299: Success

This range of status codes is used to indicate a successful completion of a request:

200 OK This is a normal response to a successful GET or POST. The browser will display the subsequently returned payload to the user or will process it in some other context-specific way.

204 No Content This code is sometimes used to indicate a successful request to which no verbose response is expected. A 204 response aborts navigation to the URL that triggered it and keeps the user on the originating page.

206 Partial Content This code is like 200, except that it is returned by servers in response to range requests. The browser must already have a portion of the document (or it would not have issued a range request) and will normally inspect the *Content-Range* response header to reassemble the document before further processing it.

300–399: Redirection and Other Status Messages

These codes are used to communicate a variety of states that do not indicate an error but that require special handling on the browser end:

301 Moved Permanently, 302 Found, 303 See Other This response instructs the browser to retry the request at a new location, specified in the *Location* response header. Despite the distinctions made in the RFC, when encountering any of these response codes, all modern browsers replace POST with GET, remove the payload, and then resubmit the request automatically.

NOTE *Redirect messages may contain a payload, but if they do, this message will not be shown to the user unless the redirection is not possible (for example, because of a missing or unsupported Location value). In fact, in some browsers, display of the message may be suppressed even in that scenario.*

304 Not Modified This nonredirect response instructs the client that the requested document hasn't been modified in relation to the copy the client already has. This response is seen after conditional requests with headers such as *If-Modified-Since*, which are issued to revalidate the browser document cache. The response body is not shown to the user. (If the server responds this way to an unconditional request, the result will be browser-specific and may be hilarious; for example, Opera will pop up a nonfunctional download prompt.)

307 Temporary Redirect Similar to 302, but unlike with other modes of redirection, browsers will not downgrade POST to GET when following a 307 redirect. This code is not commonly used in web applications, and some browsers do not behave very consistently when handling it.

400–499: Client-Side Error

This range of codes is used to indicate error conditions caused by the behavior of the client:

400 Bad Request (and related messages) The server is unable or unwilling to process the request for some unspecified reason. The response payload will usually explain the problem to some extent and will be typically handled by the browser just like a 200 response.

More specific variants, such as “411 Length Required,” “405 Method Not Allowed,” or “414 Request-URI Too Long,” also exist. It's anyone's guess as to why not specifying *Content-Length* when required has a dedicated 411 response code but not specifying *Host* deserves only a generic 400 one.

401 Unauthorized This code means that the user needs to provide protocol-level HTTP authentication credentials in order to access the resource. The browser will usually prompt the user for login information next, and it will present a response body only if the authentication process is unsuccessful. This mechanism will be explained in more detail shortly, in “HTTP Authentication” on page 62.

403 Forbidden The requested URL exists but can't be accessed for reasons other than incorrect HTTP authentication. Reasons may involve insufficient filesystem permissions, a configuration rule that prevents this request from being processed, or insufficient credentials of some sort (e.g., invalid cookies or an unrecognized source IP address). The response will usually be shown to the user.

404 Not Found The requested URL does not exist. The response body is typically shown to the user.

500–599: Server-Side Error

This is a class of error messages returned in response to server-side problems:

500 Internal Server Error, 503 Service Unavailable, and so on The server is experiencing a problem that prevents it from fulfilling the request. This may be a transient condition, a result of misconfiguration, or simply the effect of requesting an unexpected location. The response is normally shown to the user.

Consistency of HTTP Code Signaling

Because there is no immediately observable difference between returning most 2xx, 4xx, and 5xx codes, these values are not selected with any special zeal. In particular, web applications are notorious for returning “200 OK” even when an application error has occurred and is communicated on the resulting page. (This is one of the many factors that make automated testing of web applications much harder than it needs to be.)

On rare occasions, new and not necessarily appropriate HTTP codes are invented for specific uses. Some of these are standardized, such as a couple of messages introduced in the WebDAV RFC.¹³ Others, such as Microsoft's Microsoft Exchange “449 Retry With” status, are not.

Keepalive Sessions

Originally, HTTP sessions were meant to happen in one shot: Make one request for each TCP connection, rinse, and repeat. The overhead of repeatedly completing a three-step TCP handshake (and forking off a new process in the traditional Unix server design model) soon proved to be a bottleneck, so HTTP/1.1 standardized the idea of keepalive sessions instead.

The existing protocol already gave the server an understanding of where the client request ended (an empty line, optionally followed by *Content-Length* bytes of data), but to continue using the existing connection, the client also needed to know the same about the returned document; the termination of a connection could no longer serve as an indicator. Therefore, keepalive sessions require the response to include a *Content-Length* header too, always specifying the amount of data to follow. Once this many payload bytes are received, the client knows it is okay to send a second request and begin waiting for another response.

Although very beneficial from a performance standpoint, the way this mechanism is designed exacerbates the impact of HTTP request and response-splitting bugs. It is deceptively easy for the client and the server to get out of sync on which response belongs to which request. To illustrate, let's consider a server that thinks it is sending a single HTTP response, structured as follows:

```
HTTP/1.1 200 OK[CR][LF]
Set-Cookie: term=[CR]Content-Length: 0[CR][CR]HTTP/1.1 200 OK[CR]Gotcha: Yup[CR][LF]
Content-Length: 17[CR][LF]
[CR][LF]
Action completed.
```

The client, on the other hand, may see two responses and associate the first one with its most current request and the second one with the yet-to-be-issued query* (which may even be addressed to a different hostname on the same IP):

```
HTTP/1.1 200 OK
Set-Cookie: term=
Content-Length: 0

HTTP/1.1 200 OK
Gotcha: Yup
Content-Length: 17

Action completed.
```

If this response is seen by a caching HTTP proxy, the incorrect result may also be cached globally and returned to other users, which is really bad news. A much safer design for keepalive sessions would involve specifying the length of both the headers and the payload up front or using a randomly generated and unpredictable boundary to delimit every response. Regrettably, the design does neither.

Keepalive connections are the default in HTTP/1.1 unless they are explicitly turned off (*Connection: close*) and are supported by many HTTP/1.0 servers when enabled with a *Connection: keep-alive* header. Both servers and browsers can limit the number of concurrent requests serviced per connection and can specify the maximum amount of time an idle connection is kept around.

Chunked Data Transfers

The significant limitation of *Content-Length*-based keepalive sessions is the need for the server to know in advance the exact size of the returned response. This is a pretty simple task when dealing with static files, as the

*In principle, clients could be designed to sink any unsolicited server response data before issuing any subsequent requests in a keepalive session, limiting the impact of the attack. This proposal is undermined by the practice of HTTP pipelining, however; for performance reasons, some clients are designed to dump multiple requests at once, without waiting for a complete response in between.

information is already available in the filesystem. When serving dynamically generated data, the problem is more complicated, as the output must be cached in its entirety before it is sent to the client. The challenge becomes insurmountable if the payload is very large or is produced gradually (think live video streaming). In these cases, precaching to compute payload size is simply out of the question.

In response to this challenge, RFC 2616 section 3.6.1 gives servers the ability to use *Transfer-Encoding: chunked*, a scheme in which the payload is sent in portions as it becomes available. The length of every portion of the document is declared up front using a hexadecimal integer occupying a separate line, but the total length of the document is indeterminate until a final zero-length chunk is seen.

A sample chunked response may look like this:

```
HTTP/1.1 200 OK
Transfer-Encoding: chunked
...

5
Hello
6
world!
0
```

There are no significant downsides to supporting chunked data transfers, other than the possibility of pathologically large chunks causing integer overflows in the browser code or needing to resolve mismatches between *Content-Length* and chunk length. (The specification gives precedence to chunk length, although any attempts to handle this situation gracefully appear to be ill-advised.) All the popular browsers deal with these conditions properly, but new implementations need to watch their backs.

Caching Behavior

For reasons of performance and bandwidth conservation, HTTP clients and some intermediaries are eager to cache HTTP responses for later reuse. This must have seemed like a simple task in the early days of the Web, but it is increasingly fraught with peril as the Web encompasses ever more sensitive, user-specific information and as this information is updated more and more frequently.

RFC 2616 section 13.4 states that GET requests responded to with a range of HTTP codes (most notably, “200 OK” and “301 Moved Permanently”) may be implicitly cached in the absence of any other server-provided directives. Such a response may be stored in the cache indefinitely, and may be reused for any future requests involving the same request method and destination URL, even if other parameters (such as *Cookie* headers) differ. There is a prohibition against caching requests that use HTTP authentication (see “HTTP Authentication” on page 62), but other authentication methods, such as cookies, are not recognized in the spec.

When a response is cached, the implementation may opt to revalidate it before reuse, but doing so is not required most of the time. Revalidation is achieved by request with a special conditional header, such as *If-Modified-Since* (followed by a date recorded on the previously cached response) or *If-None-Match* (followed by an opaque *ETag* header value that the server returned with an earlier copy). The server may respond with a “304 Not Modified” code or return a newer copy of the resource.

NOTE *The Date/If-Modified-Since and ETag/If-None-Match header pairs, when coupled with Cache-Control: private, offer a convenient and entirely unintended way for websites to store long-lived, unique tokens in the browser.¹⁴ The same can also be achieved by depositing a unique token inside a cacheable JavaScript file and returning “304 Not Modified” to all future conditional requests to the token-generating location. Unlike purpose-built mechanisms such as HTTP cookies (discussed in the next section), users have very little control over what information is stored in the browser cache, under what circumstances, and for how long.*

Implicit caching is highly problematic, and therefore, servers almost always should resort to using explicit HTTP-caching directives. To assist with this, HTTP/1.0 provides an *Expires* header that specifies the date by which the cached copy should be discarded; if this value is equal to the *Date* header provided by the server, the response is noncacheable. Beyond that simple rule, the connection between *Expires* and *Date* is unspecified: It is not clear whether *Expires* should be compared to the system clock on the caching system (which is problematic if the client and server clocks are not in sync) or evaluated based on the *Expires* – *Date* delta (which is more robust, but which may stop working if *Date* is accidentally omitted). Firefox and Opera use the latter interpretation, while other browsers prefer the former one. In most browsers, an invalid *Expires* value also inhibits caching, but depending on it is a risky bet.

HTTP/1.0 clients can also include a *Pragma: no-cache* request header, which may be interpreted by the proxy as an instruction to obtain a new copy of the requested resource, instead of returning an existing one. Some HTTP/1.0 proxies also recognize a nonstandard *Pragma: no-cache* response header as an instruction not to make a copy of the document.

In contrast, HTTP/1.1 embraces a far more substantial approach to caching directives, introducing a new *Cache-Control* header. The header takes values such as *public* (the document is cacheable publicly), *private* (proxies are not permitted to cache), *no-cache* (which is a bit confusing—the response may be cached but should not be reused for future requests),^{*} and *no-store* (absolutely no caching at all). Public and private caching directives may be accompanied with a qualifier such as *max-age*, specifying the maximum time an old copy should be kept, or *must-revalidate*, requesting a conditional request to be made before content reuse.

^{*} The RFC is a bit hazy in this regard, but it appears that the intent is to permit the cached document to be used for purposes such as operating the “back” and “forward” navigation buttons in a browser but not when a proper page load is requested. Firefox follows this approach, while all other browsers consider *no-cache* and *no-store* to be roughly equivalent.

Unfortunately, it is typically necessary for servers to return both HTTP/1.0 and HTTP/1.1 caching directives, because certain types of legacy commercial proxies do not understand *Cache-Control* correctly. In order to reliably prevent caching over HTTP, it may be necessary to use the following set of response headers:

```
Expires: [current date]
Date: [current date]
Pragma: no-cache
Cache-Control: no-cache, no-store
```

When these caching directives disagree, the behavior is difficult to predict: Some browsers will favor HTTP/1.1 directives and give precedence to *no-cache*, even if it is mistakenly followed by *public*; others don't.

Another risk of HTTP caching is associated with unsafe networks, such as public Wi-Fi networks, which allow an attacker to intercept requests to certain URLs and return modified, long-cacheable contents on requests to the victim. If such a poisoned browser cache is then reused on a trusted network, the injected content will unexpectedly resurface. Perversely, the victim does not even have to visit the targeted application: A reference to a carefully chosen sensitive domain can be injected by the attacker into some other context. There are no good solutions to this problem yet; purging your browser cache after visiting Starbucks may be a very good idea.

HTTP Cookie Semantics

HTTP cookies are not a part of RFC 2616, but they are one of the more important protocol extensions used on the Web. The cookie mechanism allows servers to store short, opaque *name=value* pairs in the browser by sending a *Set-Cookie* response header and to receive them back on future requests via the client-supplied *Cookie* parameter. Cookies are by far the most popular way to maintain sessions and authenticate user requests; they are one of the four canonical forms of *ambient authority** on the Web (the other forms being built-in HTTP authentication, IP checking, and client certificates).

Originally implemented in Netscape by Lou Montulli around 1994, and described in a brief four-page draft document,¹⁵ the mechanism has not been outlined in a proper standard in the last 17 years. In 1997, RFC 2109¹⁶ attempted to document the status quo, but somewhat inexplicably, it also proposed a number of sweeping changes that, to this day, make this specification substantially incompatible with the actual behavior of any modern browser. Another ambitious effort—*Cookie2*—made an appearance in RFC 2965,¹⁷ but a decade later, it still has virtually no browser-level support, a situation that is

* *Ambient authority* is a form of access control based on a global and persistent property of the requesting entity, rather than any explicit form of authorization that would be valid only for a specific action. A user-identifying cookie included indiscriminately on every outgoing request to a remote site, without any consideration for why this request is being made, falls into that category.

unlikely to change. A new effort to write a reasonably accurate cookie specification—RFC 6265¹⁸—was wrapped up shortly before the publication of this book, finally ending this specification-related misery.

Because of the prolonged absence of any real standards, the actual implementations evolved in very interesting and sometimes incompatible ways. In practice, new cookies can be set using *Set-Cookie* headers followed by a single *name=value* pair and a number of optional semicolon-delimited parameters defining the scope and lifetime of the cookie.

Expires Specifies the expiration date for a cookie in a format similar to that used for *Date* or *Expires* HTTP headers. If a cookie is served without an explicit expiration date, it is typically kept in memory for the duration of a browser session (which, especially on portable computers with suspend functionality, can easily span several weeks). Definite-expiry cookies may be routinely saved to disk and persist across sessions, unless a user's privacy settings explicitly prevent this possibility.

Max-age This alternative, RFC-suggested expiration mechanism is not supported in Internet Explorer and therefore is not used in practice.

Domain This parameter allows the cookie to be scoped to a domain broader than the hostname that returned the *Set-Cookie* header. The exact rules and security consequences of this scoping mechanism are explored in Chapter 9.

NOTE *Contrary to what is implied in RFC 2109, it is not possible to scope cookies to a specific hostname when using this parameter. For example, domain=example.com will always match www.example.com as well. Omitting domain is the only way to create host-scoped cookies, but even this approach is not working as expected in Internet Explorer.*

Path Allows the cookie to be scoped to a particular request path prefix. This is not a viable security mechanism for the reasons explained in Chapter 9, but it may be used for convenience, to prevent identically named cookies used in various parts of the application from colliding with each other.

Secure attribute Prevents the resulting cookie from being sent over nonencrypted connections.

HttpOnly attribute Removes the ability to read the cookie through the *document.cookie* API in JavaScript. This is a Microsoft extension, although it is now supported by all mainstream browsers.

When making future requests to a domain for which valid cookies are found in the cookie jar, browsers will combine all applicable *name=value* pairs into a single, semicolon-delimited *Cookie* header, without any additional metadata, and return them to the server. If too many cookies need to be sent on a particular request, server-enforced header size limits will be exceeded, and the request may fail; there is no method for recovering from this condition, other than manually purging the cookie jar.

Curiously, there is no explicit method for HTTP servers to delete unneeded cookies. However, every cookie is uniquely identified by a name-domain-path tuple (the *secure* and *httponly* attributes are ignored), which permits an old cookie of a known scope to be simply overwritten. Furthermore, if the overwriting cookie has an *expires* date in the past, it will be immediately dropped, effectively giving a contrived way to purge the data.

Although RFC 2109 requires multiple comma-separated cookies to be accepted within a single *Set-Cookie* header, this approach is dangerous and is no longer supported by any browser. Firefox allows multiple cookies to be set in a single step via the *document.cookie* JavaScript API, but inexplicably, it requires newlines as delimiters instead. No browser uses commas as *Cookie* delimiters, and recognizing them on the server side should be considered unsafe.

Another important difference between the spec and reality is that cookie values are supposed to use the *quoted-string* format outlined in HTTP specs (see “Semicolon-Delimited Header Values” on page 48), but only Firefox and Opera recognize this syntax in practice. Reliance on *quoted-string* values is therefore unsafe, and so is allowing stray quote characters in attacker-controlled cookies.

Cookies are not guaranteed to be particularly reliable. User agents enforce modest settings on the number and size of cookies permitted per domain and, as a misguided privacy feature, may also restrict their lifetime. Because equally reliable user tracking may be achieved by other means, such as the *ETag/If-None-Match* behavior outlined in the previous section, the efforts to restrict cookie-based tracking probably do more harm than good.

HTTP Authentication

HTTP authentication, as specified in RFC 2617,¹⁹ is the original credential-handling mechanism envisioned for web applications, one that is now almost completely extinct. The reasons for this outcome might have been the inflexibility of the associated browser-level UIs, the difficulty of accommodating more sophisticated non-password-based authentication schemes, or perhaps the inability to exercise control over how long credentials are cached and what other domains they are shared with.

In any case, the basic scheme is fairly simple. It begins with the browser making an unauthenticated request, to which the server responds with a “401 Unauthorized” code.* The server must also include a *WWW-Authenticate* HTTP header, specifying the requested authentication method, the *realm* string (an arbitrary identifier to which the entered credentials should be bound), and other method-specific parameters, if applicable.

*The terms *authentication* and *authorization* appear to be used interchangeably in this RFC, but they have a distinctive meaning elsewhere in information security. *Authentication* is commonly used to refer to the process of proving your identity, whereas *authorization* is the process of determining whether your previously established credentials permit you to carry out a specific privileged action.

The client is expected to obtain the credentials in one way or the other, encode them in the *Authorization* header, and retry the original request with this header included. According to the specification, for performance reasons, the same *Authorization* header may also be included on subsequent requests to the same server path prefix without the need for a second *WWW-Authenticate* challenge. It is also permissible to reuse the same credentials in response to any *WWW-Authenticate* challenges elsewhere on the server, if the *realm* string and the authentication method match.

In practice, this advice is not followed very closely: Other than Safari and Chrome, most browsers ignore the *realm* string or take a relaxed approach to path matching. On the flip side, all browsers scope cached credentials not only to the destination server but also to a specific protocol and port, a practice that offers some security benefits.

The two credential-passing methods specified in the original RFC are known as *basic* and *digest*. The first one essentially sends the passwords in plaintext, encoded as *base64*. The other computes a one-time cryptographic hash that protects the password from being viewed in plaintext and prevents the *Authorization* header from being replayed later. Unfortunately, modern browsers support both methods and do not distinguish between them in any clear way. As a result, attackers can simply replace the word *digest* with *basic* in the initial request to obtain a clean, plaintext password as soon as the user completes the authentication dialog. Surprisingly, section 4.8 of the RFC predicted this risk and offered some helpful yet ultimately ignored advice:

User agents should consider measures such as presenting a visual indication at the time of the credentials request of what authentication scheme is to be used, or remembering the strongest authentication scheme ever requested by a server and produce a warning message before using a weaker one. It might also be a good idea for the user agent to be configured to demand Digest authentication in general, or from specific sites.

In addition to these two RFC-specified authentication schemes, some browsers also support less-common methods, such as Microsoft's *NTLM* and *Negotiate*, used for seamless authentication with Windows domain credentials.²⁰

Although HTTP authentication is seldom encountered on the Internet, it still casts a long shadow over certain types of web applications. For example, when an external, attacker-supplied image is included in a thread on a message board, and the server hosting that image suddenly decides to return "401 Unauthorized" on some requests, users viewing the thread will be presented out of the blue with a somewhat cryptic password prompt. After double-checking the address bar, many will probably confuse the prompt for a request to enter their forum credentials, and these will be immediately relayed to the attacker's image-hosting server. Oops.

Protocol-Level Encryption and Client Certificates

As should now be evident, all information in HTTP sessions is exchanged in plaintext over the network. In the 1990s, this would not have been a big deal: Sure, plaintext exposed your browsing choices to nosy ISPs, and perhaps to another naughty user on your office network or an overzealous government agency, but that seemed no worse than the behavior of SMTP, DNS, or any other commonly used application protocol. Alas, the growing popularity of the Web as a commerce platform has aggravated the risk, and substantial network security regression caused by the emergence of inherently unsafe public wireless networks put another nail in that coffin.

After several less successful hacks, a straightforward solution to this problem was proposed in RFC 2818:²¹ Why not encapsulate normal HTTP requests within an existing, multipurpose Transport Layer Security (TLS, aka SSL) mechanism developed several years earlier? This transport method leverages public key cryptography* to establish a confidential, authenticated communication channel between the two endpoints, without requiring any HTTP-level tweaks.

In order to allow web servers to prove their identity, every HTTPS-enabled web browser ships with a hefty set of public keys belonging to a variety of *certificate authorities*. Certificate authorities are organizations that are trusted by browser vendors to cryptographically attest that a particular public key belongs to a particular site, hopefully after validating the identity of the person who requests such attestation and after verifying his claim to the domain in question.

The set of trusted organizations is diverse, arbitrary, and not particularly well documented, which often prompts valid criticisms. But in the end, the system usually does the job reasonably well. Only a handful of bloopers have been documented so far (including a recent high-profile compromise of a company named Comodo²²), and no cases of widespread abuse of CA privileges are on the record.

As to the actual implementation, when establishing a new HTTPS connection, the browser receives a signed public key from the server, verifies the signature (which can't be forged without having access to the CA's private key), checks that the signed *cn* (common name) or *subjectAltName* fields in the certificate indicate that this certificate is issued for the server the browser wants to talk to, and confirms that the key is not listed on a public revocation list (for example, due to being compromised or obtained fraudulently). If everything checks out, the browser can proceed by encrypting messages to the server with that public key and be certain that only that specific party will be able to decrypt them.

Normally, the client remains anonymous: It generates a temporary encryption key, but that process does not prove the client's identity. Such a proof can be arranged, though. Client certificates are embraced internally by certain organizations and are adopted on a national level in several countries

* Public key cryptography relies on asymmetrical encryption algorithms to create a pair of keys: a private one, kept secret by the owner and required to decrypt messages, and a public one, broadcast to the world and useful only to encrypt traffic to that recipient, not to decrypt it.

around the world (e.g., for e-government services). Since the usual purpose of a client certificate is to provide some information about the real-world identity of the user, browsers usually prompt before sending them to newly encountered sites, for privacy reasons; beyond that, the certificate may act as yet another form of ambient authority.

It is worth noting that although HTTPS as such is a sound scheme that resists both passive and active attackers, it does very little to hide the evidence of access to a priori public information. It does not mask the rough HTTP request and response sizes, traffic directions, and timing patterns in a typical browsing session, thus making it possible for unsophisticated, passive attackers to figure out, for example, which embarrassing page on Wikipedia is being viewed by the victim over an encrypted channel. In fact, in one extreme case, Microsoft researchers illustrated the use of such packet profiling to reconstruct user keystrokes in an online application.²³

Extended Validation Certificates

In the early days of HTTPS, many public certificate authorities relied on fairly pedantic and cumbersome user identity and domain ownership checks before they would sign a certificate. Unfortunately, in pursuit of convenience and in the interest of lowering prices, some now require little more than a valid credit card and the ability to put a file on the destination server in order to complete the verification process. This approach renders most of the certificate fields other than *cn* and *subjectAltName* untrustworthy.

To address this problem, a new type of certificate, tagged using a special flag, is being marketed today at a significantly higher price: *Extended Validation SSL (EV SSL)*. These certificates are expected not only to prove domain ownership but also more reliably attest to the identity of the requesting party, following a manual verification process. EV SSL is recognized by all modern browsers by making portion of the address bar blue or green. Although having this tier of certificates is valuable, the idea of coupling a higher-priced certificate with an indicator that vaguely implies a “higher level of security” is often criticized as a cleverly disguised money-making scheme.

Error-Handling Rules

In an ideal world, HTTPS connections that involve a suspicious certificate error, such as a grossly mismatched hostname or an unrecognized certification authority, should simply result in a failure to establish the connection. Less-suspicious errors, such as a recently expired certificate or a hostname mismatch, perhaps could be accompanied by just a gentle warning.

Unfortunately, most browsers have indiscriminately delegated the responsibility for understanding the problem to the user, trying hard (and ultimately failing) to explain cryptography in layman’s terms and requiring the user to make a binary decision: Do you actually want to see this page or not? (Figure 3-1 shows one such prompt.)

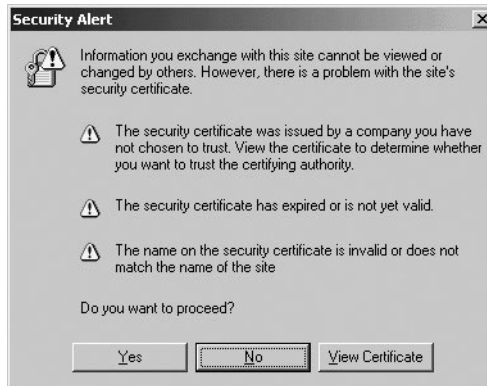


Figure 3-1: An example certificate warning dialog in the still-popular Internet Explorer 6

The language and appearance of SSL warnings has evolved through the years toward increasingly dumbed-down (but still problematic) explanations of the problem and more complicated actions required to bypass the warning. This trend may be misguided: Studies show that over 50 percent of even the most frightening and disruptive warnings are clicked through.²⁴ It is easy to blame the users, but ultimately, we may be asking them the wrong questions and offering exactly the wrong choices. Simply, if it is believed that clicking through the warning is advantageous in some cases, offering to open the page in a clearly labeled “sandbox” mode, where the harm is limited, would be a more sensible solution. And if there is no such belief, any override capabilities should be eliminated entirely (a goal sought by *Strict Transport Security*, an experimental mechanism that will be discussed in Chapter 16).

Security Engineering Cheat Sheet

When Handling User-Controlled Filenames in Content-Disposition Headers

- ✓ **If you do not need non-Latin characters:** Strip or substitute any characters except for alphanumerics, “.”, “-”, and “_”. To protect your users against potentially harmful or deceptive filenames, you may also want to confirm that at least the first character is alphanumeric and substitute all but the rightmost period with something else (e.g., an underscore).
Keep in mind that allowing quotes, semicolons, backslashes, and control characters (0x00–0x1F) will introduce vulnerabilities.
- ✓ **If you need non-Latin names:** You must use RFC 2047, RFC 2231, or URL-style percent encoding in a browser-dependent manner. Make sure to filter out control characters (0x00–0x1F) and escape any semicolons, backslashes, and quotes.

When Putting User Data in HTTP Cookies

- ✓ **Percent-encode everything except for alphanumerics.** Better yet, use base64. Stray quote characters, control characters (0x00–0x1F), high-bit characters (0x80–0xFF), commas, semicolons, and backslashes may allow new cookie values to be injected or the meaning and scope of existing cookies to be altered.

When Sending User-Controlled Location Headers

- ✓ **Consult the cheat sheet in Chapter 2.** Parse and normalize the URL, and confirm that the scheme is on a whitelist of permissible values and that you are comfortable redirecting to the specified host.
Make sure that any control and high-bit characters are escaped properly. Use Punycode for hostnames and percent-encoding for the remainder of the URL.

When Sending User-Controlled Redirect Headers

- ✓ **Follow the advice provided for Location.** Note that semicolons are unsafe in this header and cannot be escaped reliably, but they also happen to have a special meaning in some URLs. Your choice is to reject such URLs altogether or to percent-encode the “;” character, thereby violating the RFC-mandated syntax rules.

When Constructing Other Types of User-Controlled Requests or Responses

- ✓ **Examine the syntax and potential side effects of the header in question.** In general, be mindful of control and high-bit characters, commas, quotes, backslashes, and semicolons; other characters or strings may be of concern on a case-by-case basis. Escape or substitute these values as appropriate.
- ✓ **When building a new HTTP client, server, or proxy:** Do not create a new implementation unless you absolutely have to. If you can’t help it, read this chapter thoroughly and aim to mimic an existing mainstream implementation closely. If possible, ignore the RFC-provided advice about fault tolerance and bail out if you encounter any syntax ambiguities.

4

HYPERTEXT MARKUP LANGUAGE

The Hypertext Markup Language (HTML) is the primary method of authoring online documents. One of the earliest written accounts of this language is a brief summary posted on the Internet by Tim Berners-Lee in 1991.¹ His proposal outlines an SGML-derived syntax that allows text documents to be annotated with inline hyperlinks and several types of layout aids. In the following years, this specification evolved gradually under the direction of Sir Berners-Lee and Dan Connolly, but it wasn't until 1995, at the onset of the First Browser Wars, that a reasonably serious and exhaustive specification of the language (HTML 2.0) made it to RFC 1866.²

From that point on, all hell broke loose: For the next few years, competing browser vendors kept introducing all sorts of flashy, presentation-oriented features and tweaked the language to their liking. Several attempts to amend the original RFC have been undertaken, but ultimately the IETF-managed

standardization approach proved to be too inflexible. The newly formed World Wide Web Consortium took over the maintenance of the language and eventually published the HTML 3.2 specification in 1997.³

The new specification tried to reconcile the differences in browser implementations while embracing many of the bells and whistles that appealed to the public, such as customizable text colors and variable typefaces. Ultimately, though, HTML 3.2 proved to be a step back for the clarity of the language and had only limited success in catching up with the facts.

In the following years, the work on HTML 4 and 4.01⁴ focused on pruning HTML of all accumulated excess and on better explaining how document elements should be interpreted and rendered. It also defined an alternative, strict XHTML syntax derived from XML, which was much easier to consistently parse but more punishing to write. Despite all this work, however, only a small fraction of all websites on the Internet could genuinely claim compliance with any of these standards, and little or no consistency in parsing modes and error recovery could be seen on the client end. Consequently, some of the work on improving the core language fizzled out, and the W3C turned its attention to stylesheets, the Document Object Model, and other more abstract or forward-looking challenges.

In the late 2000s, some of the low-level work has been revived under the banner of HTML5,⁵ an ambitious project to normalize almost every aspect of the language syntax and parsing, define all the related APIs, and more closely police browser behavior in general. Time will tell if it will be successful; until then, the language itself, and each of the four leading parsing engines,* come with their own set of frustrating quirks.

Basic Concepts Behind HTML Documents

From a purely theoretical standpoint, HTML relies on a fairly simple syntax: a hierarchical structure of tags, *name=value* tag parameters, and text nodes (forming the actual document body) in between. For example, a simple document with a title, a heading, and a hyperlink may look like this:

```
<html>
  <head>
    <title>Hello world</title>
  </head>
  <body>
    <h1>Welcome to our example page</h1>
    <a href="http://www.example.com/">Click me!</a>
  </body>
</html>
```

*To process HTML documents, Internet Explorer uses the Trident engine (aka MSHTML); Firefox and some derived products use Gecko; Safari, Chrome, and several other browsers use WebKit; and Opera relies on Presto. With the exception of WebKit, a collaborative open source effort maintained by several vendors, these engines are developed largely in-house by their respective browser teams.

This syntax puts some constraints on what may appear inside a parameter value or inside the document body. Five characters—angle brackets, single and double quotes, and an ampersand—are reserved as the building blocks of the HTML markup, and these need to be avoided or escaped in some way when used outside of their intended function. The most important rules are:

- Stray ampersands (&) should never appear in most sections of an HTML document.
- Both types of angle brackets are obviously problematic inside a tag, unless properly quoted.
- The left angle bracket (<) is a hazard inside a text node.
- Quote characters appearing inside a tag can have undesirable effects, depending on their exact location, but are harmless in text nodes.

To allow these characters to appear in problematic locations without causing side effects, an ampersand-based encoding scheme, discussed in “Entity Encoding” on page 76, is provided.

NOTE *Of course, the availability of such an encoding scheme is not a guarantee of its use. The failure to properly filter out or escape reserved characters when displaying user-controlled data is the cause of a range of extremely common and deadly web application security flaws. A particularly well-known example of this is cross-site scripting (XSS), an attack in which malicious, attacker-provided JavaScript code is unintentionally echoed back somewhere in the HTML markup, effectively giving the attacker full control over the appearance and operation of the targeted site.*

Document Parsing Modes

For any HTML document, a top-level `<!DOCTYPE>` directive may be used to instruct the browser to parse the file in a manner that at least superficially conforms to one of the officially defined standards; to a more limited extent, the same signal can be conveyed by the *Content-Type* header, too. Of all the available parsing modes, the most striking difference exists between XHTML and traditional HTML. In the traditional mode, parsers will attempt to recover from most types of syntax errors, including unmatched opening and closing tags. In addition, tag and parameter names will be considered case insensitive, parameter values will not always need to be quoted, and certain types of tags, such as `<img>`, will be closed implicitly. In other words, the following input will be grudgingly tolerated:

```
<html>
  <BODY>
    <IMG src="/hello_world.jpg">
    <a HREF=http://www.example.com/>
      Click me!
    </oops>
  </html>
```

The XML mode, on the other hand, is strict: All tags need to be balanced carefully, named using the proper case, and closed explicitly. (The XML-specific self-closing tag syntax, such as `<img />`, is permitted.) In addition, most syntax mistakes, even trivial ones, will result in an error and prevent the document from being displayed at all.

Unlike the regular flavor of HTML, XML-based documents may also elegantly incorporate sections using other XML-compliant markup formats, such as MathML, a mathematical formula markup language. This is done by specifying a different *xmlns* namespace setting for a particular tag, with no need for one-off, language-level hacks.

The last important difference worth mentioning here is that traditional HTML parsing strategies feature a selection of special modes, entered into after certain tags are encountered and exited only when a specific terminator string is seen; everything in between is interpreted as non-HTML text. Some examples of such special tags include `<style>`, `<script>`, `<textarea>`, or `<xmp>`. In practical implementations, these modes are exited only when a literal, case-insensitive match on `</style>`, `</script>`, or a similar matching value, is made; any other markup inside such a block will not be interpreted as HTML. (Interestingly, there is one officially obsolete tag, `<plaintext>`, that cannot be exited at all; it stays in effect for the remainder of the document.)

In comparison, the XML mode is more predictable. It generally forbids stray “<” and “&” characters inside the document, but it provides a special syntax, starting with “`<![CDATA[`” and ending with “`]]>`”, as a way to encapsulate any raw text inside an arbitrary tag. For example:

```
<script>
<![CDATA[
  alert('>>> Hello world! <<<');
]]>
</script>
```

The other notable special parsing mode available in both XHTML and normal HTML is a comment block. In XML, it quite simply begins with “`<!--`” and ends with “`-->`”. In the traditional HTML parser in Firefox versions prior to 4, any occurrence of “`--`”, later followed by “`>`”, is also considered good enough.

The Battle over Semantics

The low-level syntax of the language aside, HTML is also the subject of a fascinating conceptual struggle: a clash between the ideology and the reality of the online world. Tim Berners-Lee always championed the vision of a *semantic web*, an interconnected system of documents in which every functional block, such as a citation, a snippet of code, a mailing address, or a heading, has its meaning explained by an appropriate machine-readable tag (say, `<cite>`, `<code>`, `<address>`, or `<h1>` to `<h6>`).

This approach, he and other proponents argued, would make it easier for machines to crawl, analyze, and index the content in a meaningful way, and in the near future, it would enable computers to reason using the sum of human knowledge. According to this philosophy, the markup language should provide a way to stylize the appearance of a document, but only as an afterthought.

Sir Berners-Lee has never given up on this dream, but in this one regard, the actual usage of HTML proved to be very different from what he wished for. Web developers were quick to pragmatically distill the essence of HTML 3.2 into a handful of presentation-altering but semantically neutral tags, such as `<font>`, `<b>`, and `<pre>`, and saw no reason to explain further the structure of their documents to the browser. W3C attempted to combat this trend but with limited success. Although tags such as `<font>` have been successfully obsoleted and largely abandoned in favor of CSS, this is only because stylesheets offered more powerful and consistent visual controls. With the help of CSS, the developers simply started relying on a soup of semantically agnostic `<span>` and `<div>` tags to build everything from headings to user-clickable buttons, all in a manner completely opaque to any automated content extraction tools.

Despite having had a lasting impact on the design of the language, in some ways, the idea of a semantic web may be becoming obsolete: Online content less frequently maps to the concept of a single, viewable document, and HTML is often reduced to providing a convenient drawing surface and graphic primitives for JavaScript applications to build their interfaces with.

Understanding HTML Parser Behavior

The fundamentals of HTML syntax outlined in the previous sections are usually enough to understand the meaning of well-formed HTML and XHTML documents. When the XHTML dialect is used, there is little more to the story: The minimal fault-tolerance of the parser means that anomalous syntax almost always leads simply to a parsing error. Alas, the picture is very different with traditional, laid-back HTML parsers, which aggressively second-guess the intent of the page developer even in very ambiguous or potentially harmful situations.

Since an accurate understanding of user-supplied markup is essential to designing many types of security filters, let's have a quick look at some of these behaviors and quirks. To begin, consider the following reference snippet:

```
<img src=image.jpg title="Hello world" class=examples>
```

1 2 3 4 5 6

The diagram shows the HTML tag `<img src=image.jpg title="Hello world" class=examples>` with a horizontal line above it. Below the line, six numbered arrows point to specific characters in the tag: 1 points to the opening angle bracket, 2 points to the first space, 3 points to the first equals sign, 4 points to the space before the quote, 5 points to the closing quote, and 6 points to the closing angle bracket.

Web developers are usually surprised to learn that this syntax can be drastically altered without changing its significance to the browser. For example, Internet Explorer will allow an NUL character (0x00) to be inserted in the location marked at ❶, a change that is likely to throw all naïve HTML filters off the trail. It is also not widely known that the whitespaces at ❷ and ❹ can

be substituted with uncommon vertical tab (0x0B) or form feed (0x0C) characters in all browsers and with a nonbreaking UTF-8 space (0xA0) in Opera.* Oh, and here's a really surprising bit: In Firefox, the whitespace at ❷ can also be replaced with a single, regular slash—yet the one at ❹ can't.

Moving on, the location marked ❸ is also of note. In this spot, NUL characters are ignored by most parsers, as are many types of whitespaces. Not long ago, WebKit browsers accepted a slash in this location, but recent parser improvements have eliminated this quirk.

Quote characters are a yet another topic of interest. Website developers know that single and double quotes can be used to put a string containing whitespaces or angle brackets in an HTML parameter, but it usually comes as a surprise that Internet Explorer also honors backticks (`) instead of real quotes in the location marked ❺. Similarly, few people realize that in any browser, an implicit whitespace is inserted after a quoted parameter, and that the explicit whitespace at ❻ can therefore be skipped without changing the meaning of the tag.

The security impact of these patterns is not always easy to appreciate, but consider an HTML filter tasked with scrubbing an ** tag with an attacker-controlled *title* parameter. Let's say that in the input markup, this parameter is not quoted if it contains no whitespaces and angle brackets—a design that can be seen on a popular blogging site. This practice may appear safe at first, but in the following two cases, a malicious, injected *onerror* parameter will materialize inside a tag:

```
<img ... title=""onerror="alert(1)">
```

and

```
<img ... title=`onerror=`alert(1)`>
```

Yet another wonderful quote-related quirk in Internet Explorer makes this job even more complicated. While most browsers recognize quoting only when it is used at the beginning of a parameter value, Internet Explorer simply checks for any occurrence of an equal sign (=) followed by a quote and will parse this syntax in a rather unexpected way:

```
<img src=test.jpg?value=">Yes, we are still inside a tag!">
```

Interactions Between Multiple Tags

Parsing a single tag can be a daunting task, but as you might imagine, anomalous arrangements of multiple HTML tags will be even less predictable. Consider the following trivial example:

```
<i <b>
```

*The behavior exhibited by Opera is particularly sneaky: The Unicode whitespace is not recognized by many standard library functions used in server-side HTML sanitizers, such as *isspace(...)* in libc. This increases the risk of implementation glitches.

When presented with such syntax, most browsers only interpret `<i>` and treat the “`<b>`” string as an invalid tag parameter. Firefox versions before 4, however, would automatically close the `<i>` tag first when encountering an angle bracket and, in the end, will interpret both `<i>` and `<b>`. In the spirit of fault tolerance, until recently WebKit followed that model, too.

A similar behavior can be observed in previous versions of Firefox when dealing with tag names that contain invalid characters (in this case, the equal sign). Instead of doing its best to ignore the entire block, the parser would simply reset and interpret the quoted tag:

```
<i="<b>">
```

The handling of tags that are not closed before the end of the file is equally fascinating. For example, the following snippet will prompt most browsers to interpret the `<i>` tag or ignore the entire string, but Internet Explorer and Opera use a different backtracking approach and will see `<b>` instead:

```
<i foo="<b>" [EOF]
```

In fact, Firefox versions prior to version 4 engaged in far-fetched reparsing whenever particular special tags, such as `<title>`, were not closed before the end of the document:

```
<title>This text will be interpreted as a title
<i>This text will be shown as document body!
[EOF]
```

The last two parsing quirks have interesting security consequences in any scenario where the attacker may be able to interrupt page load prematurely. Even if the markup is otherwise fairly well sanitized, the meaning of the document may change in a very unexpected way.

Explicit and Implicit Conditionals

To further complicate the job of HTML parsing, some browsers exhibit behaviors that can be used to conditionally skip some of the markup in a document. For example, in an attempt to help novice users of Microsoft’s Active Server Pages development platform, Internet Explorer treats `<% ... %>` blocks as a completely nonstandard comment, hiding any markup between these two character sequences. Another Internet Explorer-specific feature is explicit conditional expressions interpreted by the parser and smuggled inside standard HTML comment blocks:

```
<!--[if IE 6]>
  Markup that will be parsed only for Internet Explorer 6
<![endif]-->
```

Many other quirks of this type are related to the idiosyncrasies of SGML and XML. For example, due to the comment-handling behavior mentioned earlier in an aside, browsers disagree on how to parse `!-` and `?-` directives (such as `<!DOCTYPE>` or `<?xml>`), whether to allow XML-style CDATA blocks in non-XHTML modes, and on what precedence to give to overlapping special parsing mode tags (such as `<style><!-- </style> -->`).

HTML Parsing Survival Tips

The set of parsing behaviors discussed in the previous sections is by no means exhaustive. In fact, an entire book has been written on this topic: Inquisitive readers are advised to grab *Web Application Obfuscation* (Syngress, 2011) by Mario Heiderich, Eduardo Alberto Vela Nava, Gareth Heyes, and David Lindsay—and then weep about the fate of humanity. The bottom line is that building HTML filters that try to block known dangerous patterns, and allow the remaining markup as is, is simply not feasible.

The only reasonable approach to tag sanitization is to employ a realistic parser to translate the input document into a hierarchical in-memory document tree, and then scrub this representation for all unrecognized tags and parameters, as well as any undesirable tag/parameter/value configurations. At that point, the tree can be carefully reserialized into a well-formed, well-escaped HTML that will not flex any of the error correction muscles in the browser itself. Many developers think that a simpler design should be possible, but eventually they discover the reality the hard way.

Entity Encoding

Let's talk about character encoding again. As noted on the first pages of this chapter, certain reserved characters are generally unsafe inside text nodes and tag parameter values, and they will often lead to outright syntax errors in XHTML. In order to allow such characters to be used safely (and to allow a convenient way to embed high-bit text), a simple ampersand-prefixed, semicolon-terminated encoding scheme, known as entity encoding, is available to developers.

The most familiar use of this encoding method is the inclusion of certain predefined, named entities. Only a handful of these are specified for XML, but several hundred more are scattered in HTML specifications and supported by all modern browsers. In this approach, `<` is used to insert a left angle bracket; `>` substitutes a right angle bracket; `&` replaces the ampersand itself; while, say, `→` is a nice Unicode arrow.

NOTE *In XHTML documents, additional named entities can be defined using the `<!ENTITY>` directive and made to resolve to internally defined strings or to the contents of an external file URL. (This last option is obviously unsafe if allowed when processing untrusted content; the resulting attack is sometimes called External XML Entity, or XXE for short.)*

In addition to the named entities, it is also possible to insert an arbitrary ASCII or Unicode character using a decimal `&#number;` notation. In this case, `<` maps to a left angle bracket; `>` substitutes a right one; and `😹` is, I kid you not, a Unicode 6.0 character named “smiling cat face with tears of joy.” Hexadecimal notation can also be used if the number is prefixed with “x”. In this variant, the left angle bracket becomes `<`, etc.

The HTML parser recognizes entity encoding inside text nodes and parameter values and decodes it transparently when building an in-memory representation of the document tree. Therefore, the following two cases are functionally identical:

```

```

and

```

```

The following two examples, on the other hand, will not work as expected, as the encoding interferes with the structure of the tag itself:

```
<img src&#x3d;"http://www.example.com">
```

and

```
<img s&#x72;c="http://www.example.com">
```

The largely transparent behavior of entity encoding makes it important to correctly resolve it prior to making any security decisions about the contents of a document and, if applicable, to properly restore it in the sanitized output later on. To illustrate, the following syntax must be recognized as an absolute reference to a *javascript:* pseudo-URL and not to a cryptic fragment ID inside a relative resource named “./javascript&”:

```
<a href="javascript&#x3a;alert(1)">
```

Unfortunately, even the simple task of recognizing and parsing HTML entities can be tricky. In traditional parsing, for example, entities may often be accepted even if the trailing semicolon is omitted, as long as the next character is not an alphanumeric. (In Firefox, dashes and periods are also accepted in entity names.) Numeric entities are even more problematic, as they may have an overlong notation with an arbitrary number of trailing zeros. Moreover, if the numerical value is higher than 2^{32} , the standard size of an integer on many computer architectures, the corresponding character may be computed incorrectly.

Developers working with XHTML should be aware of a potential pitfall in that dialect, too. Although HTML entities are not recognized in most of the special parsing modes, XHTML differs from traditional HTML in that tags such as `<script>` and `<style>` do not automatically toggle a special parsing mode on their own. Instead, an explicit `<![CDATA[...]]>` block around any scripts or stylesheets is required to achieve a comparable effect. Therefore, the following snippet with an attacker-controlled string (otherwise scrubbed for angle brackets, quotes, backslashes, and newlines) is perfectly safe in HTML, but not in XHTML:

```
<script>
  var tmp = 'I am harmless! &#x27;+alert(1);// Or am I?';
  ...
</script>
```

HTTP/HTML Integration Semantics

From Chapter 3, we recall that HTTP headers may give new meaning to the entire response (*Location*, *Transfer-Encoding*, and so on), change the way the payload is presented (*Content-Type*, *Content-Disposition*), or affect the client-side environment in other, auxiliary ways (*Refresh*, *Set-Cookie*, *Cache-Control*, *Expires*, etc.).

But what if an HTML document is delivered through a non-HTTP protocol or loaded from a local file? Clearly, in this case, there is no simple way to express or preserve this information. We can part with some of it easily, but parameters such as the MIME type or the character set are essential, and losing them forces browsers to improvise later on. (Consider, for example, that charsets such as UTF-7, UTF-16, and UTF-32 are not ASCII-compatible and, therefore, HTML documents can't even be parsed without determining which of these transformations needs to be used.)

The security consequences of the browser-level heuristics used to detect character sets and document types will be explored in detail in Chapter 13. Meanwhile, the problem of preserving protocol-level information within a document is somewhat awkwardly addressed by a special HTML directive, `<meta http-equiv=...>`. By the time the browser examines the markup, many content-handling decisions must have already been made, but some tweaks are still on the table; for example, it may be possible to adjust the charset to a generally compatible value or to specify *Refresh*, *Set-Cookie*, and caching directives.

As an illustration of permissible syntax, consider the following directive that, when appearing in an 8-bit ASCII document, will clarify for the browser that the charset of the document is UTF-8 and not, say, ISO-8859-1:

```
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
```

On the flip side, all of the following directives will fail, because at this point it is too late to switch to an incompatible UTF-32 encoding, change the document type to a video format, or execute a redirect instead of parsing the file:

```
<meta http-equiv="Content-Type" content="text/html; charset=utf-32">
<meta http-equiv="Content-Type" content="video/mpeg">
<meta http-equiv="Location" content="http://www.example.com">
```

Be mindful that when *http-equiv* values conflict with each other, or contradict the HTTP headers received from the server earlier on, their behavior is not consistent and should not be relied upon. For example, the first supported *charset=* value usually prevails (and HTTP headers have precedence over *<meta>* in this case), but with several conflicting *Refresh* values, the behavior is highly browser-specific.

NOTE *Some browsers will attempt to speculatively extract <meta http-equiv> information before actually parsing the document, which may lead to embarrassing mistakes. For example, a security bug recently fixed in Firefox 4 caused the browser to interpret the following statement as a character set declaration: <meta http-equiv="Refresh" content="10;http://www.example.com/charset=utf-7">. ⁶*

Hyperlinking and Content Inclusion

One of the most important and security-relevant features of HTML is, predictably, the ability to link to and embed external content. HTTP-level features such as *Location* and *Refresh* aside, this can be accomplished in a couple of straightforward ways.

Plain Links

The following markup demonstrates the most familiar and most basic method for referencing external content from within a document:

```
<a href="http://www.example.com/">Click me!</a>
```

This hyperlink may point to any of the browser-recognized schemes, including pseudo-URLs (*data:*, *javascript:*, and so on) and protocols handled by external applications (such as *mailto:*). Clicking on the text (or any HTML elements) nested inside such a ** block will typically prompt the browser to navigate away from the linking document and go to the specified location, if meaningfully possible for the protocol used.

An optional *target* parameter may be used to target other windows or document views for navigation. The parameter must specify the name of the target view. If the name cannot be found, or if access is denied, the default behavior is typically to open a new window instead. The conditions in which access may be denied are the topic of Chapter 11.

Four special target names can be used, too (as shown on the left of Figure 4-1): *_blank* always opens a brand-new window, *_parent* navigates a higher-level view that embeds the link-bearing document (if any), and *_top* always navigates the top-level browser window, no matter how many document embedding levels are in between. Oh, right, the fourth special target, *_self*, is identical to not specifying a value at all and exists for no reason whatsoever.

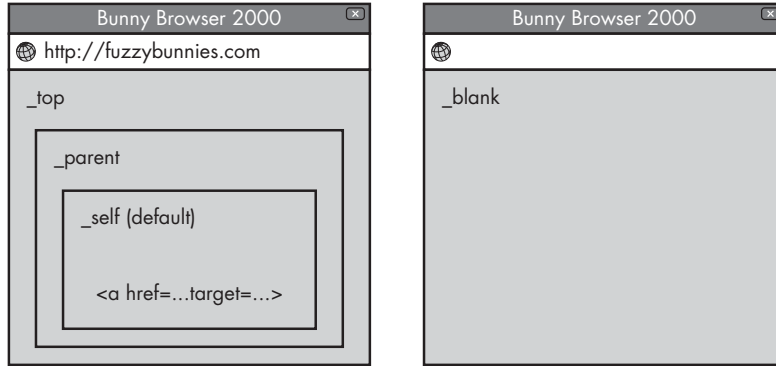


Figure 4-1: Predefined targets for hyperlinks

Forms and Form-Triggered Requests

An HTML form can be thought of as an information-gathering hyperlink: When the “submit” button is clicked, a dynamic request is constructed on the fly from the data collected via any number of input fields. Forms allow user input and files to be uploaded to the server, but in almost every other way, the result of submitting a form is similar to following a normal link.

A simple form markup may look like this:

```
<form method=GET action="/process_form.cgi">
  Given name: <input type=text name=given>
  Family name: <input type=text name=family>
  ...
  <input type=submit value="Click here when done!">
</form>
```

The *action* parameter works like the *href* value used for normal links, with one minor difference: If the value is absent, the form will be submitted to the location of the current document, whereas any destination-free *<a>* links will simply not work at all. An optional *target* parameter may also be specified and will behave as outlined in the previous section.

NOTE *Unusually, unlike <a> tags, forms cannot be nested inside each other, and only the top-level <form> tag will remain operational in such a case.*

When the *method* value is set to GET or is simply not present at all, all the nested field names and their current values will be escaped using the familiar percent-encoding scheme outlined in Chapter 2, but with two rather arbitrary differences. First, the space character (0x20) will be substituted with the plus

sign, rather than encoded as “%20”. Second, following from this, any existing plus signs need to be encoded as “%2B”, or else they will be misinterpreted as spaces.

Encoded *name=value* pairs are then delimited with ampersands and combined into a single string, such as this:

```
given=Erwin+Rudolf+Josef+Alexander&family=Schr%C3%B6dinger
```

The resulting value is inserted into the query part of the destination URL (replacing any existing contents of that section) and submitted to the server. The received response is then shown to the user in the targeted viewport.

The situation is a bit more complicated if the *method* parameter is set to POST. For that type of HTTP request, three data submission formats are available. In the default mode (referred to as *application/x-www-form-urlencoded*), the message is constructed the same way as for GET but is transmitted in the request payload instead, leaving the query string and all other parts of the destination URL intact.*

The existence of the second POST submission mode, triggered by specifying *enctype="text/plain"* on the *<form>* tag, is difficult to justify. In this mode, field names and values will not be percent encoded at all (but, depending on the browser, plus signs may be used to substitute for spaces), and a newline delimiter will be used in place of an ampersand. The resulting format is essentially useless, as it can't be parsed unambiguously: Form-originating newlines and equal signs are indistinguishable from browser inserted ones.

The last mode is triggered with *enctype="multipart/form-data"* and must be used whenever submitting user-selected files through a form (which is possible with a special *<input type="file">* tag). The resulting request body consists of a series of short MIME messages corresponding to every submitted field.† These messages are delimited with a client-selected random, unique boundary token that should otherwise not appear in the encapsulated data:

```
POST /process_form.cgi HTTP/1.1
...
Content-Type: multipart/form-data; boundary=random1234

--random1234
Content-Disposition: form-data; name="given"

Erwin Rudolf Josef Alexander
--random1234
Content-Disposition: form-data; name="family"
```

* This has the potential for confusion, as the same parameter may appear both in the query string and in the POST payload. There is no consistency in how various server-side web applications frameworks resolve this conflict.

† MIME (Multipurpose Internet Mail Extensions) is a data format intended for encapsulating and safely transmitting various types of documents in email messages. The format makes several unexpected appearances in the browser world. For example, *Content-Type* file format identifiers also have unambiguous MIME roots.

```
Schrödinger
--random1234
Content-Disposition: form-data; name="file"; filename="cat_names.txt"
Content-Type: text/plain

(File contents follow)
--random1234--
```

Despite the seemingly open-ended syntax of the tag, other request methods and submission formats are not supported by any browser, and this is unlikely to change. For a short while, the HTML5 standard tried to introduce PUT and DELETE methods in forms, but this proposal was quickly shot down.

Frames

Frames are a form of markup that allows the contents of one HTML document to be displayed in a rectangular region of another, embedding page. Several framing tags are supported by modern browsers, but the most common way of achieving this goal is with a hassle-free and flexible inline frame:

```
<iframe src="http://www.example.com/"></iframe>
```

In traditional HTML documents, this tag puts the parser in one of the special parsing modes, and all text between the opening and the closing tag will simply be ignored in frame-aware browsers. In legacy browsers that do not understand *<iframe>*, the markup between the opening and closing tags is processed normally, however, offering a decidedly low-budget, conditional rendering directive. This conditional behavior is commonly used to provide insightful advice such as “This page must be viewed in a browser that supports frames.”

The frame is a completely separate document view that in many aspects is identical to a new browser window. (It even enjoys its own JavaScript execution context.) Like browser windows, frames can be equipped with a *name* parameter and then targeted from *<a>* and *<form>* tags.

The constraints on the *src* URL for framed content are roughly similar to the rules enforced on regular links. This includes the ability to point frames to javascript: or to load externally handled protocols that leave the frame empty and open the target application in a new process.

Frames are of special interest to web security, as they allow almost unconstrained types of content originating from unrelated websites to be combined onto a single page. We will have a second look at the problems associated with this behavior in Chapter 11.

Type-Specific Content Inclusion

In addition to content-agnostic link navigation and document framing, HTML also provides multiple ways for a more lightweight inclusion of several pre-defined types of external content.

Images

Image files can be retrieved and displayed on a page using `<img>` tags, via stylesheets, and through a legacy `background=` parameter on markup such as `<body>` or `<table>`.

The most popular image type on the Internet is a lossy but very efficient JPEG file, followed by lossless and more featured (but slower) PNG. An increasingly obsolete lossless GIF format is also supported by every browser, and so is the rarely encountered and usually uncompressed Windows bitmap file (BMP). An increasing number of rendering engines support SVG, an XML-based vector graphics and animation format, too, but the inclusion of such images through the `<img>` tag is subject to additional restrictions.

The list of recognized image types can be wrapped up with odds and ends such as Windows metafiles (WMF and EMF), Windows Media Photo (WDP and HDP), Windows icons (ICO), animated PNG (APNG), TIFF images, and—more recently—WebP. Browser support for these is far from universal, however.

Cascading stylesheets

These text-based files can be loaded with a `<link rel=stylesheet href=...>` tag—even though `<style src=...>` would be a more intuitive choice—and may redefine the visual aspects of almost any other HTML tag within their parent document (and in some cases, even include embedded JavaScript). The syntax and function of CSS are the subject of Chapter 5.

In the absence of the appropriate *charset* value in the *Content-Type* header for the downloaded stylesheet, the encoding according to which this subresource will be interpreted can be specified by the including party through the *charset* parameter of the `<link>` tag.

Scripts

Scripts are text-based programs included with `<script>` tags and are executed in a manner that gives them full control over the host document. The primary scripting language for the Web is JavaScript, although an embedded version of Visual Basic is also supported in Internet Explorer and can be used at will. Chapter 6 takes an in-depth look at client-side scripts and their capabilities.

As with CSS, in the absence of valid *Content-Type* data, the charset according to which the script is interpreted may be controlled by the including party.

Plug-in content

This category spans miscellaneous binary files included with `<embed>` or `<object>` tags or via an obsolete, Java-specific `<applet>` tag. Browser plug-in content follows its own security rules, which are explored to some extent in Chapters 8 and 9. In many cases, it is safe to consider plug-in-supported content as equivalent to or more powerful than JavaScript.

NOTE *The standard permits certain types of browser-supported documents, such as text/html or text/plain, to be loaded through <object> tags, in which case they form a close equivalent of <iframe>. This functionality is not used in practice, and the rationale behind it is difficult to grasp.*

Other supplementary content

This category includes various rendering cues that may or may not be honored by the browser; they are most commonly provided through `<link>` directives. Examples include website icons (known as “favicons”), alternative versions of a page, and chapter navigation links.

Several other once-supported content inclusion methods, such as the `<bgsound>` tag for background music, were commonplace in the past but have fallen out of grace. On the other hand, as a part of HTML5, new tags such as `<video>` and `<audio>` are expected to gain popularity soon.

There is relatively little consistency in what URL schemes are accepted for type-specific content retrieval. It should be expected that protocols routed to external applications will be rejected, as they do not have a sensible meaning in this context, but beyond this, not many assumptions should be made. As a security precaution, most browsers will also reject scripting-related schemes when loading images and stylesheets, although Internet Explorer 6 and Opera do not follow this practice. As of this writing, *javascript*: URLs are also permitted on `<embed>` and `<applet>` tags in Firefox but not, for example, on `<img>`.

For almost all of the type-specific content inclusion methods, *Content-Type* and *Content-Disposition* headers provided by the server will typically be ignored (perhaps except for the *charset=* value), as may be the HTTP response code itself. It is best to assume that whenever the body of any server-provided resource is even vaguely recognizable as one of the data formats enumerated in this section, it may be interpreted as such.

A Note on Cross-Site Request Forgery

On all types of cross-domain navigation, the browser will transparently include any ambient credentials; consequently, to the server, a request legitimately originating from its own client-side code will appear roughly the same as a request originating from a rogue third-party site, and it may be granted the same privileges.

Applications that fail to account for this possibility when processing any sensitive, state-changing requests are said to be vulnerable to *cross-site request forgery* (*XSRF* or *CSRF*). This vulnerability can be mitigated in a number of ways, the most common of which is to include a secret user- and session-specific value on such requests (as an additional query parameter or a hidden form field). The attacker will not be able to obtain this value, as read access to cross-domain documents is restricted by the same-origin policy (see Chapter 9).

Security Engineering Cheat Sheet

Good Engineering Hygiene for All HTML Documents

- ✓ Always output consistent, valid, and browser-supported *Content-Type* and *charset* information to prevent the document from being interpreted contrary to your original intent.

When Generating HTML Documents with Attacker-Controlled Bits

This task is difficult to perform consistently across the entire web application, and it is one of the most significant sources of web application security flaws. Consider using context-sensitive auto-escaping frameworks, such as *JSilver* or *CTemplate*, to automate it. If that is not possible, read on.

- ✓ **User-supplied content in text body:** Always entity-encode “<”, “>”, and “&”. Note that certain other patterns may be dangerous in certain non-ASCII-compatible output encodings. If applicable, consult Chapter 13.

Keep in mind that some Unicode metacharacters (e.g., U+202E) alter the direction or flow of the subsequent text. It may be desirable to remove them in particularly sensitive uses.

- ✓ **Tag-specific style and on* parameters:** Multiple levels of escaping are required. This practice is extremely error prone, meaning not really something to attempt. If it is absolutely unavoidable, review the cheat sheets in Chapters 5 and 6.
- ✓ **All other HTML parameter values:** Always use quotes around attacker-controlled input. Entity-encode “<”, “>”, “&”, and any stray quotes. Remember that some parameters require additional validation. For URLs, see the cheat sheet in Chapter 2.

Never attempt to blacklist known bad values in URLs or any other parameters; doing so will backfire and may lead to script execution flaws.

- ✓ **Special parsing modes (e.g., <script> and <style> blocks):** For values appearing inside quoted strings, replace quote characters, backslash, “<”, “>”, and all nonprintable characters with language-appropriate escape codes. For values appearing outside strings, exercise extreme caution and allow only carefully validated, known, alphanumeric values.

In XHTML mode, remember to wrap the entire script section in a CDATA block. Avoid cases that require multiple levels of encoding, such as building parameters to the JavaScript *eval(...)* function using attacker-supplied strings. Never place user-controlled data inside HTML comments, !-type or ?-type tags, and other nonessential or unusually parsed blocks.

When Converting HTML to Plaintext

- ✓ A common mistake is to strip only well-formed tags. Remember that all left-angle brackets must be removed, even if no matching right-angle bracket is found. To minimize the risk of errors, always entity-escape angle brackets and ampersands in the generated output, too.

When Writing a Markup Filter for User Content

- ☑ Read this chapter carefully. Use a reasonably robust HTML parser to build an in-memory document tree. Walk the tree, removing any unrecognized or unnecessary tags and parameters and scrubbing any undesirable tags/parameters/value combinations.
When done, reserialize the document, making sure to apply proper escaping rules to parameter values and text content. (See the first tip on this cheat sheet.) Be aware of the impact of special parsing modes.
- ☑ Because of the somewhat counterintuitive namespace interactions with JavaScript, do not allow *name* and *id* parameters on user-supplied markup—at least not without reading Chapter 6 first.
- ☑ Do not attempt to sanitize an existing, serialized document in place. Doing so inevitably leads to security problems.

5

CASCADING STYLE SHEETS

As the Web matured through the 1990s, website developers increasingly needed a consistent and flexible way to control the appearance of HTML documents; the collection of random, vendor-specific tag parameters available at the time simply would not do. After reviewing several competing proposals, W3C eventually settled on *Cascading Style Sheets (CSS)*, a fairly simple text-based page appearance description language proposed by Håkon Wium Lie.

The initial CSS level 1 specification saw the light of day by the end of 1996,¹ but further revisions of this document continued until 2008. The initial draft of CSS level 2 followed in December 1998 and has yet to be finalized as of 2011. The work on the most recent iteration, level 3, started in 2005 and also continues to this day. Although most of the individual features envisioned for CSS2 and CSS3 have been adopted by all modern browsers after years of trial and error, many subtle details vary significantly from one implementation to another, and the absence of a finalized standard likely contributes to this.

Despite the differences from one browser to another, CSS is a very powerful tool. With only a couple of constraints, stylesheets permit almost every HTML tag to be scaled, positioned, and decorated nearly arbitrarily, thereby overcoming the constraints originally placed on it by the underlying markup language; in some implementations, JavaScript programs can be embedded in the CSS presentation directives as well. The job of placing user-controlled values inside stylesheets, or recoding any externally provided CSS, is therefore of great interest to web application security.

Basic CSS Syntax

Stylesheets can be placed in an HTML document in three ways: inlined globally for the entire document with a `<style>` block, retrieved from an external URL via the `<link rel=stylesheet>` directive, or attached to a specific tag using the `style` parameter. In addition, XML-based documents (including XHTML) may also leverage a little-known `<?xml-stylesheet href=... ?>` directive to achieve the same goal.

The first two methods of inclusion require a fully qualified stylesheet consisting of any number of selectors (directives describing which HTML tags the following ruleset will apply to) followed by semicolon-delimited *name: value* rules between curly brackets. Here is a simple example of such syntax, defining the appearance of `<img>`, `<span>`, and `<div>` tags:

```
img {
  border-size: 1px;
  border-style: solid;
}

span, div {
  color: red;
}
```

Selectors can reference a particular type of a tag (such as `img`), a period-prefixed name of a class of tags (for example, `.photos`, which will apply to all tags with an inline `class=photos` parameter), or a combination of both (`img.company_logo`). Selector suffixes such as `:hover` or `:visited` may also be used to make the selector match only under certain circumstances, such as when the mouse hovers over the content or when a particular displayed hyperlink has already been visited before.

So-called *complex selectors*² are an interesting feature introduced in CSS2 and extended in CSS3. They allow any given ruleset to apply only to tags with particular strings appearing in parameter values or that are positioned in a particular relation to other markup. One example of such a selector is this:

```
a[href^="ftp:"] {
  /* Styling applicable only to FTP links. */
}
```

NOTE *Oh, while we are at it: As evident in this example, C-style `/*...*/` comment blocks are permitted in CSS syntax anywhere outside a quoted string. On the flip side, `//`-style comments are not recognized at all.*

Property Definitions

Inside the `{ ... }` block that follows a selector, as well as inside the *style* parameter attached to a specific tag, any number of *name: value* rules can be used to redefine almost every aspect of how the affected markup is displayed. Visibility, shape, color, screen position, rendering order, local or remote typeface, and even any additional text (*content* property supported on certain pseudo-classes) and mouse cursor shape are all up for grabs.* Simple types of automation, such as counters for numbered lists, are available through CSS rules as well.

Property values can be formatted as the following:

- **Raw text** This method is used chiefly to specify numerical values (with optional units), RGB vectors and named colors, and other predefined keywords (“absolute,” “left,” “center,” etc.).
- **Quoted strings** Single or double quotes should be placed around any nonkeyword values, but there is little consistency in how this rule is enforced. For example, quoting is not required around typeface names or certain uses of URLs, but it is necessary for the aforementioned *content* property.
- **Functional notation** Two parameter-related pseudo-functions are mentioned in the original CSS specification: *rgb(...)*, for converting individual RGB color values into a single color code, and *url(...)*, required for URLs in most but not all contexts. On top of this, several more pseudo-functions have been rolled out in recent years, including *scale(...)*, *rotate(...)*, or *skew(...)*.

A proprietary *expression(...)* function is also available in Internet Explorer; it permits JavaScript statements to be inserted within CSS. This function is one of the most important reasons why attacker-controlled stylesheets can be a grave security risk.

@ Directives and XBL Bindings

In addition to selectors and properties, several @-prefixed directives are recognized in stand-alone stylesheets. All of them modify the meaning of the stylesheet; for example, by specifying the namespace or the display media that the stylesheet should be applied to. But two special directives also affect the behavior of the parsing process. The first of these is *@charset*, which sets the charset of the current CSS block; the other is *@import*, which inserts an external file into the stylesheet.

*The ability to redefine mouse cursors using an arbitrary bitmap has predictably resulted in some security bugs. An oversized cursor combined with script-based mouse position tracking could be used to obscure or replace important elements of the browser UI and trick the user into doing something dangerous.

The `@import` directive itself serves as a good example of the idiosyncrasies of CSS parsing; the parser views all of the following examples as equivalent:

```
@import "foo.css";
@import url('foo.css');
@import 'foo.css';
```

In Firefox, external content directives, including JavaScript code, may be also loaded from an external source using the `-moz-binding` property, a vendor-specific way to weave XML Binding Language³ files (an obscure method of providing automation to XML content) into the document. There is some talk of supporting XBL in other browsers, too, at which point the name of the property would change and the XSS risk may or may not be addressed in some way.

NOTE *As can be expected, the handling of pseudo-URLs in `@import`, `url(...)` and other CSS-based content inclusion schemes is a potential security risk. While most current browsers do not accept scripting-related schemes in these contexts, Internet Explorer 6 allows them without reservations, thereby creating a code injection vector if the URL is not validated carefully enough.*

Interactions with HTML

It follows from the discussion in the previous chapter that for any stylesheets inlined in HTML documents, HTML parsing is performed first and is completely independent of CSS syntax rules. Therefore, it is unsafe to place certain HTML syntax characters inside CSS properties, as in the following example, even when quoted properly. A common mistake is permitting this:

```
<style>
some_descriptor {
  background: url('http://www.example.com/</style><h1>Gotcha!');
}
</style>
```

We'll discuss a way to encode problematic characters in stylesheets shortly, but first, let's have a quick look at another very distinctive property of CSS.

Parser Resynchronization Risks

An undoubtedly HTML-inspired behavior that sets CSS apart from most other languages is that compliant parsers are expected to continue after encountering a syntax error and restart at the next matching curly bracket (some superficial nesting-level tracking is mandated by the spec). In particular, the following stylesheet snippet, despite being obviously malformed, will still apply the specified border style to all `<img>` tags:

```
a {
  $$$ This syntax makes absolutely no sense $$$
  !(@*#)!!@ 123
}
```

```
img {  
  border: 1px solid red;  
}
```

This unusual behavior creates an opportunity to exploit parser incompatibilities in an interesting way: If there is any way to derail a particular CSS implementation with inputs that seem valid to other parsers, the resynchronization logic may cause the attacked browser to resume parsing at an incorrect location, such as in the middle of an attacker-supplied string.

A naïve illustration of this issue may be Internet Explorer’s support for multiline string literals. In this browser, it is seemingly safe not to scrub CR and LF characters in user-supplied CSS strings, so some webmasters may allow it. Unfortunately, the same pattern will cause any other browser to resume at an unexpected offset and interpret the *evil_rule* ruleset:

```
some_benign_selector {  
  content: 'Attacker-controlled text...  
           } evil_rule { margin-left: -1000px; }';  
}
```

The support for multiline strings is a Microsoft-specific extension, and the aforementioned problem is easily fixed by avoiding such noncompliant syntax to begin with. Unfortunately, other desynchronization risks are introduced by the standard itself. For example, recall complex selectors: This CSS3 syntax makes no sense to pre-CSS3 parsers. In the following example, an older implementation may bail out after encountering an unexpected angle bracket and resume parsing from the attacker-supplied *evil_rule* instead:

```
a[href^=''] evil_rule { margin-left: -1000px; }' ] {  
  /* Harmless, validated rules here. */  
}
```

The still-popular browser Internet Explorer 6 would be vulnerable to this trick.

Character Encoding

To make it possible to quote reserved or otherwise problematic characters inside strings, CSS offers an unorthodox escaping scheme: a backslash (\) followed by one to six hexadecimal digits. For example, according to this scheme, the letter *ø* may be encoded as “\65”, “\065”, or “\000065”. Alas, only the last syntax, “\000065”, will be unambiguous if the next character happens to be a valid hexadecimal digit; encoding “teak” as “\65ak” would not work as expected, because the escape sequence would be interpreted as “\65A”, an Arabic sign in the Unicode character map.

To avoid this problem, the specification embraces an awkward compromise: A whitespace can follow an escape sequence and will be interpreted as a terminator, and then removed from the string (e.g., “\65 ak”). Regrettably, more familiar and predictable fixed-length C-style escape sequences such as \x65 cannot be used instead.

In addition to the numerical escaping scheme, it is also possible to place a backslash in front of a character that is not a valid hexadecimal digit. In this case, the subsequent character will be treated as a literal. This mechanism is useful for encoding quote characters and the backslash itself, but it should not be used to escape HTML control characters such as angle brackets. The aforementioned precedence of HTML parsing over CSS parsing renders this approach inadequate.

In a bizarre twist, due to somewhat ambiguous guidance in the W3C drafts, many CSS parsers recognize arbitrary escape sequences in locations other than quote-enclosed strings. To add insult to injury, in Internet Explorer, the substitution of these sequences apparently takes place before the pseudo-function syntax is parsed, effectively making the following two examples equivalent:

```
color: expression(alert(1))
```

```
color: expression\028 alert \028 1 \029 \029
```

Even more confusingly, in a misguided bid to maintain fault tolerance, Microsoft’s implementation does not recognize backslash escape codes inside *url(...)* values; this is, once more, to avoid hurting the feelings of users who type the wrong type of a slash when specifying a URL.

These and similar quirks make the detection of known dangerous CSS syntax extremely error prone.

Security Engineering Cheat Sheet

When Loading Remote Stylesheets

- ☑ You are linking the security of your site to the originating domain of the stylesheet. Even in browsers that do not support JavaScript expressions inside stylesheets, features such as conditional selectors and `url(...)` references can be used to exfiltrate portions of your site.⁴
- ☑ When in doubt, make a local copy of the data instead.
- ☑ On HTTPS sites, require stylesheets to be served over HTTPS as well.

When Putting Attacker-Controlled Values into CSS

- ☑ **Strings and URLs inside stand-alone blocks.** Always use quotes. Backslash-escape all control characters (0x00–0x1F), “\”, “<”, “>”, “[”, “]”, and quotes using numerical codes. It is also preferable to escape high-bit characters. For URLs, consult the cheat sheet in Chapter 2 to avoid code injection vulnerabilities.
- ☑ **Strings in style parameters.** Multiple levels of escaping are involved. The process is error prone, so do not attempt it unless absolutely necessary. If it is unavoidable, apply the above CSS escaping rules first and then apply HTML parameter encoding to the resulting string.
- ☑ **Nonstring attributes.** Allow only whitelisted alphanumeric keywords and carefully validated numerical values. Do not attempt to reject known bad patterns instead.

When Filtering User-Supplied CSS

- ☑ Remove all content outside of functional rulesets. Do not preserve or generate user-controlled comment blocks, @-directives, and so on.
- ☑ Carefully validate selector syntax, permitting only alphanumerics; underscores; white-spaces; and correctly positioned colons, periods, and commas before “{”. Do not permit complex text-matching selectors; they are unsafe.
- ☑ Parse and validate every rule in the { ... } block. Permit only whitelisted properties with well-understood consequences and confirm that they take expected, known safe values. Note that strings passed to certain properties may sometimes be interpreted as URLs even in the absence of a `url(...)` wrapper.
- ☑ Encode every parameter value using the rules outlined earlier in this section. Bail out on any syntax abnormalities.
- ☑ Keep in mind that unless specifically prevented from doing so, CSS may position user content outside the intended drawing area or redefine the appearance of any part of the UI of your application. The safest way to avoid this problem is to display the untrusted content inside a separate frame.

When Allowing User-Specified Class Values on HTML Markup

- ☑ Ensure that user-supplied content can’t reuse class names that are used for any part of the application UI. If a separate frame is not being used, it’s advisable to maintain separate namespace prefixes.

6

BROWSER-SIDE SCRIPTS

The first browser scripting engine debuted in Netscape Navigator around 1995, thanks to the work of Brendan Eich. The integrated Mocha language, as it was originally called, gave web developers the ability to manipulate HTML documents, display simple, system-level dialogs, open and reposition browser windows, and use other basic types of client-side automation in a hassle-free way.

While iterating through beta releases, Netscape eventually renamed Mocha LiveScript, and after an awkward branding deal was struck with Sun Microsystems, JavaScript was chosen as the final name. The similarities between Brendan's Mocha and Sun's Java were few, but the Netscape Corporation bet that this odd marketing-driven marriage would secure JavaScript's dominance in the more lucrative server world. It made this sentiment clear

in a famously confusing 1995 press release that introduced the language to the world and immediately tried to tie it to an impressive range of random commercial products:¹

Netscape and Sun Announce JavaScript, the Open, Cross-Platform Object Scripting Language for Enterprise Networks and the Internet

[. . .]

Netscape Navigator Gold 2.0 enables developers to create and edit JavaScript scripts, while Netscape LiveWire enables JavaScript programs to be installed, run and managed on Netscape servers, both within the enterprise and across the Internet. Netscape LiveWire Pro adds support for JavaScript connectivity to high-performance relational databases from Illustra, Informix, Microsoft, Oracle and Sybase. Java and JavaScript support are being built into all Netscape products to provide a unified, front-to-back, client/server/tool environment for building and deploying live online applications.

Despite Netscape's misplaced affection for Java, the value of JavaScript for client-side programming seemed clear, including to the competition. In 1996 Microsoft responded by shipping a near-verbatim copy of JavaScript in Internet Explorer 3.0 along with a counterproposal of its own: a Visual Basic-derived language dubbed VBScript. Perhaps because it was late to the party, and perhaps because of VBScript's clunkier syntax, Microsoft's alternative failed to gain prominence or even any cross-browser support. In the end, JavaScript secured its position in the market, and in part due to Microsoft's failure, no new scripting languages have been attempted in mainstream browsers since.

Encouraged by the popularity of the JavaScript language, Netscape handed over some of the responsibility for maintaining it to an independent body, the European Computer Manufacturers Association (ECMA). The new overseers successfully released ECMAScript, 3rd edition in 1999² but had substantially more difficulty moving forward from there. The 4th edition, an ambitious overhaul of the language, was eventually abandoned after several years of bickering between the vendors, and a scaled-down 5th edition,³ published in 2009, still enjoys only limited (albeit steadily improving) browser support. The work on a new iteration, called "Harmony," begun in 2008, still has not been finalized. Absent an evolving and widely embraced standard, vendor-specific extensions of the language are common, but they usually cause only pain.

Basic Characteristics of JavaScript

JavaScript is a fairly simple language meant to be interpreted at runtime. It has vaguely C-influenced syntax (save for pointer arithmetic); a straightforward classless object model, said to be inspired by a little-known programming language named Self; automatic garbage collection; and weak, dynamic typing.

JavaScript as such has no built-in I/O mechanisms. In the browser, limited abilities to interact with the host environment are offered through a set

of predefined methods and properties that map to native code inside the browser, but unlike what can be seen in many other programming languages, these interfaces are fairly limited and purpose built.

Most of the core features of JavaScript are fairly unremarkable and should be familiar to developers already experience with C, C++, or, to a lesser extent, Java. A simple JavaScript program might look like this:

```
var text = "Hi mom!";

function display_string(str) {
    alert(str);
    return 0;
}

// This will display "Hi mom!".
display_str(text);
```

Because it is beyond the scope of this book to provide a more detailed overview of the semantics of JavaScript, we'll summarize only some of its more unique and security-relevant properties later in this chapter. For readers looking for a more systematic introduction to the language, Marijn Haverbeke's *Eloquent JavaScript* (No Starch Press, 2011) is a good choice.

Script Processing Model

Every HTML document displayed in a browser—be it in a separate window or in a frame—is given a separate instance of the JavaScript execution environment, complete with an individual namespace for all global variables and functions created by the loaded scripts. All scripts executing in the context of a particular document share this common sandbox and can also interact with other contexts through browser-supplied APIs. Such cross-document interactions must be done in a very explicit way; accidental interference is unlikely. Superficially, script-isolation rules are reminiscent of the process-compartmentalization model in modern multitasking operating systems but a lot less inclusive.

Within a particular execution context, all encountered JavaScript blocks are processed individually and almost always in a well-defined order. Each code block must consist of any number of self-contained, well-formed syntax units and will be processed in three distinct, consequent steps: parsing, function resolution, and code execution.

Parsing

The parsing stage validates the syntax of the script block and, usually, converts it to an intermediate binary representation, which can be subsequently executed at a more reasonable speed. The code has no global effects until this step completes successfully. In case of syntax errors, the entire problematic block is abandoned, and the parser proceeds to the next available chunk of code.

To illustrate the behavior of a compliant JavaScript parser, consider the following HTML snippet:

block #1:

```
<script>
var my_variable1 = 1;
var my_variable2 =
</script>
```

block #2:

```
<script>
2;
</script>
```

Contrary to what developers schooled in C may be accustomed to, the above sequence is not equivalent to the following snippet:

```
<script>
var my_variable1 = 1;
var my_variable2 = 2;
</script>
```

This is because *<script>* blocks are not concatenated before parsing. Instead, the first script segment will simply cause a syntax error (an assignment with a missing right-hand value), resulting in the entire block being ignored and not reaching execution stage. The fact that the whole segment is abandoned before it can have any global side effects also means that the original example is not equivalent to this:

```
<script>
var my_variable1 = 1;
</script>

<script>
2;
</script>
```

This sets JavaScript apart from many other scripting languages such as Bash, where the parsing stage is not separated from execution in such a strong way.

What will happen in the original example provided earlier in this section is that the first block will be ignored but the second one (*<script>2;</script>*) will be parsed properly. That second block will amount to a no-op when executed, however, because it uses a pure, numerical expression as a code statement.

Function Resolution

Once the parsing stage is completed successfully, the next step involves registering every named, global function that the parser found within the currently processed block. Past this point, each function found will be reachable

from the subsequently executed code. Because of this extra pre-execution step, the following syntax will work flawlessly (contrary to what programmers may be accustomed to in C or C++, *hello_world()* will be registered before the first code statement—a call to said function—is executed):

```
<script>
hello_world();

function hello_world() {
    alert('Hi mom!');
}
</script>
```

On the other hand, the modified example below will not have the desired effect:

```
<script>
hello_world();
</script>

<script>
function hello_world() {
    alert('Hi mom!');
}
</script>
```

This modified case will fail with a runtime error because individual blocks of code are not processed simultaneously but, rather, are looked at based on the order in which they are made available to the JavaScript engine. The block that defines *hello_world()* will not yet be parsed when the first block is already executing.

To further complicate the picture, the mildly awkward global name resolution model outlined here applies only to functions, not to variable declarations. Variables are registered sequentially at execution time, in a way similar to other interpreted scripting languages. Consequently, the following code sample, which merely replaces our global *hello_world()* with an unnamed function assigned to a global variable, will not work as planned:

```
<script>
hello_world();

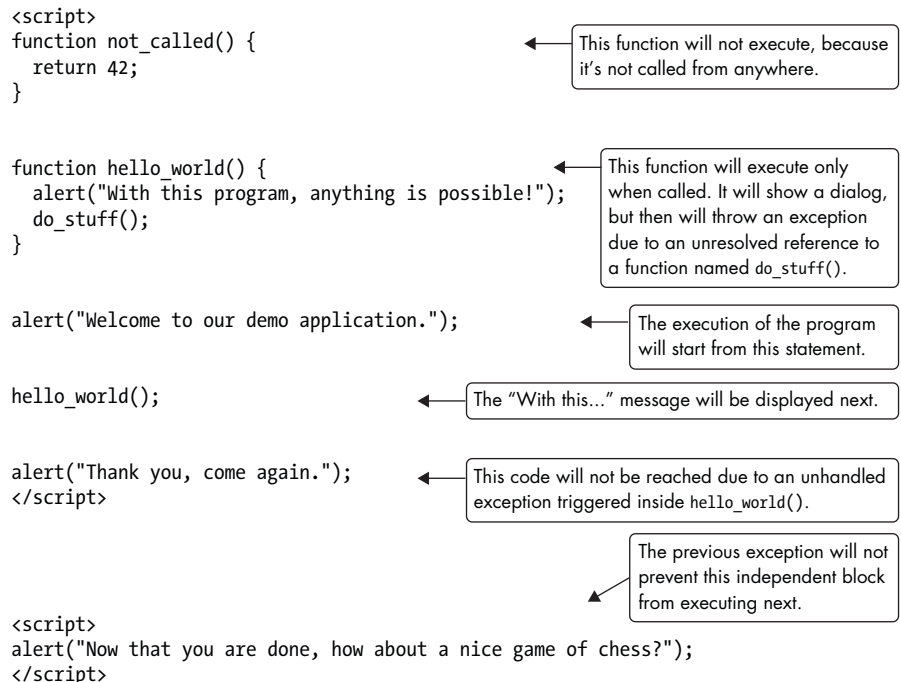
var hello_world = function() {
    alert('Hi mom!');
}
</script>
```

In this case, the assignment to the *hello_world* variable will not be done by the time the *hello_world()* call is attempted.

Code Execution

Once function resolution is completed, the JavaScript engine normally proceeds with the ordered execution of all statements outside of function blocks. The execution of a script may fail at this point due to an unhandled exception or for a couple of other, more esoteric reasons. If such an error is encountered, however, any resolved functions within the offending code block will remain callable, and any effects of the already executed code will persist in the current scripting context.

Exception recovery and several other JavaScript execution characteristics are illustrated by the following lengthy but interesting code snippet:



Try to follow this example on your own and see if you agree with the annotations provided on the right.

As should be evident from this exercise, any unexpected and unhandled exceptions have an unusual consequence: They may leave the application in an inconsistent but still potentially executable state. Because exceptions are meant to prevent error propagation caused by unanticipated errors, this design is odd—especially given that on many other fronts (such as the ban on *goto* statements), JavaScript exhibits a more fundamentalist stance.

Execution Ordering Control

In order to properly analyze the security properties of certain common web application design patterns, it is important to understand the JavaScript engine's execution ordering and timing model. Thankfully, this model is remarkably sane.

Virtually all JavaScript living within a particular execution context is executed synchronously. The code can't be reentered due to an external event while it is still executing, and there is no support for threads that would be able to simultaneously modify any shared memory. While the execution engine is busy, the processing of events, timers, page navigation requests, and so on, is postponed; in most cases, the entire browser, or at least the HTML renderer, will also remain largely unresponsive. Only once the execution stops and the scripting engine enters an idle state will the processing of queued events resume. At this point, the JavaScript code may be entered again.

Further, JavaScript offers no *sleep(...)* or *pause(...)* function to temporarily release the CPU and later resume execution from the same location. Instead, if a programmer desires to postpone the execution of a script, it is necessary to register a timer to initiate a new execution flow later on. This flow will need to start at the beginning of a specified handler function (or at the beginning of an ad hoc, self-contained snippet of code provided when setting up a timer). Although these design decisions can be annoying, they substantially reduce the risk of race conditions in the resulting code.

NOTE *There are several probably unintentional loopholes in this synchronous execution model. One of them is the possibility of code execution while the execution of another piece of JavaScript is temporarily suspended after calling `alert(...)` or `showModalDialog(...)`. Such corner cases do not come into play very often, though.*

The disruptive, browser-blocking behavior of busy JavaScript loops requires the implementation of some mitigation on the browser level. We will explore these mitigations in detail in Chapter 14. For now, suffice it to say that they have another highly unusual consequence: Any endless loop may, in fact, terminate, in a fashion similar to throwing an unhandled exception. The engine will then return to the idle state but will remain operational, the offending code will remain callable, and all timers and event handlers will stay in place.

When triggered on purpose by the attacker, the ability to unexpectedly terminate the execution of CPU-intensive code may put the application in an inconsistent state by aborting an operation that the author expects to always complete successfully. And that's not all: Another, closely related consequence of these semantics should become evident in "JavaScript Object Notation and Other Data Serializations" on page 104.

Code and Object Inspection Capabilities

The JavaScript language has a rudimentary provision for inspecting the decompiled source code of any nonnative functions, simply by invoking the *toString()* or *toSource()* method on any function that the developer wishes to examine. Beyond that capability, opportunities to inspect the flow of programs are limited. Applications may leverage access to the in-memory representation of their host document and look up all inlined `<script>` blocks, but there is no direct visibility into any remotely loaded or dynamically generated code. Some insight into the call stack may also be gained through a nonstandard *caller* property, but there is also no way to tell which line of code is being currently executed or which one is coming up next.

The ability to dynamically create new JavaScript code is a more prominent part of the language. It is possible to instruct the engine to synchronously interpret strings passed to the built-in *eval(...)* function. For example, this will display an alert dialog:

```
eval("alert(\"Hi mom!\")")
```

Syntax errors in any input text provided to *eval(...)* will cause this function to throw an exception. Similarly, if parsing succeeds, any unhandled exceptions thrown by the interpreted code will be passed down to the caller. Finally, in the absence of syntax errors or runtime problems, the value of the last statement evaluated by the engine while executing the supplied code will be used as the return value of *eval(...)* itself.

In addition to this function, other browser-level mechanisms can be leveraged to schedule deferred parsing and execution of new JavaScript blocks once the execution engine returns to the idle state. Examples of such mechanisms include timers (*setTimeout*, *setInterval*), event handlers (*onclick*, *onload*, and so on), and interfaces to the HTML parser itself (*innerHTML*, *document.write(...)*, and such).

Whereas the ability to inspect the code is somewhat underhanded, runtime object introspection capabilities are well developed in JavaScript. Applications are permitted to enumerate almost any object method or property using simple *for ... in* or *for each ... in* iterators and can leverage operators such as *typeof*, *instanceof*, or “strictly equals” (*===*) and properties such as *length* to gain additional insight into the identity of every discovered item.

All of the foregoing features make it largely impossible for scripts running in the same context to keep secrets from each other. The functionality also makes it more difficult to keep secrets across document contexts, a problem that browser vendors had to combat for a very long time—and that, as you’ll learn in Chapter 11, is still not completely a thing of the past.

Modifying the Runtime Environment

Despite the relative simplicity of the JavaScript language, executed scripts have many unusual ways of profoundly manipulating the behavior of their own JavaScript sandbox. In some rare cases, these behaviors can impact other documents, as well.

Overriding Built-Ins

One of the more unusual tools at the disposal of a rogue script is the ability to delete, overwrite, or shadow most of the built-in JavaScript functions and virtually all browser-supplied I/O methods. For example, consider the behavior of the following code:

```
// This assignment will not trigger an error.  
eval = alert;  
  
// This call will unexpectedly open a dialog prompt.  
eval("Hi mom!");
```

And this is just where the fun begins. In Chrome, Safari, and Opera, it is possible to subsequently remove the *eval(...)* function altogether, using the *delete* operator. Confusingly, attempting the same in Firefox will restore the original built-in function, undoing the effect of the original override. Finally, in Internet Explorer, the deletion attempt will generate a belated exception that seems to serve no meaningful purpose at that point.

Further along these lines, almost every object, including built-ins such as *String* or *Array*, has a freely modifiable prototype. This prototype is a master object from which all existing and future object instances derive their methods and properties (forming a crude equivalent of class inheritance present in more fully featured programming languages). The ability to tamper with object prototypes can cause rather counterintuitive behavior of newly created objects, as illustrated here:

```
Number.prototype.toString = function() {  
    return "Gotcha!";  
};  
  
// This will display "Gotcha!" instead of "42":  
alert(new Number(42));
```

Setters and Getters

More interesting features of the object model available in contemporary dialects of JavaScript are *setters* and *getters*: ways to supply custom code that handles reading or setting properties of the host object. Although not as powerful as operator overloading in C++, these can be used to make existing objects or object prototypes behave in even more confusing ways. In the following snippet, the acts of setting the object property and reading it back later on are both subverted easily:

```
var evil_object = {  
    set foo() { alert("Gotcha!"); },  
    get foo() { return 2; }  
};  
  
// This will display "Gotcha!" and have no other effect.  
evil_object.foo = 1;  
  
// This comparison will fail.  
if (evil_object.foo != 1) alert("What's going on?!");
```

NOTE *Setters and getters were initially developed as a vendor extension but are now standardized under ECMAScript edition 5. The feature is available in all modern browsers but not in Internet Explorer 6 or 7.*

Impact on Potential Uses of the Language

As a result of the techniques discussed in the previous two sections, a script executing inside a context once tainted by any other untrusted content has no reliable way to examine its operating environment or take corrective

action; even the behavior of simple conditional expressions or loops can't necessarily be relied upon. The proposed enhancements to the language are likely to make the picture even more complicated. For example, the failed proposal for ECMAScript edition 4 featured full-fledged operator overloading, and this idea may return.

Even more interestingly, these design decisions also make it difficult to inspect any execution context from outside the per-page sandbox. For example, blind reliance on the reliability of the *location* object of a potentially hostile document has led to a fair number of security vulnerabilities in browser plug-ins, JavaScript-based extensions, and several classes of client-side web application security features. These vulnerabilities eventually resulted in the development of browser-level workarounds designed to partially protect this specific object against sabotage, but most of the remaining object hierarchy is up for grabs.

NOTE *The ability to tamper with one's own execution context is limited in the "strict" mode of ECMAScript edition 5. This mode is not fully supported in any browser as of this writing, however, and is meant to be an opt-in, discretionary mechanism.*

JavaScript Object Notation and Other Data Serializations

A very important syntax structure in JavaScript is its very compact and convenient in-place object serialization, known as JavaScript Object Notation, or JSON (RFC 4627⁴). This data format relies on overloading the meaning of the curly bracket symbol (`{}`). When such a brace is used to open a fully qualified statement, it is treated in a familiar way, as the start of a nested code block. In an expression, however, it is assumed to be the beginning of a serialized object. The following example illustrates a correct use of this syntax and will display a simple prompt:

```
var impromptu_object = {  
    "given_name"      : "John",  
    "family_name"     : "Smith",  
    "lucky_numbers"  : [ 11630, 12067, 12407, 12887 ]  
};  
  
// This will display "John".  
alert(impromptu_object.given_name);
```

In contrast to the unambiguous serializations of numbers, strings, or arrays, the overloading of the curly bracket means that JSON blocks will not be recognized properly when used as a standalone statement. This may seem insignificant, but it is an advantage: It prevents any server-supplied responses that comply with this syntax from being meaningfully included across domains via `<script src=...>`.^{*} The listing that follows will cause a syntax error, ostensibly

^{*}Unlike most other content inclusion schemes available to scripts (such as *XMLHttpRequest*), `<script src=...>` is not subject to the cross-domain security restrictions outlined in Chapter 9. Therefore, the mechanism is a security risk whenever ambient authority credentials, such as cookies, are used by the server to dynamically generate user-specific JavaScript code. This class of vulnerabilities is unimaginatively referred to as *cross-site script inclusion*, or *XSSI*.

due to an illegal quote (❶) in what the interpreter attempts to treat as a code label,* and will have no measurable side effects:

```
<script>
{
❶  "given_name"   : "John",
    "family_name" : "Smith",
    "lucky_numbers" : [ 11630, 12067, 12407, 12887 ]
};
</script>
```

NOTE *The inability to include JSON via `<script src=...>` is an interesting property, but it is also a fragile one. In particular, wrapping the response in parentheses or square brackets, or removing quotes around the labels, will render the syntax readily executable in a standalone block, which may have observable side effects. Given the rapidly evolving syntax of JavaScript, it is not wise to bank on this particular code layout always causing a parsing error in the years to come. That said, in many noncritical uses, this level of assurance will be good enough to rely on as a simple security mechanism.*

Once retrieved through a channel such as *XMLHttpRequest*, the JSON serialization can be quickly and effortlessly converted to an in-memory object using the *JSON.parse(...)* function in all common browsers, other than Internet Explorer. Unfortunately, for purposes of compatibility with Internet Explorer, and sometimes just out of custom, many developers resort to an equally fast yet far more dangerous hack:

```
var parsed_object = eval("(" + json_text + ")");
```

The problem with this syntax is that the *eval(...)* function used to compute the “value” of a JSON expression permits not only pure JSON inputs but any other well-formed JavaScript syntax to appear in the string. This can have undesirable, global side effects. For example, the function call embedded in this faux JSON response will execute:

```
{ "given_name": alert("Hi mom!") }
```

This behavior creates an additional burden on web developers to accept JSON payloads only from trusted sources and always to correctly escape feeds produced by their own server-side code. Predictably, failure to do so has contributed a fair number of application-level security bugs.

NOTE *The difficulty of getting *eval(...)* right is embodied by the JSON specification (RFC 4627) itself: The allegedly secure parser implementation included in that document unintentionally permits rogue JSON responses to freely increment or decrement any program variables that happen to consist solely of the letters “a”, “e”, “f”, “l”, “n”, “r”,*

*Somewhat unexpectedly, JavaScript supports C-style labeled statements, such as *my_label: alert("Hi mom!")*. This is interesting because for philosophical reasons, the language has no support for *goto* and, therefore, such a label can't be meaningfully referenced in most cases.

“s”, “t”, “u”, plus digits; that’s enough to spell “unsafe” and about 1,000 other common English words. The faulty regular expression legitimized in this RFC appears all over the Internet and will continue to do so.

Thanks to their ease of use, JSON serializations are ubiquitous in server-to-client communications across all modern web applications. The format is rivaled only by other, less secure string or array serializations and by JSONP.* All of these schemes are incompatible with *JSON.parse(...)*, however, and must rely on unsafe *eval(...)* to be converted to in-memory data. The other property of these formats is that, unlike proper JSON, they will parse properly when loaded with `<script src=...>` on a third-party page. This property is advantageous in some rare cases, but mostly it just constitutes an unobvious risk. For example, consider that even though loading an array serialization via a `<script>` tag normally has no measurable side effects, an attacker could, at least until recent improvements, modify the setters on an *Array* prototype to retrieve the supplied data. A common but often insufficient practice of prefixing a response with a *while(1);* loop to prevent this attack can backfire in interesting ways if you recall the possibility of endless loops terminating in JavaScript.

E4X and Other Syntax Extensions

Like HTML, JavaScript is quickly evolving. Some of the changes made to it over the years have been fairly radical and may end up turning text formats that were previously rejected by the parser into a valid JavaScript code. This, in turn, may lead to unexpected data disclosure, especially in conjunction with the extensive code and object inspection and modification capabilities discussed earlier in this chapter—and the ability to use `<script src=...>` to load cross-domain code.

One of the more notable examples of this trend is *ECMAScript for XML* (E4X),⁵ a completely unnecessary but elegant plan to incorporate XML syntax directly into JavaScript as an alternative to JSON-style serializations. In any E4X-compatible engine, such as Firefox, the following two snippets of code would be roughly equivalent:

```
// Normal object serialization
var my_object = { "user": {
    "given_name": "John",
    "family_name": "Smith",
    "id": make_up_value()
} };

// E4X serialization
var my_object = <user>
    <given_name>John</given_name>
    <family_name>Smith</family_name>
    <id>{ make_up_value() }</id>
</user>;
```

*JSONP literally means “JSON with padding” and stands for JSON serialization wrapped in some supplementary code that turns it into a valid, standalone JavaScript statement for convenience. Common examples may include a function call (e.g., *callback_function({ ...JSON data... })*) or a variable assignment (*var return_value = { ...JSON data... }*).

The unexpected consequence of E4X is that, under this regime, any well-formed XML document suddenly becomes a valid `<script src=...>` target that will parse as an expression-as-statement block. Moreover, if an attacker can strategically place “{” and “}” characters on an included page, or alter the setters for the right object prototype, the attacker may be able to extract user-specific text displayed in an unrelated document. The following example illustrates the risk:

```
<html xmlns="http://www.w3.org/1999/xhtml">
...
{ steal_stuff(                                     ← attacker-supplied string
...
  <span>User-specific secrets here</span>
...
) }                                                 ← attacker-supplied string
...
</html>
```

To their credit, after several years of living with the flaw, Firefox developers decided to disallow any E4X statements that span the entirety of any parsed script, partly closing this loophole. Nevertheless, the fluidity of the language is evident, and it casts some doubt on the robustness of using of JSON responses as a defense against cross-domain script inclusion. The moment a third meaning is given to the “{” symbol or quotes-as-labels start having a purpose, the security of this server-to-client data exchange format will be substantially degraded. Be sure to plan ahead.

Standard Object Hierarchy

The JavaScript execution environment is structured around an implicit root object, which is used as the default namespace for all global variables and functions created by the program. In addition to a handful of language-mandated built-ins, this namespace is prepopulated with a hierarchy of functions that implement input and output capabilities in the browser environment. These capabilities include manipulating browser windows (*open(...)*, *close()*, *moveTo(...)*, *resizeTo(...)*, *focus()*, *blur()*, and such); configuring JavaScript timers (*setTimeout(...)*, *setInterval(...)*, and so on); displaying various UI prompts (*alert(...)*, *prompt(...)*, *print(...)*); and performing a variety of other vendor-specific and frequently risky functions, such as accessing the system clipboard, creating bookmarks, or changing the home page.

The top-level object also provides JavaScript references to root objects belonging to related contexts, including the parent frame (*parent*), the top-level document in the current browser window (*top*), the window that created the current one (*opener*), and all subframes of the current document (*frames[]*). Several circular references to the current root object itself are also included—say, *window* and *self*. In browsers other than Firefox, elements with specified *id* or *name* parameters will be automatically registered in this namespace, too, permitting syntax such as this:

```

...
```

```
<script>
    alert(hello.src);
</script>
```

Thankfully, in case of any name conflicts with JavaScript variables or built-ins, *id* data will not be given precedence, largely avoiding any possible interference between otherwise sanitized, user-supplied markup and in-document scripts.

The remainder of the top-level hierarchy consists primarily of a couple of distinguished children objects that group browser API features by theme:

***location* object**

This is a collection of properties and methods that allow the program to read the URL of the current document or initiate navigation to a new one. This last action, in most cases, is lethal to the caller: The current scripting context will be destroyed and replaced with a new one shortly thereafter. Updating just the fragment identifier (*location.hash*) is an exception to this rule, as explained in Chapter 2.

Note that when using *location.** data to construct new strings (HTML and JavaScript code in particular), it is unsafe to assume that it is escaped in any specific way. Internet Explorer will keep angle brackets as is in the *location.search* property (which corresponds to the URL query string). Chrome, on the other hand, will escape them, but it will glance over double quotes (") or backslashes. Most browsers also do not apply any escaping to the fragment ID.

***history* object**

This hierarchy provides several infrequently used methods for moving through the per-window browsing history, in a manner similar to clicking the “back” and “forward” buttons in the browser UI. It is not possible to directly examine any of the previously visited URLs; the only option is to navigate to the history blindly by providing numerical offsets, such as *history.go(-2)*. (Some recent additions to this hierarchy will be discussed in Chapter 17.)

***screen* object**

A basic API for examining the dimensions of the screen and the browser window, monitor DPI, color depth, and so on. This is offered to help websites optimize the presentation of a page for a particular display device.

***navigator* object**

An interface for querying the browser version, the underlying operating system, and the list of installed plug-ins.

***document* object**

By far the most complex of the hierarchies, this is a doorway to the Document Object Model⁶ of the current page; we will have a look at this model in the following section. A couple of functions not related to document structure also appear under the *document* hierarchy, usually due to arbitrary design decisions. Examples include *document.cookie* for manipulating cookies, *document.write(...)* for appending HTML to the current page, and *document.execCommand(...)* for performing certain WYSIWYG editing tasks.

NOTE *Interestingly, the information available through the navigator and screen objects is sufficient to uniquely fingerprint many users with a high degree of confidence. This long-known property is emphatically demonstrated by Panopticklick, a project of the Electronic Frontier Foundation: <https://panopticklick.eff.org/>.*

Several other language-mandated objects offer simple string-processing or arithmetic capabilities. For example, `Math.random()` implements an unsafe, predictable pseudo-random number generator (a safe PRNG alternative is unfortunately not available at this time in most browsers*), while `String.fromCharCode()` can be used to convert numerical values into Unicode strings. In privileged execution contexts, which are not reachable by normal web applications, a fair number of other task-specific objects will also appear.

NOTE *When accessing any of the browser-supplied objects, it is important to remember that while JavaScript does not use NUL-terminated ASCII strings, the underlying browser (written in C or C++) sometimes will. Therefore, the outcomes of assigning NUL-containing strings to various DOM properties, or supplying them to native functions, may be unpredictable and inconsistent. Almost all browsers truncate assignments to location.* at NUL, but only some engines will do the same when dealing with DOM *.innerHTML.*

The Document Object Model

The Document Object Model, accessible through the `document` hierarchy, provides a structured, in-memory representation of the current document as mapped out by the HTML parser. The resulting object tree exposes all HTML elements on the page, their tag-specific methods and properties, and the associated CSS data. This representation, not the original HTML source, is used by the browser to render and update the currently displayed document.

JavaScript can access the DOM in a very straightforward way, similarly to any normal objects. For example, the following snippet will go to the fifth tag within the document's `<body>` block, look up the first nested subtag, and set that element's CSS color to red:

```
document.body.children[4].children[0].style.color = "red";
```

To avoid having to waddle through the DOM tree in order to get to a particular deeply nested element, the browser provides several document-wide lookup functions, such as `getElementById(...)` and `getElementsByName(...)`, as well as partly redundant grouping mechanisms such as `frames[]`, `images[]`, or `forms[]`. These features permit syntax such as the following two lines of code, both of which directly reference an element no matter where in the document hierarchy it happens to appear:

```
document.getElementsByTagName("input")[2].value = "Hi mom!";  
document.images[7].src = "/example.jpg";
```

*There are a recently added `window.crypto.getRandomValues(...)` API in Chrome and a currently nonoperational `window.crypto.random(...)` API in Firefox.

For legacy reasons, the names of certain HTML elements (**, *<form>*, *<embed>*, *<object>*, and *<applet>*) are also directly mapped to the *document* namespace, as illustrated in the following snippet:

```


<script>
  alert(document.hello.src);
</script>
```

Unlike in the more reasonable case of *name* and *id* mapping in the global namespace (see previous section), such *document* entries may clobber built-in functions and objects such as *getElementById* or *body*. Therefore, permitting user-specified tag names, for example for the purpose of constructing forms, can be unsafe.

In addition to providing access to an abstract representation of the document, many DOM nodes may expose properties such as *innerHTML* and *outerHTML*, which permit a portion of the document tree to be read back as a well-formed, serialized HTML string. Interestingly, the same property can be written to in order to replace any portion of the DOM tree with the result of parsing a script-supplied snippet of HTML. One example of that last use is this:

```
document.getElementById("output").innerHTML = "<b>Hi mom!</b>";
```

Every assignment to *innerHTML* must involve a well-formed and self-contained block of HTML that does not alter the document hierarchy outside the substituted fragment. If this condition is not met, the input will be coerced to a well-formed syntax before the substitution takes place. Therefore, the following example will not work as expected; that is, it will not display “Hi mom!” in bold and will not put the remainder of the document in italics:

```
some_element.innerHTML = "<b>Hi";
some_element.innerHTML += " mom!</b><i>";
```

Instead, each of these two assignments will be processed and corrected individually, resulting in a behavior equivalent to this:

```
some_element.innerHTML = "<b>Hi</b> mom!<i></i>";
```

It is important to note that the *innerHTML* mechanism should be used with extreme caution. In addition to being inherently prone to markup injection if proper HTML escaping is not observed, browser implementations of the DOM-to-HTML serialization algorithms are often imperfect. A recent (now fixed) example of such a problem in WebKit⁷ is illustrated here:

```
<textarea>
  &lt;/textarea&gt;&lt;script&gt;alert(1)&lt;/script&gt;
</textarea>
```

Because of the confusion over the semantics of `<textarea>`, this seemingly unambiguous input markup, when parsed to a DOM tree and then accessed through `innerHTML`, would be incorrectly read back as:

```
<textarea>
  </textarea><script>alert(1)</script>
</textarea>
```

In such a situation, even performing a no-op assignment of this serialization (such as `some_element.innerHTML += ""`) would lead to unexpected script injection. Similar problems tend to plague other browsers, too. For example, Internet Explorer developers working on the `innerHTML` code were unaware that MSHTML recognizes backticks (``) as quote characters and so ended up handling them incorrectly. In their implementation, the following markup:

```

```

would be reserialized as this:

```
<img src=test.jpg alt=``onload=alert(1)>
```

Individual bugs aside, the situation with `innerHTML` is pretty dire: Section 10.3 of the current draft of HTML5 simply acknowledges that certain script-created DOM structures are completely impossible to serialize to HTML and does not require browsers to behave sensibly in such a case. *Caveat emptor!*

Access to Other Documents

Scripts may come into possession of object handles that point to the root hierarchy of another scripting context. For example, by default, every context can readily reference *parent*, *top*, *opener*, and *frames[]*, all supplied to it in the top-level object. Calling the `window.open(...)` function to create a new window will also return a reference, and so will an attempt to look up an existing named window using this syntax:

```
var window_handle = window.open("", "window_name");
```

Once the program holds a handle pointing to another scripting context, it may attempt to interact with that context, subject to security checks discussed in Chapter 9. An example of a simple interaction might be as follows:

```
top.location.path = "/new_path.html";
```

or

```
frames[2].document.getElementById("output").innerHTML = "Hi mom!";
```

In the absence of a valid handle, JavaScript-level interaction with an unrelated document should not be possible. In particular, there is no way to look up unnamed windows opened in completely separate navigation flows, at least until their name is explicitly set by one of the visited pages (the *window.name* property permits this).

Script Character Encoding

JavaScript engines support several familiar, backslash-based string-encoding methods that can be employed to escape quote characters, HTML markup, and other problematic bits in the embedded text. These methods are as follows:

- C-style shorthand notation for certain control characters: `\b` for backspace, `\t` for horizontal tab, `\v` for vertical tab, `\f` for form feed, `\r` for CR, and `\n` for LF. This exact set of escape codes is recognized by both ECMAScript and the JSON RFC.
- Three-digit, zero-padded, 8-bit octal character codes with no prefix (such as `"\145"` instead of `"e"`). This C-inspired syntax is not a part of ECMAScript but is in practice supported by all scripting engines, both in normal code and in *JSON.parse(...)*.
- Two-digit, zero-padded, 8-bit hexadecimal character codes, prefixed with `"x"` (`"e"` becomes `"x65"`). Again, this scheme is not endorsed by ECMAScript or RFC 4627, but having its roots in the C language, it is widely supported in practice.
- Four-digit, zero-padded, 16-bit hexadecimal Unicode values, prefixed with `"u"` (`"e"` turns into `"u0065"`). This format is sanctioned by ECMAScript and RFC 4627 and is supported by all modern browsers.
- A backslash followed by any character other than an octal digit; `"b"`, `"t"`, `"v"`, `"f"`, `"r"`, or `"n"` characters used for other predefined escape sequences; and `"x"` or `"u"`. In this scheme, the subsequent character will be treated as a literal. ECMAScript permits this scheme to be used to escape only quotes and the backslash character itself, but in practice, any other value is accepted as well.

This approach is somewhat error prone, and as in the case of CSS, it should not be used to escape angle brackets and other HTML syntax delimiters. This is because JavaScript parsing takes place after HTML parsing, and the backslash prefix will be not treated in any special way by the HTML parser itself.

NOTE *Somewhat inexplicably, Internet Explorer does not recognize the vertical tab (`"\v"`) shorthand, thereby creating one of the more convenient (but very naughty!) ways to test for that particular browser:*

```
if ("\v" == "v") alert("Looks like Internet Explorer!");
```

Surprisingly, the Unicode-based escaping method (but not the other ones) is also recognized outside strings. Although the idea seems arbitrary, the behavior is a bit more sensible than with CSS: Escape codes can be used only in identifiers, and they will not work as a substitute for any syntax-sensitive symbols. Therefore, the following is possible:

```
\u0061lert("This displays a message!");
```

On the other hand, any attempt to substitute the parentheses or quotes in a similar fashion would fail.

Unlike in some C or C++ implementations, stray multiline string literals are not tolerated by any JavaScript engine. That said, despite a strongly worded prohibition in ECMAScript specs, there is one exception: A lone backslash at the end of a line may be used to join multiline literals seamlessly. This behavior is illustrated below:

```
var text = 'This syntax
           is invalid.';

var text = 'This syntax, on the other hand, \
           is OK in all browsers.';
```

Code Inclusion Modes and Nesting Risks

As should be evident from the earlier discussions in this chapter, there are several ways to execute scripts in the context of the current page. It is probably useful to enumerate some of the most common ones:

- Inline `<script>` blocks
- Remote scripts loaded with `<script src=...>*`
- *javascript:* URLs in various HTML parameters and in CSS
- CSS *expression(...)* syntax and XBL bindings in certain browsers
- Event handlers (*onload*, *onerror*, *onclick*, etc.)
- Timers (*setTimeout*, *setInterval*)
- *eval(...)* calls

Combining these methods often seems natural, but doing so can create very unexpected and dangerous parsing chains. For example, consider the transformation that would need to be applied to the value inserted by the server in place of *user_string* in this code:

```
<div onclick="setTimeout('do_stuff(\'user_string\')', 1000)">
```

* On both types of `<script>` blocks, Microsoft supports a pseudo-dialect called *JScript.Encode*. This mode can be selected by specifying a *language* parameter on the `<script>` tag and simply permits the actual script to be encoded using a trivial alphabet substitution cipher to make it unreadable to casual users. The mechanism is completely worthless from the security standpoint, as the “encryption” can be reverted easily.

It is often difficult to notice that the value will go through no fewer than three rounds of parsing! First, the HTML parser will extract the *onclick* parameter and put it into DOM; next, when the button is clicked, the first round of JavaScript parsing will extract the *setTimeout(...)* syntax; and finally, one second after the initial click, the actual *do_stuff(...)* sequence will be parsed and executed.

Therefore, in the example above, in order to survive the process, *user_string* needs to be double-encoded using JavaScript backslash sequences, and then encoded again using HTML entities, in that exact order. Any different approach will likely lead to code injection.

Another tricky escaping situation is illustrated here:

```
<script>
var some_value = "user_string";
...
setTimeout("do_stuff('" + some_value + "')", 1000);
</script>
```

Even though the initial assignment of *some_value* requires *user_string* to be escaped just once, the subsequent ad hoc construction of a second-order script in the *setTimeout(...)* parameter introduces a vulnerability if no additional escaping is applied beforehand.

Such coding patterns happen frequently in JavaScript programs, and they are very easy to miss. It is much better to consistently discourage them than to audit the resulting code.

The Living Dead: Visual Basic

Having covered most of the needed ground related to JavaScript, it's time for an honorable mention of the long-forgotten contender for the scripting throne. Despite 15 years of lingering in almost complete obscurity, browser-side VBScript is still supported in Internet Explorer. In most aspects, Microsoft's language is supposed to be functionally equivalent to JavaScript, and it has access to exactly the same Document Object Model APIs and other built-in functions as JavaScript. But, as one might expect, some tweaks and extensions are present—for example, a couple of VB-specific functions in place of the JavaScript built-ins.

There is virtually no research into the security properties of VBScript, the robustness of the parser, or its potential incompatibilities with the modern DOM. Anecdotal evidence suggests that the language receives no consistent scrutiny on Microsoft's end, either. For example, the built-in *MsgBox*⁸ can be used to display modal, always-on-top prompts with a degree of flexibility completely unheard of in the JavaScript world, leaving *alert(...)* in the dust.

It is difficult to predict how long VBScript will continue to be supported in this browser and what unexpected consequences for user and web application security it is yet to have. Only time will tell.

Security Engineering Cheat Sheet

When Loading Remote Scripts

As with CSS, you are linking the security of your site to the originating domain of the script. When in doubt, make a local copy of the data instead. On HTTPS sites, require all scripts to be served over HTTPS.

When Parsing JSON Received from the Server

Rely on *JSON.parse(...)* where supported. Do not use *eval(...)* or the *eval*-based implementation provided in RFC 4627. Both are unsafe, especially when processing data from third parties. A later implementation from the author of RFC 4627, *json2.js*,⁹ is probably okay.

When Putting User-Supplied Data Inside JavaScript Blocks

- ☑ **Stand-alone strings in `<script>` blocks:** Backslash-escape all control characters (0x00–0x1F), “\”, “<”, “>”, and quotes using numerical codes. It is also preferable to escape high-bit characters.

Do not rely on user-supplied strings to construct dynamic HTML. Always use safe DOM features such as *innerText* or *createTextNode(...)* instead. Do not use user-supplied strings to construct second-order scripts; avoid *eval(...)*, *setTimeout(...)*, and so on.

- ☑ **Stand-alone strings in separately served scripts:** Follow the same rules as for `<script>` blocks. If your scripts contain any sensitive, user-specific information, be sure to account for cross-site script inclusion risks; use reliable parser-busting prefixes, such as “`]]\n`”, near the beginning of a file or, at the very minimum, use a proper JSON serialization with no padding or other tweaks. Additionally, consult Chapter 13 for tips on how to prevent cross-site scripting in non-HTML content.
- ☑ **Strings in inlined event handlers, *javascript:* URLs, and so on:** Multiple levels of escaping are involved. Do not attempt this because it is error prone. If unavoidable, apply the above JS escaping rules first and then apply HTML or URL parameter encoding, as applicable, to the resulting string. Never use in conjunction with *eval(...)*, *setTimeout(...)*, *innerHTML*, and such.
- ☑ **Nonstring content:** Allow only whitelisted alphanumeric keywords and carefully validated numerical values. Do not attempt to reject known bad patterns instead.

When Interacting with Browser Objects on the Client Side

- ☑ **Generating HTML content on the client side:** Do not resort to *innerHTML*, *document.write(...)*, and similar tools because they are prone to introducing cross-site scripting flaws, often in unexpected ways. Use safe methods such as *createElement(...)* and *appendChild(...)* and properties such as *innerText* or *textContent* to construct the document instead.
- ☑ **Relying on user-controlled data:** Make no assumptions about the escaping rules applied to any values read back from the browser and, in particular, to *location* properties and other external sources of URLs, which are inconsistent and vary from one implementation to another. Always do your own escaping.

If You Want to Allow User-Controlled Scripts on Your Page

It is virtually impossible to do this safely. Experimental JavaScript rewriting frameworks, such as Caja (<http://code.google.com/p/google-caja/>), are the only portable option. Also see Chapter 16 for information on sandboxed frames, an upcoming alternative for embedding untrusted gadgets on web pages.

7

NON-HTML DOCUMENT TYPES

In addition to HTML documents, about a dozen other file formats are recognized and displayed by the rendering engines of modern web browsers; a list that is likely to grow over time.

Because of the powerful scripting capabilities available in some of these formats, and because of the antics of browser-content handling, the set of natively supported non-HTML inputs deserves a closer examination at this point, even if a detailed discussion of some of their less-obvious security consequences—such as *content sniffing*—will have to wait until Part II of this book.

Plaintext Files

Perhaps the most prosaic type of non-HTML document recognized by every single browser is a plaintext file. In this rendering mode, the input is simply displayed as is, typically using a nonproportional typeface, and save for optional character set transcoding, the data is not altered in any way.

All browsers recognize plaintext files served with *Content-Type: text/plain* in the HTTP headers. In all implementations but Internet Explorer, plaintext is also the fallback display method for headerless HTTP/0.9 responses and HTTP/1.x data with *Content-Type* missing; in both these cases, plaintext is used when all other content detection heuristics fail. (Internet Explorer unconditionally falls back to HTML rendering, true to the letter of Tim Berners-Lee's original protocol drafts.)

For the convenience of developers, most browsers also automatically map several other MIME types, including *application/javascript* and friends* or *text/css*, to plaintext. Interestingly, *application/json*, the value mandated for JSON responses in RFC 4627, is not on the list (perhaps because it is seldom used in practice).

Plaintext rendering has no specific security consequences. That said, due to a range of poor design decisions in other browser components and in third-party code, even seemingly harmless non-HTML formats are at a risk of being misidentified as, for example, HTML. Attacker-controlled plaintext documents are of special concern because their layout is often fairly unconstrained and therefore particularly conducive to being misidentified. Chapter 13 dissects these threats and provides advice on how to mitigate the risk.

Bitmap Images

Browser-rendering engines recognize direct navigation to the same set of bitmap image formats that are normally supported in HTML documents when loaded via the `<img>` tag, including JPEG, PNG, GIF, BMP, and a couple more. When the user navigates directly to such a resource, the decoded bitmap is shown in the document window, allowing the user little more than the ability to scroll, zoom in and out, and save the file to disk.

In the absence of *Content-Type* information, images are detected based on file header checks. When a *Content-Type* value is present, it is compared with about a dozen predefined image types, and the user is routed accordingly. But if an attempt to decode the image fails, file headers are used to make a second guess. It is therefore possible (but, for the reasons explored in Chapter 13, often unwise) to serve a GIF file as *image/jpeg*.

As with text files, bitmap images are a passive resource and carry no unusual security risks.† However, whenever serving user-supplied images, remember that attackers will have a degree of control over the data, even if the format is carefully validated and scaled or recompressed. Therefore, the concerns about such a document format being misinterpreted by a browser or a plug-in still remain.

* The official MIME type for JavaScript is *application/javascript*, as per RFC 4329, but about a dozen other values have been used in the past (e.g., *text/javascript*, *application/x-javascript*, *application/ecmascript*).

† Naturally, exploitable coding errors occasionally happen in all programs that deal with complex data formats, and image parsers are no exception.

Audio and Video

For a very long time, browsers had no built-in support for playing audio and video content, save for an obscure and oft-ridiculed `<bgsound>` tag in Internet Explorer, which to this day can be used to play simple MID or WAV files. In the absence of real, cross-browser multimedia playback functionality, audio and video were almost exclusively the domain of browser plug-ins, whether purpose-built (such as Windows Media Player or Apple QuickTime) or generic (Adobe Flash, Microsoft Silverlight, and so on).

The ongoing work on HTML5 seeks to change this through support for `<audio>` and `<video>` tags: convenient, scriptable methods to interface with built-in media decoders. Unfortunately, there is substantial vendor-level disagreement as to which video formats to support and what patent consequences this decision may have. For example, while many browsers already support Ogg Theora (a free, open source, but somewhat niche codec), spirited arguments surrounding the merits of supporting the very popular but patent- and royalty-encumbered H.264 format and the prospects of a new, Google-backed WebM alternative will probably continue for the foreseeable future.

As with other passive media formats (and unlike some types of plug-in-rendered content!), neither `<bgsound>` nor HTML5 multimedia are expected to have any unusual implications for web application security, as long as the possibility of content misidentification is mitigated appropriately.*

XML-Based Documents

Readers who found the handling of the formats discussed so far to be too sane for their tastes are in for a well-deserved treat. The largest and definitely most interesting family of browser-supported non-HTML document types relies on the common XML syntax and provides more than a fair share of interesting surprises.

Several of the formats belonging to this category are forwarded to specialized, single-purpose XML analyzers, usually based on the received *Content-Type* value or other simple heuristics. But more commonly, the payload is routed to the same parser that is relied upon to render XHTML documents and then displayed using this common pipeline.

In the latter case, the actual meaning of the document is determined by the URL-like *xmlns* namespace directives present in the markup itself, and the namespace parameter may have nothing to do with the value originally supplied in *Content-Type*. Quite simply, there is no mechanism that would prevent a document served as *application/mathml+xml* from containing nothing but XHTML markup and beginning with `<html xmlns="http://www.w3.org/1999/xhtml">`.

* But some far-fetched interactions between various technologies are a distinct possibility. For example, what if the `<audio>` tag supports raw, uncompressed audio and is pointed to a sensitive nonaudio document, and then the proposed HTML5 microphone API is used by another website to capture the resulting waveform and reconstruct the contents of the file?

In the most common scenario, the namespace for the entire XML file is defined only once and is attached to the top-level tag. In principle, however, any number of different *xmlns* directives may appear in a single file, giving different meanings to each section of the document. For example:

```
<html xmlns="http://www.w3.org/1999/xhtml">
  <u>Hello world!</u>
  <svg xmlns="http://www.w3.org/2000/svg">
    <line x1="0" y1="0" x2="100" y2="100" style="stroke: red" />
  </svg>
</html>
```

Faced with such input, the general-purpose renderer will usually do its best to make sense of all the recognized namespaces and assemble the markup into a single, consistent document with a normal Document Object Model representation. And, if any one of the recognized namespaces happens to support scripting, any embedded scripts will execute, too.

Because of the somewhat counterintuitive *xmlns* handling behavior, *Content-Type* is not a suitable way to control how a particular XML document will be parsed; the presence of a particular top-level *xmlns* directive is also not a guarantee that no other data formats will be honored later on. Any attacker-controlled XML-based formats must therefore be handled with care and sanitized very thoroughly.

Generic XML View

In most browsers, a valid XML document with no renderer-recognized namespaces present anywhere in the markup will be shown as an interactive, pretty-printed representation of the document tree, as shown in Figure 7-1. This mode is not particularly useful to end users, but it can aid debugging.

That said, when any of the namespaces in the document is known to the browser (even when the top-level one is not recognized at all!), the document will be rendered differently: All recognized markup will work as intended, all unsupported tags will simply have no effect, and any text between them will be shown as is.

To illustrate this rendering strategy, consider the following input:

```
<foo xmlns="http://www.example.com/nonexistent">
  <u>Hello</u>
  <html xmlns="http://www.w3.org/1999/xhtml">
    <u>world!</u>
  </html>
</foo>
```

The above example will be rendered as “Hello world!” The first *<u>* tag, with no semantics-defining namespace associated with it, will have no visible effect. The second one will be understood as an XHTML tag that triggers underlining.

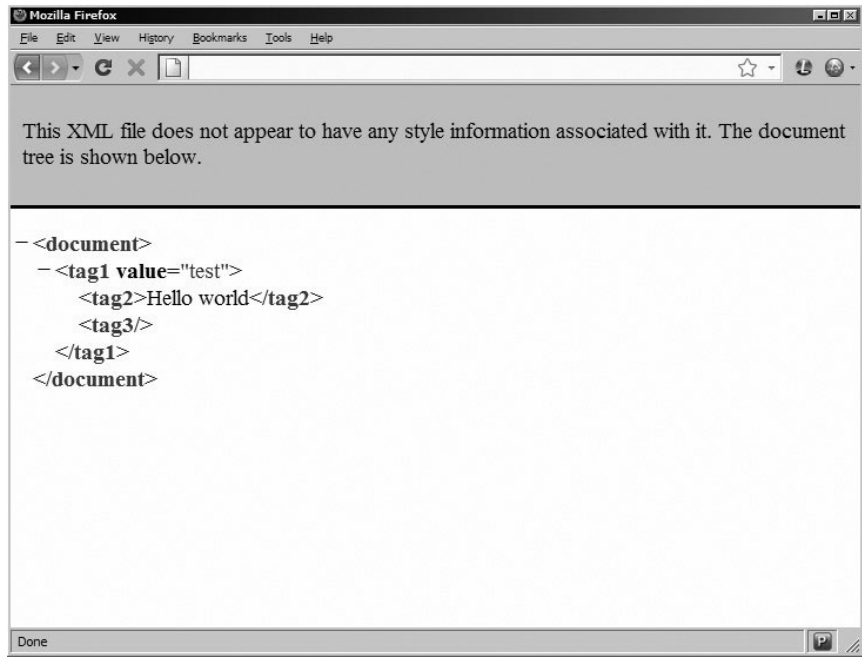


Figure 7-1: Firefox displaying an XML document with no recognized namespaces

The consequences of this fault-tolerant approach to the rendering of unknown XML documents and unrecognized namespaces are subtle but fairly important. For example, it will not be safe to proxy an unsanitized RSS feed, even though this format is typically routed to a specialized renderer and thus not subject to XSS risks. Any browser with no built-in RSS reader may fall back to generic rendering and then find HTML buried deep inside the feed.

Scalable Vector Graphics

Scalable Vector Graphics (SVG)¹ is a quickly evolving, XML-based vector graphics format. First published in 2001 by W3C, it is noteworthy for its integrated animation capabilities and direct JavaScript scripting features. The following example of a vector image draws a circle and displays a message when this circle is clicked:

```
<svg xmlns="http://www.w3.org/2000/svg">
  <script><![CDATA[
    function clicked() { alert("Hi mom!"); }
  ]]></script>
  <circle onclick="clicked()" cx="50" cy="50"
    r="50" fill="pink" />
</svg>
```

The SVG file format is recognized all modern browsers except for Internet Explorer prior to 9, and it is handled by the general-purpose XML renderer. SVG images can be embedded into XHTML with an appropriate *xmlns* directive or inlined in non-XML HTML5 documents using a pre-defined `<svg>` tag.

Interestingly, in several browsers the format can also be placed in a standalone XML document and then viewed directly, or it can be loaded on third-party pages via the `<img>` markup. While it is safe to load SVG images via `<img>` (scripting should be disabled in this scenario), it is fairly dangerous to host user-supplied SVG data because in cases of direct navigation, all embedded scripts will execute in the context of the hosting domain. This unexpected problem means that serving any externally originating SVG images will require very careful syntax sanitization to eliminate non-SVG *xmlns* content from the XML container and to permit only certain types of markup in the remainder of the document.

NOTE *The Content-Disposition header on the relevant HTTP responses is a potential workaround that permits SVG to be included via `<img>` but not accessed directly. This approach is not perfect, but it limits the risk. Using a throwaway domain to host such images is another possibility.*

Mathematical Markup Language

Mathematical Markup Language (MathML)² is a fairly straightforward means to facilitate the semantic, if a bit verbose, representation of mathematical equations. The standard was originally proposed by the W3C in 1998, and it has been substantially refined through the years. Because of its somewhat niche application, MathML needed over a decade to gain partial support in Opera and Firefox browsers, but it is slowly gaining acceptance today. In the browsers that support the language, it may be placed in a standalone file or inline in XHTML and HTML5 documents.

Unlike SVG, MathML has no additional security considerations beyond those associated with generically handled XML.

XML User Interface Language

The XML User Interface Language (XUL)³ is a presentation markup language created by Mozilla specifically for building browser-based applications, rather than documents. XUL exists because although modern HTML is often powerful enough to build basic graphical user interfaces, it is not particularly convenient for certain specialized tasks that desktop applications excel in, such as implementing common dialog windows or system menus.

XUL is not currently supported by any browser other than Firefox and appears to be disabled in the recent release, Firefox 6. In Firefox, it is handled by the general-purpose renderer, based on the appropriate *xmlns* namespace. Firefox uses XUL for much of its internal UI, but otherwise the language is seldom encountered on the Internet.

From the standpoint of web application security, Internet-originating XUL documents can be considered roughly equivalent to HTML documents.

Essentially, the language has JavaScript scripting capabilities and allows broad control over the appearance of the rendered page. Other than that property, it has no unusual quirks.

Wireless Markup Language

Wireless Markup Language (WML)⁴ is a largely obsolete “optimized” HTML syntax developed in the 1990s by a consortium of mobile handset manufacturers and cellular network operators. This XML-based language, a part of the Wireless Application Protocol suite (WAP), offered a simplified weblike browsing experience for pre-smartphone devices with limited bandwidth and CPU resources.* A simple WML page might have looked like this:

```
<wml>
  <card title="Hello world!">
    <a href="elsewhere.wml">Click here!</a>
  </card>
</wml>
```

Because WAP services needed to be engineered independently of normal HTML content and had to deal with closed and underspecified client architectures and other carrier-imposed restrictions, WML never became as popular as its proponents hoped. In almost all developed markets, WML has been displaced by fast, Internet-enabled smartphones with fully featured HTML browsers. Nevertheless, the legacy of the language lives on, and it is still routed to specialized renderers in Opera and in Internet Explorer Mobile.

In the browsers that support the format, it is often possible to use WML-based scripts. There are two methods to achieve this. The canonical way is to use WMLScript (WMLS), a JavaScript-derived execution environment that depends on stand-alone script files, coupled with an extremely inconsiderate abuse of fragment IDs for an equivalent of possibly attacker-controlled *eval(...)* statements:

```
<a href="scriptfile.wmls#some_function()">Click here!</a>
```

The other method of executing scripts, available in more featured browsers, is to simply embed normal *javascript:* URLs or insert `<script>` blocks into the WML file.

RSS and Atom Feeds

Feeds are a standardized way for clients to periodically poll sites of interest to users (such as their favorite blogs) for machine-readable updates to said sites' content. Really Simple Syndication (RSS)⁵ and Atom⁶ are two superficially similar but fiercely competing XML-based feed formats. The first (RSS) is popular; the second (Atom) is said to be good.

*Astute readers will note that XML is not a particularly good way to conserve bandwidth or CPU resources. To that effect, the WAP suite provides an alternative, binary-only serialization of XML, known as WBXML.

Built-in, specialized RSS and Atom renderers are available in Firefox, Safari, and Opera. The determination to route an XML document to these modules is based on simple, browser-specific heuristics, such as the top-level tag being named `<rss>` or `<feed>` (and not having any conflicting *xmlns* directives). In Firefox, RSS parsing may kick in even if *Content-Type* is *image/svg+xml* or *text/html*. Safari will happily recognize feeds in even more unrelated MIME types.

One interesting feature of both feed formats is that they permit a subset of HTML, including CSS, to be embedded in a document in a rather peculiar, indirect way: as an entity-escaped text. Here is an example of this syntax:

```
<rss>
...
<description type="html">
  &lt;u&gt; Underlined text! &lt;/u&gt;
</description>
...
</rss>
```

The subset of HTML permitted in RSS and Atom feeds is not well defined, and some feed renderers have previously permitted direct scripting or navigation to potentially dangerous pseudo-URLs. Perhaps more importantly, however, any browser that does not have built-in feed previews may render the file using the generic XML parsing approach; if such feeds are not sanitized carefully, script execution will ensue.

A Note on Nonrenderable File Types

For the sake of completeness, it should be noted that all modern browsers support a number of specialized file formats that remain completely opaque to the renderer or to the web application layer but that are nevertheless recognized by a variety of in-browser subsystems.

A detailed investigation of these formats is beyond the scope of this book, but some notable examples include plug-in and extension installation manifests, automatic HTTP proxy autoconfiguration files (PAC), installable visual skins, Certificate Revocation Lists (CRLs), antimalware site blacklists, and downloadable TrueType and OpenType fonts.

The security properties of these mechanisms should be studied individually before deciding to allow any of these formats to be served to the user. Save for the generic content-hosting considerations outlined in Chapter 13, they are unlikely to harm the hosting web application directly, but they may cause problems for users.

Security Engineering Cheat Sheet

When Hosting XML-Based Document Formats

Assume that the payload may be interpreted as XHTML or some other script-enabled document type, regardless of the *Content-Type* and the top-level *xmlns* directive. Do not allow unconstrained attacker-controlled markup anywhere inside the file. Use the *Content-Disposition: attachment* if data is not meant to be viewed directly; `<img>` and feeds will still work.

On All Non-HTML Document Types

Use correct, browser-recognized *Content-Type* and *charset* values. Specify the *Content-Disposition: attachment* where possible. Verify and constrain output syntax. Consult the cheat sheet in Chapter 13 to avoid security problems related to content-sniffing flaws.

8

CONTENT RENDERING WITH BROWSER PLUG-INS

Browser plug-ins come in many forms and shapes, but the most common variety give the ability to display new file formats in the browser, as if they were HTML. The browser simply hands over the retrieved file, provides the helper application with a rectangular drawing surface in the document window, and essentially backs away from the scene. Such content-rendering plug-ins are clearly distinguished from browser extensions, a far more numerous bunch that commonly relies on JavaScript code to tweak how the already-supported, in-browser content is presented to the user.

Browser plug-ins have a long and colorful history of security flaws. In fact, according to some analysts, 12 out of the 15 most frequently exploited client-side vulnerabilities in 2010 could be attributed to the quality of plug-in software.¹ Many of these problems are because the underlying parsers were originally not meant to handle malicious inputs gracefully and have not benefited from the intense scrutiny that the remainder of the Web has been subject to. Other problems stem from the unusual security models devised by

plug-in developers and the interference between these permissions, the traditional design of web browsers, and the commonsense expectations of application developers.

We will review some of the security mechanisms used by popular plug-ins in the next chapter of this book. Before taking this dive, it makes sense to look at the ways plug-ins integrate with other online content and the common functionality they offer.

Invoking a Plug-in

Content-rendering plug-ins can be activated in a couple of ways. The most popular explicit method is to use `<embed src=...>` or `<object data=...>` markup in a “host” HTML document, with the `src` or `data` parameter pointing to the URL from which the actual plug-in-recognized document is to be retrieved. The dimensions and position of the drawable area allocated for the plug-in can be controlled with CSS (or with legacy HTML parameters).

In this scenario, every `<embed>` or `<object>` tag should be accompanied by an additional `type` parameter. The MIME type specified there will be compared to the list of MIME types registered by all the active plug-ins, and the retrieved file will be routed to the appropriate handler. If no match is found, a warning asking the user to download a plug-in should be theoretically displayed instead, although most browsers look at other signals before resorting to this unthinkable possibility; examining *Content-Type* or the apparent file extension spotted in the URL are two common choices.

NOTE *An obsolete `<applet>` tag, used to load Java programs (roughly equivalent to `<object type="application/x-java-applet">`), works in a comparable way but unconditionally disregards these auxiliary signals.*

Additional input to the plug-in is commonly passed using `<param>` tags nested inside the `<object>` block or through nonstandard additional parameters attached to the `<embed>` markup itself. The former, more modern approach may look like this:

```
<object data="app.swf" type="application/x-shockwave-flash">
  <param name="some_param1" value="some_value1">
  <param name="some_param2" value="some_value2">
  ...
</object>
```

In this content-inclusion mode, the *Content-Type* header returned by the server when retrieving the subresource is typically ignored, unless the `type` parameter is unknown to the browser. This is an unfortunate design, for reasons that will be explained shortly.

The other method for displaying plug-in content involves navigating directly to a suitable file. In this case, and in the case of `<embed>` or `<object>` with a missing `type` parameter, the *Content-Type* value obtained from the server is honored, and it will be compared with the list of plug-in-recognized MIME

types. If a match is found, the content is routed to the appropriate component. If the *Content-Type* lookup fails or the header is missing, some browsers will examine the response body for known content signatures; others just give up.

NOTE *The aforementioned content-focused methods aside, several types of plug-ins can be loaded directly from within JavaScript or VBScript programs without the need to explicitly create any HTML markup or retrieve any external data. Such is the case for ActiveX, an infamous script-to-system integration bridge available in Internet Explorer. (We will devote some time to ActiveX later in this chapter, but first things first.)*

The Perils of Plug-in Content-Type Handling

As noted in the previous section, in certain scenarios the *Content-Type* parameter on a retrieved plug-in-handled file is ignored, and the *type* parameter in the corresponding markup on the embedding page is used instead. While this decision is somewhat similar to the behavior of other type-specific content-inclusion tags (say, `<img>`), as discussed in “Type-Specific Content Inclusion” on page 82, it has some unique and ultimately disastrous consequences in the plug-in world.

The big problem is that several types of plug-ins are essentially full-fledged code execution environments and give the executed applications (*applets*) a range of special privileges to interact with the originating domain. For example, a Flash file retrieved from *fuzzybunnies.com* would be granted access to its originating domain (complete with a user’s cookies) when embedded on the decidedly rogue *bunnyoutlet.com*.

In such a scenario, it would seem to be important for *fuzzybunnies.com* to be able to clearly communicate that a particular type of a document is indeed meant to be interpreted by a plug-in—and, consequently, that some documents aren’t meant to be used this way. Unfortunately, there is no way for this to happen: The handling of a retrieved file is fully controlled by the embedding site (in our example, by the mean-spirited bullies who own *bunnyoutlet.com*). Therefore, if the originating domain hosts any type of user-controlled content, even in a nominally harmless format (such as *text/plain* or *image/jpeg*), the owners of *bunnyoutlet.com* may instruct the browser to disregard the existing metadata and route that document to a plug-in of their choice. A simple markup to achieve this sinister goal may be

```
<object data="http://fuzzybunnies.com/avatars/user11630.jpg"
type="application/x-shockwave-flash">
```

If this turn of events seems wrong, that’s because it is. Security researchers have repeatedly demonstrated that it is quite easy to construct documents that are, for example, simultaneously a valid image and a valid plug-in-recognized executable. The well-known “GIFAR” vulnerability, discovered in 2008 by Billy Rios,² exploited that very trick: It smuggled a Java applet inside a perfectly kosher GIF image. In response, Sun Microsystems reportedly tightened down the Java JAR file parser to mitigate the risk, but the general threat of such mistakes is still very real and will likely rear its ugly head once more.

Interestingly, the decision by some developers to rely on *Content-Type* and other signals if the *type* parameter is unrecognized is almost as bad. This decision makes it impossible for the well-intentioned *fuzzybunnies.com* to safely embed a harmless video from the rogues at *bunnyoutlet.com* by simply specifying *type="video/x-ms-wmv"*, because if any of the visitors do not have a plug-in for that specific media type, *bunnyoutlet.com* will suddenly have a say in what type of plug-in should be loaded on the embedding site instead. Some browsers, such as Internet Explorer, Chrome, or Opera, may also resort to looking for apparent file extensions present in the URL, which can lead to an interesting situation where neither the embedding nor the hosting party has real control over how a document is displayed—and quite often only the attacker is in charge.

A much safer design would require the embedder-controlled *type* parameter and the host-controlled *Content-Type* header to match (at least superficially). Unfortunately, there is currently no way to make this happen. Several individual plug-ins try to play nice (for example, following a 2008 overhaul, Adobe Flash rejects applets served with *Content-Disposition: attachment*, as does the built-in PDF reader in Chrome), but these improvements are few and far between.

Document Rendering Helpers

A significant portion of the plug-in landscape belongs to programs that allow certain very traditional, “nonweb” document formats to be shown directly in the browser. Some of these programs are genuinely useful: Windows Media Player, RealNetworks RealPlayer, and Apple QuickTime have been the backbone of online multimedia playback for about a decade, at least until their displacement by Adobe Flash. The merits of others are more questionable, however. For example, Adobe Reader and Microsoft Office both install in-browser document viewers, increasing the user’s attack surface appreciably, though it is unclear whether these viewers offer a real benefit over opening the same document in a separate application with one extra click.

Of course, in a perfect world, hosting or embedding a PDF or a Word document should have no direct consequences for the security of the participating websites. Yet, predictably, the reality begs to differ. In 2009, a researcher noted that PDF-based forms that submit to *javascript:* URLs can apparently lead to client-side code execution on the embedding site.³ Perhaps even more troubling than this report alone, according to that researcher’s account, Adobe initially dismissed the report with the following note: “Our position is that, like an HTML page, a PDF file is active content.”

It is regrettable that the hosting party does not have full control of when this active content is detected and executed and that otherwise reasonable webmasters may think of PDFs or Word documents as just a fancy way to present text. In reality, despite their harmless appearance, in a bid to look cool, many such document formats come equipped with their own hyperlinking capabilities or even scripting languages. For example, JavaScript code can be embedded in PDF documents, and Visual Basic macros are possible in

Microsoft Office files. When a script-bearing document is displayed on an HTML page, some form of a programmatic plug-in-to-browser bridge usually permits a degree of interaction with the embedding site, and the design of such bridges can vary from vaguely questionable to outright preposterous.

In one 2007 case, Petko D. Petkov noticed that a site that hosts any PDF documents can be attacked simply by providing completely arbitrary JavaScript code in the fragment identifier. This string will be executed on the hosting page through the plug-in bridge:⁴

```
http://example.com/random_document.pdf#foo=javascript:alert(1)
```

The two vulnerabilities outlined here are now fixed, but the lesson is that special care should be exercised when hosting or embedding any user-supplied documents in sensitive domains. The consequences of doing so are not well documented and can be difficult to predict.

Plug-in-Based Application Frameworks

The boring job of rendering documents is a well-established role for browser plug-ins, but several ambitious vendors go well beyond this paradigm. The aim of some plug-ins is simply to displace HTML and JavaScript by providing alternative, more featured platforms for building interactive web applications. That reasoning is not completely without merit: Browsers have long lacked in performance, in graphics capabilities, and in multimedia codecs, stifling some potential uses of the Web. Reliance on plug-ins is a reasonable short-term way to make a difference. On the flip side, when proprietary, patent- and copyright-encumbered plug-ins are promoted as the ultimate way to build an online ecosystem, without any intent to improve the browsers themselves, the openness of the Web inevitably suffers. Some critics, notably Steve Jobs, think that creating a tightly controlled ecosystem is exactly what several plug-in vendors, most notably Adobe, aspire to.⁵

In response to this perceived threat of a hostile takeover of the Web, many of the shortcomings that led to the proliferation of alternative application frameworks are now being hastily addressed under the vaguely defined umbrella of HTML5; *<video>* tags and WebGL* are the prime examples of this work. That said, some of the features available in plug-ins will probably not be captured as a part of any browser standard in the immediate future. For example, there is currently no serious plan to add inherently dangerous elevated privilege programs supported by Java or security-by-obscurity content protection schemes (euphemistically called Digital Rights Management, or DRM).

Therefore, while the landscape will change dramatically in the coming years, we can expect that in one form or another, proprietary web application frameworks are here to stay.

* WebGL is a fairly recent attempt to bring OpenGL-based 3D graphics to JavaScript applications. The first specification of the standard appeared in March 2011, and wide browser-level support is expected to follow.

Adobe Flash

Adobe Flash is a web application framework introduced in 1996, in the heat of the First Browser Wars. Before its acquisition by Adobe in 2005, the Flash platform was known as Macromedia Flash or Shockwave Flash (hence the *.swf* file extension used for Flash files), and it is still sometimes referred to as such.

Flash is a fairly down-to-earth platform built on top of a JavaScript-based language dubbed ActionScript.⁷ It includes a 2-D vector and bitmap graphics-rendering engine and built-in support for several image, video, and audio formats, such as the popular and efficient H.264 codec (which is used for much of today's online multimedia).

By most estimates, Flash is installed on around 95 to 99 percent of all desktop systems.^{8,9} This user base is substantially higher than that of any other media player plug-in. (Support for the Windows Media Player and QuickTime plug-ins is available on only about 60 percent of PCs, despite aggressive bundling strategies, while the increasingly unpopular RealPlayer is still clinging to 25 percent.) The market position contributes to the product's most significant and unexpected use: the replacement of all multimedia playback plug-ins previously relied upon for streaming video on the Web. Although the plug-in is also used for a variety of other jobs (including implementing online games, interactive advertisements, and so on), simple multimedia constitutes a disproportionately large slice of the pie.

NOTE *Confusingly, a separate plug-in called Adobe Shockwave Player (without the word “Flash”) is also available, which can be used to play back content created with Adobe Director. This plug-in is sometimes mistakenly installed in place of or alongside Adobe Flash, contributing to an approximately 20 percent install base,⁶ but it is almost always unnecessary. The security properties of this plug-in are not particularly well studied.*

Properties of ActionScript

The capabilities of ActionScript in SWF files are generally analogous to those of JavaScript code embedded on HTML pages with some minor, yet interesting, differences. For example, Flash programs are free to enumerate all fonts installed on a system and collect other useful system fingerprinting signals not available to normal scripts. Flash programs can also use full screen rendering, facilitating UI spoofing attacks, and they can request access to input devices such as a camera or a microphone (this requires the user's consent). Flash also tends to ignore browser security and privacy settings and uses its own configuration for mechanisms such as in-plug-in persistent data storage (although some improvements in this area were announced in May 2011).

The remaining features are less surprising. We'll discuss the network and DOM access permissions of Flash applications in more detail in the next chapter, but in short, by default, every Flash applet can use the browser HTTP stack (and any ambient credentials managed therein) to talk back to its originating server, request a limited range of subresources from other sites, and navigate the current browser window or open a new one. ActionScript programs may also negotiate browser-level access to other currently running

Flash applications and, in some cases, access the DOM of the embedding page. This last functionality is implemented by injecting *eval(...)*-like statements into the target JavaScript context.

ActionScript offers fertile ground for web application vulnerabilities. For example, the *getURL(...)* and *navigateToURL(...)* functions, used to navigate the browser or open new windows, are sometimes invoked with attacker-controlled inputs. Such a use is dangerous. Even though *javascript:* URLs do not have a special meaning to Flash, the function will pass such strings to the browser, in some cases resulting in script injection on the embedding site.

Until recently, a related problem was present with other URL-handling APIs, such as *loadMovie(...)*. Even though the function did not rely on the browser to load the document, it would recognize an internal *asfunction:* scheme, which works similarly to *eval(...)* and could be trivially leveraged to perform a call to *getURL(...)*:

```
asfunction:getURL,javascript:alert('Hi mom!')
```

The issue with loading scripts from untrusted sources, discussed in Chapter 6, also has an equivalent in the plug-in world. In Flash, it is very unsafe to invoke certain functions that affect the state of the ActionScript execution environment (such as the *LoadVars.load(...)*) with attacker-controlled URLs, even if the scheme from which the resource is loaded is *http:* or *https:*.

Another commonly overlooked attack surface is the internal, simplified HTML parser offered by the Flash plug-in: Basic HTML markup can be assigned to properties such as *TextField.htmlText* and *TextArea.htmlText*. It is easy to forget that user-supplied content must be escaped correctly in this setting. Failure to do so may permit attackers to modify the appearance of the application UI or to inject potentially problematic scripting-oriented links.

Yet another class of Flash-related security bugs may arise due to design or implementation problems in the plug-in itself. For example, take the *ExternalInterface.call(...)* API. It is meant to allow ActionScript to call existing JavaScript functions on the embedding page and takes two parameters: the name of the JavaScript function to call and an optional string to be passed to this routine. While it is understood that the first parameter should not be attacker controlled, it appears to be safe to put user data in the second one. In fact, the documentation provides the following code snippet outlining this specific use case:¹⁰

```
ExternalInterface.call("sendToJavaScript", input.text);
```

This call will result in the following *eval(...)* statement being injected on the embedding page:

```
try {  
  __flash__toXML(sendToJavaScript, "value of input.text");  
} catch (e) {  
  "<undefined/>";  
}
```

When writing the code behind this call, the authors of the plug-in remembered to use backslash escaping when outputting the second parameter: `hello"world` becomes `hello\"world`. Unfortunately, they overlooked the need to escape any stray backslash characters, too. Because of this, if the value of `input.text` is set to the following string, the embedded script will unexpectedly execute:

```
Hello world!\"+alert(1)); } catch(e) {} //
```

I contacted Adobe about this particular problem in March 2010. Over a year later, its response was this: “We have not made any change to this behavior for backwards compatibility reasons.”

That seems unfortunate.

Microsoft Silverlight

Microsoft Silverlight is a versatile development platform built on the Windows Presentation Foundation, a GUI framework that is a part of Microsoft’s .NET stack. It debuted in 2007 and combines an Extensible Application Markup Language (XAML)¹¹ (Microsoft’s alternative to Mozilla’s XUL) with code written in one of several managed .NET languages,* such as C# or Visual Basic.

Despite substantial design differences and a more ambitious (and confusing) architecture, this plug-in is primarily meant to compete with Adobe Flash. Many of the features available to Silverlight applications mirror those implemented in its competitor, including a nearly identical security model and a similar `eval(...)`-based bridge to the embedding page. To Microsoft’s credit, Silverlight does not come with an equivalent of the `asfunction:` scheme or with a built-in HTML renderer, however.

Silverlight is marketed by Microsoft fairly aggressively, and it is bundled with some editions of Internet Explorer. As a result, depending on the source, it is believed to have about a 60 to 75 percent desktop penetration.¹² Despite its prevalence, Silverlight is used fairly infrequently to develop actual web applications, perhaps because it usually offers no compelling advantages over its more established counterpart or because its architecture is seen as more contrived and platform-specific. (Netflix, a popular video streaming and rental service, is one of the very few high-profile websites that actually relies on Silverlight for playback on some devices.)

Sun Java

Java is a programming language coupled with a platform-independent, managed-code execution platform. Developed in the early to mid-1990s by James Gosling for Sun Microsystems, Java has a well-established role as a server-side programming language and a very robust presence in many other niches,

* Managed code is not executed directly by the CPU (which would be inherently unsafe, because CPUs are not designed to enforce web security rules). Rather, it is compiled to an intermediate binary form and then interpreted at runtime by a specialized virtual machine. This approach is faster than interpreting scripts at runtime and permits custom security policy enforcement as the program is being executed.

including mobile devices. Yet, from the beginning, Sun hoped that Java would also occupy a prominent place on the browser end.

Java in the browser predated Flash and most similar plug-ins, and the now-obsolete `<applet>` tag is a testament to how important and unique and novel this addition must have seemed back in its day. Yet, despite this head start, the Java language is nearly extinct as an in-browser development platform, and even in its heyday it never enjoyed real prominence. It retains a remarkable 80 percent installed base, but this high percentage is attributed largely to the fact that the Java plug-in is bundled with Java Runtime Environment (JRE), a more practically useful and commonly preinstalled component that is required to run normal, desktop Java applications on the system without any involvement on the browser end.

The reasons for the failure of Java as a browser technology are difficult to pinpoint. Perhaps it's due to the plug-in's poor startup performance, the clunky UI libraries that made it difficult to develop snappy and user-friendly web applications, or the history of vicious litigation between Sun and Microsoft that cast a long shadow over the future of the language on Microsoft's operating systems.* Whatever the reasons may be, the high install base of Java coupled with its marginal use means that the risks it creates far outweigh any potential benefits to the users. (The plug-in had close to 80 security vulnerabilities in 2010,¹³ and the vendor is commonly criticized for patching such bugs very slowly.)

Java's security policies are somewhat similar to those of other plug-ins, but in some aspects, such as its understanding of the same-origin policy or its ability to restrict access to the embedding page, it compares unfavorably. (The next chapter provides an overview of this.) It is also worth noting that unlike with Flash or Silverlight, certain types of cryptographically signed applets may request access to potentially dangerous OS features, such as unconstrained networking or file access, and only a user's easily coaxed consent stands in the way.

XML Browser Applications (XBAP)

XML Browser Applications (XBAP)¹⁴ is Microsoft's heavy-handed foray into the world of web application frameworks, attempted in the years during which the battle over Java started going sour and before the company released Silverlight.

XBAP is reminiscent of Silverlight in that it leverages the same Windows Presentation Foundation and .NET architecture. However, instead of being a self-contained and snappy browser plug-in, it depends on the large and unwieldy .NET runtime, in a manner similar to the Java plug-in's dependence on JRE. It executes the managed code in a separate process called *PresentationHost.exe*, often loading extensive dependencies at initialization time. By Microsoft's own admission, the load time of a medium-size previously uncached application

* The legal battles started in 1997, when Microsoft decided to roll out its own (and in some ways, superior) version of the Java virtual machine. Sun Microsystems sued, hoping to win an injunction that would force Microsoft to bundle Sun's version instead. The two companies initially settled in 2001, but shortly thereafter they headed back to court. In the final settlement in 2004, Sun walked away with \$1.6 billion in cash, but Windows users were not getting any Java runtime at all.

could easily reach 10 seconds or more. When the technology premiered in 2002, most users were already expecting Internet applications to be far more responsive than that.

The security model of XBAP applications is poorly documented and has not been researched to date, perhaps due to XBAP's negligible real-world use and obtuse, multilayer architecture. One would reasonably expect that XBAP's security properties would parallel the model eventually embraced for Silverlight, but with broader access to certain .NET libraries and UI widgets. And, apparently as a result of copying from Sun, XBAP programs can also be given elevated privileges when loaded from the local filesystem or signed with a cryptographic certificate.

Microsoft bundled XBAP plug-ins with its .NET framework to the point of silently installing nonremovable Windows Presentation Foundation plug-ins—not only in Internet Explorer but also in the competing Firefox and Chrome. This move stirred some well-deserved controversy, especially once the first vulnerability reports started pouring in. (Mozilla even temporarily disabled the plug-in through an automated update to protect its users.) Still, despite such bold and questionable moves to popularize it, nobody actually wanted to write XBAP applets, and inch by inch, the technology followed Java into the dustbin of history.

Eventually, Microsoft appeared to acknowledge this failure and chose to focus on Silverlight instead. Beginning with Internet Explorer 9, XBAP is disabled by default for Internet-originating content, and the dubious Firefox and Chrome plug-ins are no longer automatically pushed to users. Nevertheless, it seems reasonable to assume that at least 10 percent of all Internet users may be still browsing with a complex, partly abandoned, and largely unnecessary plug-in installed on their machines and will continue to do so for the next couple of years.

ActiveX Controls

At its core, ActiveX is the successor to Object Linking and Embedding (OLE), a 1990 technology that made it possible for programs to reuse components of other applications in a standardized, language-independent way. A simple use case for ActiveX would be a spreadsheet application wishing to embed an editable vector image from a graphics-editing program or a simple game that wants to embed a video player.

The idea is not controversial, but by the mid-1990s Microsoft had decided that ActiveX made sense in the browser, too. After all, wouldn't websites want to benefit from the same Windows components that desktop applications could rely on? The approach violates the idea of nurturing an open, OS-independent web, but it's otherwise impressive, as illustrated by the following JavaScript example that casually creates, edits, and saves an Excel spreadsheet:

```
var sheet = new ActiveXObject("Excel.Sheet");
sheet.ActiveSheet.Cells(42,42).Value = "Hi mom!";
sheet.SaveAs("c:\\spreadsheet.xls");
sheet.Application.Quit();
```

Standards compliance aside, Microsoft's move to ActiveX proved disastrous from a security standpoint. Many of the exposed ActiveX components were completely unprepared to behave properly when interacting with untrusted environments, and over the next 15 years, researchers discovered several hundred significant security vulnerabilities in web-accessible ActiveX controls. Heck, the simple observation that Firefox does not support this technology helped bolster its security image at the onset of the Second Browser Wars.

Despite this fiasco, Microsoft stood by ActiveX defiantly, investing in gradually limiting the number of controls that could be accessed from the Internet and fixing the bugs in those it considered essential. Not until Internet Explorer 9 did Microsoft finally decide to let go: Internet Explorer 9 disables all ActiveX access by default, requiring several extra clicks to use it when needed.

NOTE *The wisdom of delegating the choice to the user is unclear, especially since the permission granted to a site extends not only to legitimate content on that website but also to any payloads injected due to application bugs such as XSS. Still, Internet Explorer 9 is some improvement.*

Living with Other Plug-ins

So far, we have covered almost all general-purpose browser plug-ins in use today. Although there is a long tail of specialized or experimental plug-ins, their use is fairly insignificant and not something that we need to take into account when surveying the overall health of the online ecosystem.

Well, with one exception. An unspecified but probably significant percentage of online users can be expected to have an assortment of web-exposed browser plug-ins or ActiveX controls that they never knowingly installed, or that they were forced to install even though it's doubtful that they would ever benefit from the introduced functionality.

This inexcusable practice is sometimes embraced by otherwise reputable and trusted companies. For example, Adobe forces users who wish to download Adobe Flash to also install GetRight, a completely unnecessary third-party download utility. Microsoft does the same with Akamai Download Manager on its developer-oriented website, complete with a hilarious justification (emphasis mine):¹⁵

What is the Akamai Download Manager and why do I *have* to use it?

To help you download large files with reduced chance of interruption, *some downloads require* the use of the Akamai Download Manager.

The primary concern with software installed this way and exposed directly to malicious input from anywhere on the Internet is that unless it is designed with extreme care, it is likely to have vulnerabilities (and sure enough, both GetRight and Akamai Download Manager had some). Therefore, the risks of browsing with a completely unnecessary plug-in that only served a particular purpose once or twice far outweigh the purported (and usually unwanted) benefits.

Security Engineering Cheat Sheet

When Serving Plug-in-Handled Files

- ☑ **Data from trusted sources:** Data from trusted sources is generally safe to host, but remember that security vulnerabilities in Flash, Java, or Silverlight applets, or in the Adobe Reader JavaScript engine, may impact the security of your domain. Avoid processing user-supplied URLs and generating or modifying user-controlled HTML from within plug-in-executed applets. Exercise caution when using the JavaScript bridge.
- ☑ **User-controlled simple multimedia:** User-controlled multimedia is relatively safe to host, but be sure to validate and constrain the format, use the correct *Content-Type*, and consult the cheat sheet in Chapter 13 to avoid security problems caused by content-sniffing flaws.
- ☑ **User-controlled document formats:** These are not inherently unsafe, but they have an increased risk of contributing security problems due to plug-in design flaws. Consider hosting from a dedicated domain when possible. If you need to authenticate the request to an isolated domain, do so with a single-use request token instead of by relying on cookies.
- ☑ **User-controlled active applications:** These are unsafe to host in sensitive domains.

When Embedding Plug-in-Handled Files

Always make sure that plug-in content on HTTPS sites is also loaded over HTTPS,^{*} and always explicitly specify the *type* parameter on `<object>` or `<embed>`. Note that because of the non-authoritative handling of *type* parameters, restraint must be exercised when embedding plug-in content from untrusted sources, especially on highly sensitive sites.

- ☑ **Simple multimedia:** It is generally safe to load simple multimedia from third-party sources, with the caveats outlined above.
- ☑ **Document formats:** These are usually safe, but they carry a greater potential for plug-in and browser content-handling issues than simple multimedia. Exercise caution.
- ☑ **Flash and Silverlight:** In principle, Flash and Silverlight apps can be embedded safely from external sources if the appropriate security flags are present in the markup. If the flags are not specified correctly, you may end up tying the security of your site to that of the provider of the content. Consult the cheat sheet in Chapter 9 for advice.
- ☑ **Java:** Java always ties the security of your service to that of the provider of the content, because DOM access to the embedding page can't be reliably restricted. See Chapter 9. Do not load Java apps from untrusted sites.

If You Want to Write a New Browser Plug-in or ActiveX Component

Unless you are addressing an important, common-use case that will benefit a significant fraction of the Internet, please reconsider. If you are scratching an important itch, consider doing it in a peer-reviewed, standardized manner as a part of HTML5.

^{*} If loading an HTTP-delivered applet on an HTTPS page is absolutely unavoidable, it is safer to place it inside an intermediate HTTP frame rather than directly inside the HTTPS document, as this prevents the applet-to-JavaScript bridge from being leveraged for attacks.